

TEHNICI WEB

INTRO

Claudia Chiriță . 2024/2025

ADMIN INFO

CE?

- tehnologii pentru dezvoltarea de aplicații web client
- HTML + CSS + JavaScript
- DOM, AJAX, Canvas & SVG
- misc: frameworkuri, biblioteci, unelte

CUM?

- curs + laborator
- canal Teams
- office hours

<https://cs.unibuc.ro/~cechirita/tw>

NOTE?

```
notă-curs = nlab + nexa
```

```
nlab = notă-laborator // <= 40 puncte
```

```
nexa = notă-examen-pe-calculator // <= 60 puncte
```

```
nlab = nlab1 + nlab2
```

```
nlab1 = notă-activitate-laborator // <= 10 puncte
```

```
nlab2 = notă-proiect // <= 30 de puncte
```

```
nlab >= 20 & nexa >= 30 // condiția de promovare
```

PROIECT

- HTML + CSS + JavaScript
- 2 etape
- prima etapă: HTML + CSS
- cerințele detaliate pe [github](#)

RESURSE ONLINE

<https://developer.mozilla.org/>

<https://www.w3schools.com/>

<https://www.w3.org/standards>

ÎNTREBĂRI?

HTTP://WWWWWWWWWWWWWWWWW

INTERNET

“interconnection of computer networks”

“network of networks”

sistem global de rețele de calculatoare interconectate
care folosește suita de **protocoale TCP/IP**

PROTOCOL

specificație tehnică

mulțime de reguli și convenții care facilitează și controlează schimbul de date între dispozitive diferite

TCP/IP (INTERNET PROTOCOL SUITE)

stivă de protocoale cu 4 straturi:

- Link layer: MAC
- Internet layer: IP (v4, v6)
- Transport layer: TCP, UDP
- Application layer: DNS, FTP, SSH, TLS/SSL, HTTP

TCP/IP (INTERNET PROTOCOL SUITE)

principalele protocoale:

- Transmission Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Internet Protocol (IP)

un echipament conectat la Internet este identificat printr-o adresă IP (etichetă numerică)

INTERNET

“interconnection of computer networks”

“network of networks”

oferă servicii: www, email, telefonie, file sharing

WORLD WIDE WEB

“ The Web is an abstract (imaginary) space of information.

On the Net, you find computers – on the Web, you find document, sounds, videos, ... information.

On the Net, the connections are cables between computers; on the Web, connections are hypertext links.”

Tim Berners-Lee

WORLD WIDE WEB

un sistem de documente hypertext interconectate,
care poate fi accesat în Internet

WORLD WIDE WEB

The World Wide Web Consortium (W3C)

*“international community that develops open standards
to ensure the long-term growth of the Web”*

World Wide Web Foundation

*“advance the open web as a public good and a basic
right”*

HYPertext

text (afișat pe un dispozitiv electronic) care conține referințe (*hyperlinks*) la date ce pot fi accesate imediat

- *hyper* - folosit cu sensul de extensie, generalizare din gr. ὑπέρ (peste, mai departe)
- depășirea constrângerilor liniare ale textului scris

“SÎNT VECHI, DOMNULE!”

prima pagină web

istoria web-ului

WORLD WIDE WEB

tehnologii fundamentale

- **HTML = Hypertext Markup Language**
limbaj pentru crearea documentelor web
- **URI = Uniform Resource Identifier**
un identificator unic pentru fiecare resursă web
- **HTTP = HyperText Transfer Protocol**
principalul mijloc pentru cererea și primirea de resurse hypermedia web

URI

șir de caractere care identifică o resursă
prin nume sau locație

- **URN** = Uniform Resource Name

identificare prin nume

urn:ietf:rfc: 7230 sau urn:isbn:9780241339053

- **URL** = Uniform Resource Locator

identificare prin locație

<https://datatracker.ietf.org/doc/html/rfc7230>

URL

protocol:// host:port /location?query#fragment

[https://www.youtube.com:443/watch?
v=npqD602G90o&t=72s](https://www.youtube.com:443/watch?v=npqD602G90o&t=72s)

PARSE IT!

FILE URL

legătura la un fișier local

`file:///home/Documents/fmi/wp/tw/c1/index.html#/23`

HTTP

HTTPS = HTTP + secure (port 443)

criptare Transport Layer Security (TLS)

HTTP (port 80)

de ce nu HTTPS, <http://mae.ro/> ?

CLIENT-SERVER PROTOCOL

SERVER: program care rulează pe un calculator conectat la Internet și care furnizează resurse web

CLIENT: browser; program care facilitează accesul la resurse web

CLIENT-SERVER PROTOCOL

#1: stabilirea unei conexiuni HTTP

#2: HTTP request

#3: HTTP response

METODE HTTP

GET - solicită o resursă de pe server

POST - transmite date către server și primește un răspuns

HEAD - solicită informații despre o resursă

PUT - actualizează/depune o resursă pe server

DELETE - șterge o resursă pe server

CODURI DE STARE

103 Early Hints

200 OK

301 Moved Permanently

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

503 Service Unavailable

ISTORIA HTTP

de la HTTP/0.9 la HTTP/3

The state of HTTP in 2022

HTTP/3 in 2023

State of the web

LIMBAJE MARKUP

- Standardised Generalised Markup Language (SGML)
Charles Goldfarb, Ed Mosher, Ray Lorie, 1970, IBM
- HyperText Markup Language (HTML)
Tim Berners-Lee, 1989, CERN
- Extensible Markup Language (XML)
[W3C](#), 1996
- HTML4 (1997); XHTML(2000)
- HTML5 (2012)

HTML + CSS ALIGNMENT

```
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <!-- lawful good -->
  </head>
  <body>
    <p>
      <a href="https://hmpg.net/">Hello, world!
    </a>
    </p>
  </body>
</html>
```

>

HTML | CSS | JS

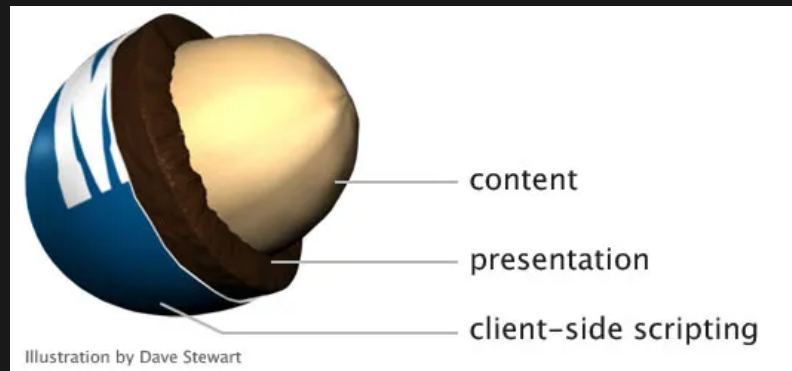
CSS în fișier separat

```
<link rel="stylesheet" type="text/css" href="style.css">
```

cod JavaScript în fișier separat

```
<script type="text/javascript" src="script.js"></script>
```

HTML | CSS | JS



“Start with your content peanut, marked up in rich, semantic HTML. Coat that content with a layer of rich, creamy CSS. Finally, add JavaScript as the hard candy shell to make a wonderfully tasty treat (and keep it from melting in your hands).”

Aaron Gustafson, Understanding progressive enhancement

HYPER BIBLIOGRAFIE

HTTP caching

HTTP cookies

The Garden Of Forking Paths, Jorge Luis Borges

New Media from Borges to HTML, Lev Manovich

TEHNICI WEB

HTML

Claudia Chiriță . 2024/2025

<https://html.spec.whatwg.org/>

HTML = HYPERTEXT MARKUP LANGUAGE

limbaj de marcare folosit pentru

- definirea structurii
- descrierea conținutului

unei pagini web

DOCUMENTE HTML

- conțin *elemente* (tags) care descriu diferite tipuri de conținut: paragrafe, ancore, liste, imagini, tabele ș.a.
- browserele interpretează elementele și afișează documentele HTML într-o formă grafică

DOCUMENTE HTML

- pot fi scrise folosind orice editor text
- au extensia *htm* sau *html*

ELEMENTE HTML

`<tag>` conținut `</tag>`
tag de deschidere tag de închidere

nu sunt case sensitive (`<tag>`, `<TAG>`, `<tAg>`)

ELEMENTE HTML

- cu conținut - au tag de deschidere și de închidere
ex. <p> un paragraf despre ornitorinci </p>
- elemente fără conținut - au doar tag de deschidere
*ex.
, <hr>, , <link>, <meta>*

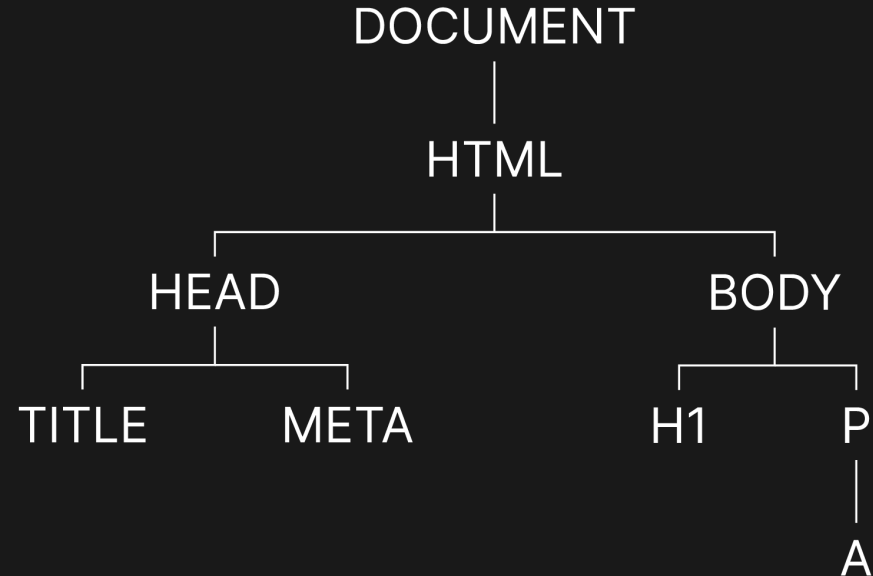
STRUCTURA UNUI DOCUMENT HTML

```
<!DOCTYPE html> <!-- obligatoriu HTML5 -->
<html lang="ro">
  <head>
    <title>Titlul paginii</title>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <h1>O pagină simplă</h1>
    <p> Aceasta este o pagină
      <a
href="http://dexonline.ro/definitie/simplu">simplă</a>.
    </p>
    <!-- acesta este un comentariu -->
  </body>
</html>
```

>

DOCUMENT HTML = ARBORE DE ELEMENTE

DOM = Document Object Model



IMBRICARE

elementele HTML trebuie să fie corect imbricate

```
<p><b> greșit imbricat </p></b>
```

```
<p><b> corect imbricat </b></p>
```

IMBRICARE

```
<!DOCTYPE html>
<html>
  <head>
    <title>Test</title>
  </head>
  <body>
    <span><p>ornitorinc</p></span>
  </body>
</html>
```

>

<https://validator.w3.org/>

flow content

ELEMENTE BLOCK

- încep și se termină cu o linie nouă și ocupă întreaga lățime a elementului părinte
- pot conține alte elemente block sau inline
- în HTML5 corespund categoriei *flow content*
- ex: <p>, <div>, <h1>-<h6>

ELEMENTE BLOCK

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p {background-color: yellow;}
    </style>
  </head>
  <body>
    <p>ornitorinc 1</p>
    <p>ornitorinc 2</p>
  </body>
</html>
```

>

ELEMENTE INLINE

- ocupă doar spațiul rezervat elementului, fără a ocupa toată lățimea părintelui
- nu încep neapărat cu o linie nouă
- pot conține alte elemente inline
- în HTML5 corespund categoriei *phrasing content*
- ex: `<span>`, `<img>`, `<a>`

ELEMENTE INLINE

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      span {background-color: yellow;}
    </style>
  </head>
  <body>
    <p>ornitorinc <span>galben</span></p>
  </body>
</html>
```

>

ATTRIBUTE

- elementele HTML pot avea attribute
- attributele specifică o informație adițională asociată elementului respectiv

nume-atribut="valoare"

```
<tag atribut1="valoare1" atribut2="valoare2" ...=""></tag>
```

ATTRIBUTE

- globale (se pot asocia tuturor elementelor)

```
<p id="p1" class="abc">...</p>
```

- specifice unui element anume

href este atribut al elementelor a, link

src este atribut al elementelor img, iframe

```

```

ISTORIC HTML

- HyperText Markup Language (HTML)
Tim Berners-Lee, 1989, CERN
- Extensible Markup Language (XML)
[W3C](#), 1996
- HTML4 (1997); XHTML(2000)
- HTML5 (2012)

HTML5

- HTML Living Standard
- specificația HTML = living document, fără număr de versiune din 2011
- adaugă tag-uri multimedia: <video>, <audio>
- adaugă tag-uri pentru grafică: <svg>, <canvas>

HTML5

- Îmbunătățește conținutul semantic prin elemente structurale noi:
<main>, <section>, <article>, <header>, <footer>, <aside>, <figure>, <figcaption>, ...

STRUCTURA UNUI DOCUMENT HTML

```
<!DOCTYPE html> <!-- HTML5 -->
<html lang="ro"> <!-- elementul rădăcină -->
<head> <!-- metadata -->
    <meta charset="utf-8">
    <title> titlul documentului </title>
    ...
</head>
<body>
    <!-- conținutul și structura paginii web -->
</body>
</html>
```

SEMANTIC WEB

Web 3.0

“ Web pages are designed to be read by people, not machines. The semantic web is a vision of information that can be readily interpreted by machines, so machines can perform more of the tedious work involved in finding, combining, and acting upon information on the web. ”

SEMANTIC HTML

“ Semantic HTML is the use of HTML markup to reinforce the semantics, or meaning, of the information in webpages rather than merely to define its presentation or look. ”

SEMANTICS

“Elements, attributes, and attribute values in HTML are defined (by this specification) to have certain meanings (semantics). ... Authors must not use elements, attributes, or attribute values for purposes other than their appropriate intended semantic purpose, as doing so prevents software from correctly processing the page.”

ELEMENTE SEMANTICE

- structură mai clară a documentului HTML
- evaluare optimă pentru motoarele de căutare (SEO)
- paginile web pot fi mai bine înțelese de diverse aplicații (ex. aplicații text-to-speech)

<header>, <nav>, <main>, <article>
<section>, <aside>, <footer>

STRUCTURARE SEMANTICĂ VS NON-SEMANTICĂ

```
<header></header>
<section>
  <article>
    <figure>
      <img>
      <figcaption></figcaption>
    </figure>
  </article>
</section>
<footer></footer>
```

```
<div id="header"></div>
<div class="section">
  <div class="article">
    <div class="figure">
      <img>
      <div class="figcaption"></div>
    </div>
  </div>
</div>
<div id="footer"></div>
```

ELEMENTELE **HEADING**

- h1, ..., h6 - pentru structurarea documentului
- 1 cel mai important, 6 cel mai puțin important
- stil implicit: text bold și mare
- folosite de motoarele de căutare pentru indexarea conținutului paginii
- marchează titlurile elementelor de secționare article, section, nav, aside

ELEMENTELE HEADING

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <h1>Ornitorinc 1</h1>
    <h2>Ornitorinc 2</h2>
    <h3>Ornitorinc 3</h3>
    <h4>Ornitorinc 4</h4>
    <h5>Ornitorinc 1</h5>
    <h6>Ornitorinc 6</h6>
  </body>
</html>
```

>

ELEMENTUL ARTICLE

```
<article> conținut articol </article>
```

- element cu conținut de sine stătător, poate fi folosit independent de restul site-ului
- fiecare element <article> trebuie să fie identificat printr-un titlu (element <h1> - <h6>)

ELEMENTUL ARTICLE

```
<article> conținut articol </article>
```

- nu are efecte vizuale
- ex: postare într-un forum, un articol de revistă online, un articol de blog

ELEMENTUL SECTION

```
<section> conținut secțiune </section>
```

- definește o secțiune generică într-un document, având o temă clară dar subordonată temei generale a documentului
- tema trebuie indicată printr-un titlu (element <h1> - <h6>)
- nu are efecte vizuale
- ex: capitolele unei cărți

SECȚIUNI CU ARTICOLE

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Animale minunate</title>
  </head>
  <body>
    <section>
      <h2>Mamifere</h2>
      <article><h4>Ornitorincul</h4></article>
      <article><h4>Tapirul</h4></article>
    </section>
    <section>
      <h2>Păsări</h2>
      <article><h4>Dodo</h4></article>
      <article><h4>Coțofana</h4></article>
    </section>
  </body>
</html>
```

>

ARTICOL CU SECȚIUNI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Cel mai minunat animal din
lume</title>
  </head>
  <body>
    <article>
      <h2>Ornitorincul</h2>
      <section>
        <h4>Habitat</h4>
      </section>
      <section>
        <h4>Caracteristici generale</h4>
      </section>
      <section>
        <h4>Legenda</h4>
      </section>
    </article>
  </body>
</html>
```

>

ELEMENTUL NAV

```
<nav> elemente de navigare </nav>
```

- reprezintă o porțiune a unei pagini cu hiperlegături
- marchează blocuri de navigare importante
- grupuri de linkuri pot apărea și în elemente footer sau header

ELEMENTUL NAV

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Trei prieteni</title>
  </head>
  <body>
    <nav>
      <a href="">Ornitorincul</a>
      <a href="">Tapirul</a>
      <a href="">Dodo</a>
    </nav>
  </body>
</html>
```



ELEMENTUL **ASIDE**

```
<aside> conținut </aside>
```

- reprezintă o porțiune a paginii cu conținut tangențial legat de conținutul paginii
- ex: pentru reclame

ELEMENTUL **ASIDE**

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitorincul</title>
  </head>
  <body>
    <section>
      <h1>Ornitorincul</h1>
      <p>Ornitorincul este un mamifer arhaic,
reprezentant al familiei Ornithorhynchidae a
ordinului Monotremata, cu o alcătuire anatomică
prezentând o neobișnuită îmbinare de caractere.
</p>
    </section>
    <aside><a
href="https://www.youtube.com/watch?
v=gEtM6fzaG68">RĂZĂTOAREA SUPER RĂZĂTOAREA!</a>
    </aside>
  </body>
</html>
```



ELEMENTUL HEADER

```
<header> conținut </header>
```

- conținut introductiv pentru pagină sau pentru un element (secțiune, articol)
- de obicei conține un heading h1,...,h6

ELEMENTUL FOOTER

```
<footer> conținut </footer>
```

- conținut adițional despre elementul de secționare cel mai apropiat
- ex. autorul, drepturi de copyright
- poate conține un element address cu datele de contact ale autorului

ELEMENTUL MAIN

```
<main> conținut </main>
```

- conținutul propriu-zis al paginii
- există un singur main pe pagină

ELEMENTE SEMANTICE **STRONG, EM, SMALL**

```
<strong> text important, de atenționare </strong>
```

```
<em> text accentuat </em>
```

```
<small> comentarii adiționale </small>
```

ELEMENTE SEMANTICE **STRONG, EM, SMALL**

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitorincul</title>
  </head>
  <body>
    <section>
      <h1>Ornitorincul</h1>
      <p>Ornitorincul este un mamifer
<strong>arhaic</strong>, reprezentant al
familiei <em>Ornithorhynchidae</em> a ordinului
<em>Monotremata</em>, cu o alcătuire anatomică
prezentând o neobișnuită îmbinare de caractere.
</p>
      <p><small>P.S. Iubesc ornitorincii.
</small></p>
    </section>
  </body>
</html>
```

>

ELEMENTE SEMANTICE **STRONG**, **EM**, **SMALL**

elementele **b** (bold), *i* (italic) au devenit și ele
semantice în HTML5

```
<b> cuvinte cheie </b>
```

```
<i> termeni științifici / cuvinte în altă limbă  
/ text alternativ </i>
```


ELEMENTELE P ȘI HR

<p> reprezintă un paragraf (fragment de text ce evidențiază o idee)

<hr> reprezintă o pauză tematică între grupuri de paragrafe

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitorincul</title>
  </head>
  <body>
    <h1>Ornitorincii</h1>
    <p>Primul ornitorinc</p>
    <p>Al doilea ornitorinc</p>
    <hr>
    <p>Al treilea ornitorinc</p>
    <p>Al patrulea ornitorinc</p>
  </body>
</html>
```

>

ELEMENTELE **DIV**, **SPAN**

```
<div> conținut </div>
```

```
<span> conținut </span>
```

- <div>, nu au înțeles special
- folosite pentru stilizare, de obicei cu attributele id și class
- înainte de <div> și după </div> se trece la rând nou

ELEMENTUL BR

`<br>` pentru întreruperea liniei

- nu se folosește pentru a marca trecerea la o idee nouă
- versurile unei poezii, componentele unei adrese poștale

LISTE NEORDONATE

`<ul>` o listă în care ordinea elementelor (item-urilor) nu are importanță

`<li>` folosit pentru a crea elementele unei liste

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitorincul</title>
  </head>
  <body>
    <h1>Ornitorincii</h1>
    <ul>
      <li>sunt veninoși</li>
      <li>sunt misterioși</li>
      <li>fac ouă</li>
    </ul>
  </body>
</html>
```

>

LISTE ORDONATE

`<ol>` o listă în care ordinea elementelor este importantă

attribute specifice: type, start, reversed

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitorincul</title>
  </head>
  <body>
    <h1>Cele mai minunate animale din lume</h1>
    <ol>
      <li>ornitorincul</li>
      <li>tapirul</li>
      <li>dodo</li>
    </ol>
  </body>
</html>
```

>

LISTE DE DESCRIERI

`<dl>, <dt>, <dd>`

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Animale</title>
  </head>
  <body>
    <dl>
      <dt>ornitorincii
        <dd>sunt veninoși</dd>
        <dd>sunt misterioși</dd>
      </dt>
      <dt>păsările dodo
        <dd>au cam dispărut</dd>
      </dt>
      <dt>tapirii
        <dd>pe ducă și ei</dd>
      </dt>
    </dl>
  </body>
</html>
```

>

ELEMENTUL **A** - ANCORĂ, HIPERLEGĂTURĂ

```
<a href="URL"> conținut </a>
```

- referință către o resursă
- **attribute:**
 - href: destinația linkului (un URL sau id al unui element din pagină)
 - download: descărcarea resursei țintă (în loc de vizualizare)
 - target = modul în care este încărcată în browser resursa țintă
_blank | _parent | _self | _top | framename
- legături interne = legături la un fragment din pagină

```
<a href="#id-meniu"> legătură la meniu  
...  
<nav id="id-meniu">apasă</nav>
```

ELEMENTUL A - ANCORĂ, HIPERLEGĂTURĂ

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Trei prieteni</title>
  </head>
  <body>
    <nav>
      <a
href="https://archive.org/details/isbn_9780151
004478">Ornitorincul</a>
      <a
href="https://www.beholder.uk/dwindle/">Tapiru
l</a>
      <a
href="https://edition.cnn.com/2023/01/31/world
/dodo-bring-back-from-extinction-colossal-
```



ELEMENTUL IMG

```

```

- attributele src și alt sunt obligatorii
- alt = text alternativ care va fi afișat dacă imaginea nu poate fi procesată
- attributele width și height pentru specificarea dimensiunii în pixeli
- formate acceptate: PNG, GIF, JPEG, PDF, XML, APNG, SVG, MNG

ELEMENTUL **IMG**

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitoricul</title>
  </head>
  <body>
    
    <br>
    
  </body>
</html>
```



ELEMENTELE **FIGURE** ȘI **FIGCAPTION**

```
<figure>
  
  <figcaption> Figura 1. Descriere </figcaption>
</figure>
```

- figure, împreună cu figcaption, formează un element de sine stătător (flow content)
- pot fi utilizate și cu alte elemente, precum <pre>, <code> și <video>

ELEMENTELE **FIGURE** ȘI **FIGCAPTION**

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Ornitoricul</title>
  </head>
  <body>
    <figure>
      
      <figcaption>Figure 1. fake baby
platypus</figcaption>
    </figure>
  </body>
</html>
```



ELEMENTELE AUDIO ȘI VIDEO

```
<audio src="URL-audio" controls> mesaj </audio>
```

```
<video src="URL-video" controls autoplay> mesaj </video>
```

```
<video src="URL-video" controls poster="URL-img"> mesaj </vide
```

- poster: imaginea va fi afișată când elementul video nu poate fi redat
- mai multe surse alternative pot fi precizate în elemente copil de tipul <source>

ELEMENTUL IFRAME

```
<iframe src="URL" width="n1" height="n2"> text afișat  
dacă website-ul nu poate fi inclus </iframe>
```

înglobarea unei pagini în interiorul paginii curente

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link rel="stylesheet" href="demo.css">  
    <title>Ornitoricul</title>  
  </head>  
  <body>  
    <iframe  
src="https://en.wikipedia.org/wiki/Platypus"  
width="350" height="200">iframe  
missing</iframe>  
    <!-- https://www.youtube.com/watch?  
v=LLYKlQ2nwqQ -->  
  </body>
```

>

TABELE

se folosesc numai pentru reprezentarea datelor
NU pentru layoutul paginii

```
<table> conținutul tabelului, linie cu linie </table>  
<caption> descrierea tabelului (opțională) </caption>  
<tr> linie formată din celule </tr>  
<td> celulă date </td>  
<th> celulă header </th>
```

TABELE

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <title>Animale</title>
  </head>
  <body>
    <table>
      <tr>
        <th>Mamifere</th>
        <th>Păsări</th>
      </tr>
      <tr>
        <td>ornitorinc</td>
        <td>dodo</td>
      </tr>
      <tr>
        <td>tapir</td>
        <td>coțofană</td>
      </tr>
    </table>
  </body>
</html>
```



TABELE

- pentru definirea tabelelor se pot folosi și elementele `thead`, `tbody`, `tfoot`, `colgroup`, `col`
- attribute specifice pentru `td` și `th`: `colspan` și `rowspan` pentru celule care ocupă mai multe coloane/linii

TEHNICI WEB

CSS

Claudia Chiriță . 2024/2025

METADATA

HTML METADATA

```
<head>
  <title>Titlul paginii</title>
  <meta charset="utf-8">
  <meta name="autor" content="nume autor">
  <meta name="description" content="descriere">
  <meta name="keywords" content="cuvinte cheie">
  <base href="base URL for relative paths">

  <style type="text/css"></style>
  <script type="text/javascript"></script>
  <script type="text/javascript" src="script.js"></script>
  <link rel="stylesheet" href="style.css" type="text/css">
  <link rel="icon" href="favicon.ico" type="image/x-icon">
</head>
```

ELEMENTUL BASE

- definește URL-ul de bază pentru toate URL-urile relative din document
- într-un document poate exista un singur element *base*

ELEMENTUL BASE

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <title>Titlul paginii</title>
    <base
href="https://cs.unibuc.ro/~cechirita/tw/c3/"
target="_blank">
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <p>
      <a href="secret/secret.html">secret</a>
    </p>
    
  </body>
</html>
```

>

CSS

CSS = CASCADING STYLE SHEETS

- limbaj style sheet folosit pentru definirea aspectului unei pagini web:
 - stilizarea elementelor: culoare, mărime, font etc.
 - așezarea elementelor în pagină
 - determinarea vizibilității
- folosit și pentru animații

FIȘIERUL DE STIL

```
<head>  
...  
<link rel="stylesheet" href="filename.css" type="text/css">  
...  
</head>
```

REGULI CSS

- fișierul de stil conține una sau mai multe reguli CSS

```
selector {  
  prop1: valoare1;  
  prop2: valoare2;  
  prop3: valoare3;  
  ...  
}
```

```
p {  
  color: pink;  
  font-family: sans-serif;  
  font-size: 12pt;  
}
```

HTML + CSS ALIGNMENT

în fișier separat

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css"
type="text/css">
  </head>
  <body>
    <p>
      <a
href="https://en.wikipedia.org/wiki/Alignment_(D
ungeons_%26_Dragons)">lawful good</a>
    </p>
  </body>
</html>
```

>

HTML + CSS ALIGNMENT

în fișier

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      a {color: pink;}
    </style>
  </head>
  <body>
    <p>
      <a
href="https://en.wikipedia.org/wiki/Alignment_
(Dungeons_%26_Dragons)">chaotic neutral</a>
    </p>
  </body>
</html>
```

>

HTML + CSS ALIGNMENT

in line

```
<!DOCTYPE html>
<html lang="en">
  <head>
  </head>
  <body>
    <p>
      <a style="color: red"
href="https://en.wikipedia.org/wiki/Alignment_(D
ungeons_%26_Dragons)">chaotic evil</a>
    </p>
  </body>
</html>
```

>

PRECEDENȚA STILURILOR

în ordine crescătoare a precedenței:

1. stilul implicit al browser-ului
2. fișier CSS extern și stiluri *in file*; se aplică stilul care apare scris ultimul în <head>
3. stiluri inline

SELECTORI

```
selector1, selector2 {  
  prop1: valoare1;  
}  
selector1 {  
  prop2: valoare2;  
}
```

- unui element i se pot aplica mai multe reguli
- se grupează elementele care au un stil comun; stilurile specifice se scriu în reguli separate
- unele stiluri sunt moștenite de la elementele părinte (în arborele DOM), dar pot fi schimbate folosind reguli specifice

PROPRIETĂȚI: CULORI

```
color: definirea culorii textului;
```

```
background-color: definirea culorii fundalului;
```

- specificate prin **nume**: red, green, blue, pink, ...

PROPRIETĂȚI: CULORI

- specificate prin valoarea `rgb(red, green, blue)`
red, green, blue au valori între 0-255
- specificate prin valoarea hexazecimală `#rrggbb`
rr, gg, bb au valori între 0-255

PROPRIETĂȚI: CULORI

- prin valoarea `hsl(hue, saturation, lightness)`
 - hue reprezintă gradul culorii (0-360)
 - saturation reprezintă intensitatea (0%-100%)
 - lightness reprezintă luminozitatea (0%-100%)

PROPRIETĂȚI: CULORI

- prin `valoarea rgba(red, green, blue, alpha)`
extensie a valorii `rgb` cu valoarea `alpha` care reprezintă opacitatea (0.0-1.0)
- prin `hsla(hue, saturation, lightness, alpha)`
extensie a valorii `hsl` cu `alpha`
- `convertor culori`

DISTANȚA/MĂRIMEA

- unități absolute: mm, cm, px, in, pt
- unități relative:
 - em: relativ la dimensiunea fontului elementului
 - %: relativ la dimensiunea părintelui
 - vw, vh: relativ la 1% din lățimea/înălțimea viewportului
 - rem: relativ la dimensiunea fontului rădăcinii

SELECTORI

SELECTORUL UNIVERSAL *

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      * {border: 1px solid pink;}
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div><p>paragraf în div</p></div>
    <p>paragraf fără div</p>
    <p>alt paragraf fără div</p>
  </body>
</html>
```

>

SELECTAREA PRIN TIP

numele unui element din document: h1, p, div, a...

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      p { color: pink;
        margin-left: 25px;}
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div><p>paragraf în div</p></div>
    <p>paragraf fără div</p>
    <p>alt paragraf fără div</p>
  </body>
</html>
```

>

SELECTAREA PRIN ID

- #identif selectează elementul cu id="identif"
- <tag id="identif"> conținut </tag>
un document HTML poate conține un singur atribut id cu valoarea "identif"

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      div#galben { background-color:
yellow;}
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div id="galben"><p>paragraf în div</p>
</div>
    <p>paragraf fără div</p>
    <p>alt paragraf fără div</p>
  </body>
</html>
```

>

SELECTAREA PRIN CLASĂ

- .cls selectează elementele cu class="cls"
- <tag1 class="cls"> conținut </tag1>
<tag2 class="cls"> conținut </tag2>
cu atributul class grupăm mai multe elemente, care vor avea un stil comun

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      .galben { background-color: yellow;}
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div class="galben"><p>paragraf în div</p>
  </div>
    <p>paragraf fără div</p>
    <p class="galben">alt paragraf fără
  div</p>
  </body>
</html>
```

>

SELECTAREA PRIN CLASĂ

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      p.a.b { background-color: yellow;}
      p.a, p.b { border: 1px solid pink; }
      p.a:not(.b){ color: red; }
      p.b:not(.a){ color: cyan; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div class="a">div</div>
    <p class="a b">paragraf 1</p>
    <p class="a">paragraf 2</p>
    <p class="b">paragraf 3</p>
    <p> paragraf 4</p>
  </body>
</html>
```



SPECIFICITATE

selectarea prin id are specificitate mai mare decât
selectarea prin class

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #rosu { color: red; }
      .bleu { color: cyan; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <p id="rosu" class="bleu">paragraf</p>
  </body>
</html>
```

>

SPECIFICITATE

când două stiluri au aceeași specificitate, câștigă cel care este scris ultimul în fișierul CSS

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      .rosu { color: red; }
      .bleu { color: cyan; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <p class="rosu bleu">paragraf</p>
  </body>
</html>
```

>

!IMPORTANT

proprietățile marcate !important sunt aplicate
indiferent de specificitatea lor

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      .rosu { color: red; }
      .bleu { color: cyan; }
      p { color: pink !important; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <p class="rosu bleu">paragraf</p>
  </body>
</html>
```

>

INHERIT

- unele proprietăți pot lua valoarea inherit: proprietatea este moștenită de la elementul părinte
- folosită pentru a suprascrie un alt stil și a forța comportamentul implicit

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      div { color: red; }
      p { color: cyan; }
      div>p { color: inherit; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <p>paragraf în afara divului</p>
    <div><p>paragraf în div</p></div>
  </body>
</html>
```

>

COMBINAREA SELECTORILOR

- selectori pentru descendenți

$E F$ selectează toate elementele F care sunt descendenți ai lui E

- selectori pentru copii

$E > F$ selectează elementele F care sunt copii ai lui E

- selectori pentru frați

$E + F$ selectează toate elementele F care sunt frați cu E și care urmează imediat după E

$E \sim F$ selectează toate elementele F care sunt frați cu E și care urmează după E

COMBINAREA SELECTORILOR

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      div > p { color:black; background-color:
yellow; }
    </style>
  </head>
  <body>
    <h1>Titlu</h1>
    <div><p>paragraf 1 în div</p>
      <p>paragraf 2 în div</p>
      <aside><p>paragraf 3 în div</p></aside>
    </div>
    <p>paragraf 4 în afara divului</p>
    <p>paragraf 5 în afara divului</p>
  </body>
</html>
```

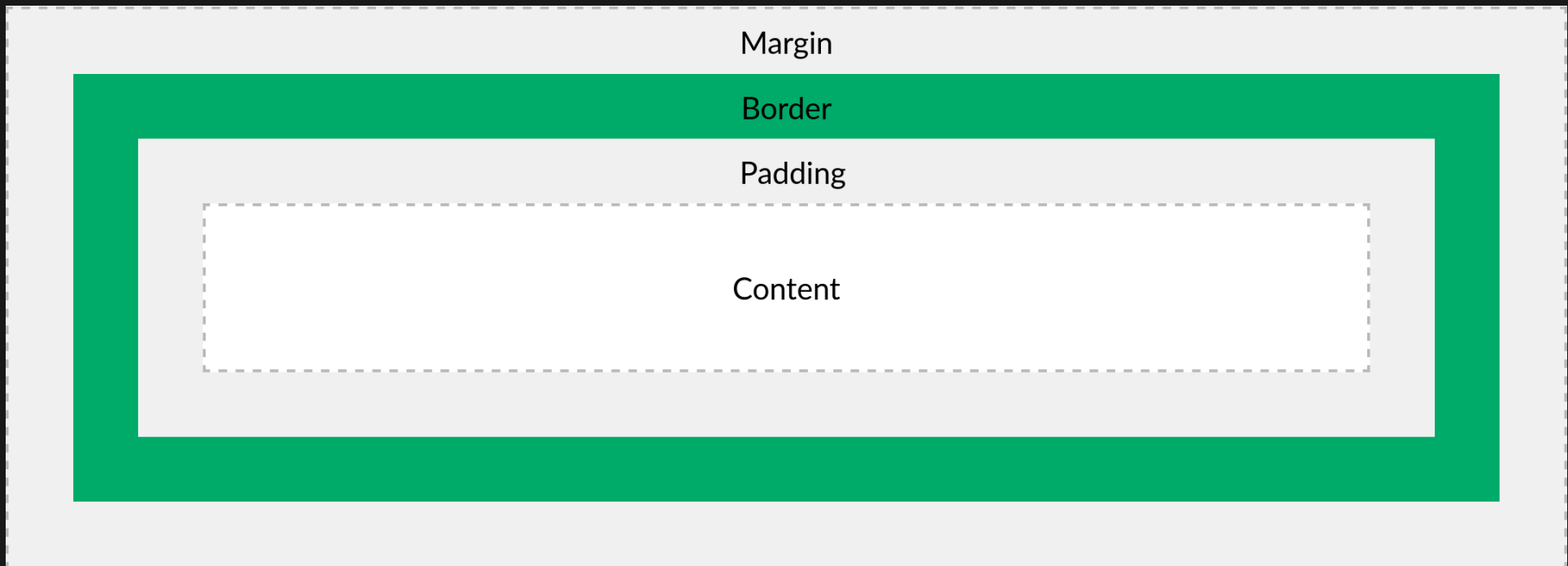
>

SELECTAREA PRIN ATRIBUTE

- `e[attr]` -> elementele e care au atributul attr
- `e[attr="val"]` -> elementele e cu attr="val"
- `e[attr~="val"]` -> elementele e cu proprietatea că valoarea lui attr conține cuvântul val
- `e[attr*="val"]` -> elementele e cu proprietatea că valoarea lui attr conține subșirul val

CSS BOX MODEL

- reprezentare a spațiului pe care îl ocupă un element pe o pagină și a modului în care acesta se raportează la alte elemente



CSS BOX MODEL

- width / height
- padding
- border
- margin

valori CSS default

WIDTH + HEIGHT

```
width: 500px /* 50% */  
/* height, max-width, min-width, max-height, min-height */
```

BORDER

```
border: grosime stil culoare;  
border-left: grosime stil culoare;  
border-right: grosime stil culoare;  
border-top: grosime stil culoare;  
border-bottom: grosime stil culoare;
```

- grosime: exprimată în px, pt, em, *thin, medium, thick*
- stil: *none, solid, dotted, dashed, inset, outset, double, groove, ridge, hidden*

BORDER

```
border-color: ...;  
border-style: ...;  
border-width: ...;  
border-left-color: ...;  
border-bottom-width: ...;
```

```
border-radius: 12px; /* margini cu colțuri rotunjite */
```

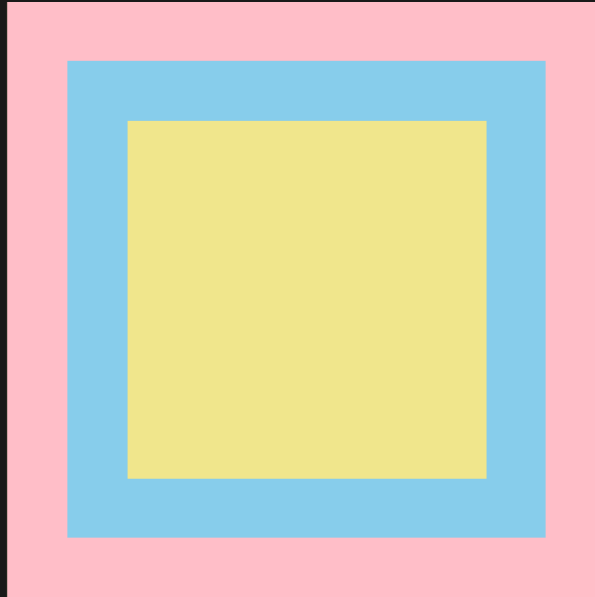
PADDING + MARGIN

```
padding: 15px;  
padding-top: 15px;  
padding-right: 20%;  
padding-bottom: 15px;  
padding-left: 10%;
```

```
margin: 15px;  
margin-top: 15px;  
margin-right: 20%;  
margin-bottom: 15px;  
margin-left: 10%;
```

```
margin: 15px 20% 15px 10%;  
padding: 15px 20% 15px 10%;
```

FLASH QUIZ



- pătratul galben are lățimea și înălțimea de 150px
- pătratul mare are lățimea și înălțimea de 250px
- banda roz și bleu au grosimea de 25px

FLASH QUIZ

```
<div id="big-square">  
  <div id="small-square"></div>  
</div>
```

- pătratul galben are lățimea și înălțimea de 150px
- pătratul mare are lățimea și înălțimea de 250px
- banda roz și bleu au grosimea de 25px

FLASH QUIZ

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #big-square {}

      #small-square {}
    </style>
  </head>
  <body>
    <div id="big-square">
      <div id="small-square"></div>
    </div>
  </body>
</html>
```

>

FLASH QUIZ SOLUTION #1

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #big-square { background-color: pink;
        height: 200px;
        padding: 25px;
        width: 200px; }

      #small-square { background-color: khaki;
        border: 25px solid skyblue;
        height: 150px;
        width: 150px;
        }
    </style>
  </head>
  <body>
    <div id="big-square">
      <div id="small-square"></div>
    </div>
  </body>
</html>
```



FLASH QUIZ SOLUTION #2

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #big-square { background-color: skyblue;
        border: 25px solid pink;
        height: 200px;
        width: 200px; }

      #small-square { background-color: khaki;
        margin: 25px;
        height: 150px;
        width: 150px;
        }
    </style>
  </head>
  <body>
    <div id="big-square">
      <div id="small-square"></div>
    </div>
  </body>
</html>
```



FONTS

```
font-family: Helvetica,"Times New Roman",serif;  
font-size: 16px; /* 16pt, 1.25em, 125%, x-large*/  
font-weight: normal; /* bold, lighter, bolder, 100,...,900 */  
font-style: normal; /* italic, oblique */
```

font-family:

- fonturi generice: serif, sans-serif, monospace, cursive, fantasy
- fonturi specifice: Times New Roman, Arial, Georgia, Verdana

FONTS

```
font-family: Helvetica, "Times New Roman", serif;  
font-size: 16px; /* 16pt, 1.25em, 125%, x-large*/  
font-weight: normal; /* bold, lighter, bolder, 100, ..., 900 */  
font-style: normal; /* italic, oblique */
```

font-size:

- unități: px, pt, em (1em = dimensiunea fontului curent)
- valori: xx-small, x-small, small, medium, large, x-large, xx-large, ...
- procente: procent din dimensiunea fontului părintelui

FONTURI CUSTOM

```
/* pentru importarea unui font*/  
  
@font-face {  
  font-family: nume-font;  
  src:url(file-font.ext);  
}  
  
selector { font-family: nume-font;  
}
```

FONTURI CUSTOM

exemplu: embed mr. Roboto

FORMATARE TEXT

```
color: pink;
text-align: center; /* left, right, justify */
text-decoration: underline; /* overline, line-through */
text-shadow: -2px 5px 2px gray; /* none initial */
/* x-offset, y-offset, blur-radius, culoare */
text-indent: 3em;
text-transform: uppercase; /* lowercase, capitalize */
line-height: 150%; /* 1.5em , 30px */
word-spacing: 1em; /* +15mm */
letter-spacing: 0.1em; /* -1px */
```

VIZIBILITATE

```
visibility: visible; /* hidden */
```

- setează vizibilitatea unui element;
chiar dacă vizibilitatea este *hidden*, elementul ocupă
loc în pagină

OVERFLOW

```
overflow: hidden; /* visible, scroll */
```

- descrie modul în care este redat textul care iese dintr-un cadru

DISPLAY

```
display: inline; /* block, inline-block, none */
```

- descrie modul în care este afișat un element în pagină
- spațiul pe care îl ocupă este lungimea elementului
- ignoră proprietățile width și height, dar acceptă border, padding

DISPLAY

```
display: block; /* inline, inline-block, none */
```

- descrie modul în care este afișat un element în pagină
- permisă stilizarea folosind box model
- plasare pe rând nou

DISPLAY

```
display: inline-block; /* inline, block, none */
```

- descrie modul în care este afișat un element în pagină
- element poziționat inline care poate fi stilizat ca un element bloc
- acceptă proprietățile width și height

DISPLAY

```
display: none; /* inline, block, inline-block */
```

- descrie modul în care este afișat un element în pagină
- elementul nu este afișat
- deși elementul este eliminat din pagină, el este prezent în DOM

EXTRA++

- cascading style sheets
- utf-8 vs utf-16 vs ascii

TEHNICI WEB

CSS #2

Claudia Chiriță . 2024/2025

PSEUDO-CLASE

PSEUDO-CLASE

```
selector:pseudo-class {  
  proprietate: valoare;  
}
```

- cuvânt-cheie adăugat unui selector ce specifică o stare specială a elementului selectat
- ex. folosite pentru definirea stilului
 - unui element la trecerea cu mouse-ul peste el
 - diferit pentru legături vizitate și nevizitate

PSEUDO-CLASE

```
:link /* legături care nu au fost vizitate */  
:visited /* legături care au fost vizitate */  
:hover /* elemente peste care se trece cu mouse-ul */  
:active /* legături active */  
:not() /* elementele care nu apar în argumentul lui not */
```

PSEUDO-CLASA :LINK

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      a {color:white;}
      a:link {color:red;
              text-decoration:none;}
      a:visited {color:cyan;
                  text-decoration:none;}
    </style>
  </head>
  <body>
    <a href="https://github.com">visited</a>
  <br>
  <a
```



PSEUDO-CLASA :HOVER

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      p {color:white;}
      p:hover {color:red;
              text-decoration:underline;}
      a {color:white;}
      a:active {background-color:yellow;
               color:black;}
    </style>
  </head>
  <body>
    <p>hover over me</p>
    <a>click me</a>
  </body>
</html>
```



PSEUDO-CLASA :NOT

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <style type="text/css">
      p {color:white;}
      p:not(.foo){color:red;}
    </style>
  </head>
  <body>
    <p>paragraf fără clasă</p>
    <p class="foo">paragraf cu .foo</p>
  </body>
</html>
```



PSEUDO-CLASA :FIRST-CHILD

selectează un element care este primul copil al unui alt element

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <style type="text/css">
      p {color:white;}
      p:first-child {color:red;}
      p:last-child {color:cyan;}
    </style>
  </head>
  <body>
    <p>paragraf 1</p>
    <p>paragraf 2</p>
    <div><p>paragraf 3</p>
    <p>paragraf 4</p></div>
  </body>
</html>
```

>

PSEUDO-CLASA :FIRST-CHILD

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      p {color:white;}
      p:first-child b{color:red;}
      /* p b:first-child {color:red;} */
    </style>
  </head>
  <body>
    <p>the panda <b>eats</b>, <b>shoots</b> &
    <b>leaves</b></p>
    <p>the panda eats <b>shoots</b> &
    <b>leaves</b></p>
  </body>
</html>
```



PSEUDO-CLASE

`:first-of-type`

selectează elementul care este primul copil de tipul specificat al unui alt element

`:nth-of-type(n)`

selectează elementul care este al n-lea copil de tipul specificat al unui alt element

`:nth-child(n)`

selectează elementul de tipul specificat care este al n-lea copil al unui alt element

n poate fi număr natural, cuvânt cheie (*odd*, *even*), formulă ($an+b$, cu n pornind de la 0)

PSEUDO-CLASE

```
<!DOCTYPE html>
<html lang="ro">
  <head>
    <style type="text/css">
      p {color:white;}
      p:nth-of-type(2){color:red;}
      p:nth-of-type(odd){color: cyan;}
      p:nth-of-type(4n+0){color: pink;}
    </style>
  </head>
  <body>
    <p>paragraf 1</p>
    <p>paragraf 2</p>
    <p>paragraf 3</p>
    <p>paragraf 4</p>
    <p>paragraf 5</p>
    <p>paragraf 6</p>
    <p>paragraf 7</p>
    <p>paragraf 8</p>
  </body>
</html>
```

>

PSEUDO-CLASE

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      * {color:white;}
      p:nth-of-type(3){color:red;}
      p:nth-child(3){color:cyan;}
    </style>
  </head>
  <body>
    <div>div</div>
    <p>paragraf 1</p>
    <p>paragraf 2</p>
    <p>paragraf 3</p>
    <p>paragraf 4</p>
    <p>paragraf 5</p>
    <p>paragraf 6</p>
    <p>paragraf 7</p>
    <p>paragraf 8</p>
  </body>
</html>
```

>

PSEUDO-ELEMENTE

PSEUDO-ELEMENTE (::)

- folosite pentru a defini stilul unei părți a unui element
- elemente virtuale, create cu CSS, care nu apar în arborele HTML al paginii

```
::first-letter /* prima literă a unui text */  
::first-line  /* prima linie a unui element block */  
  
::before, ::after /* folosite pentru a adăuga conținut*/  
                /* înainte/după un element HTML; */  
                /* folosite cu proprietatea content */
```

PSEUDO-ELEMENTE (::)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <style type="text/css">
      * {color:white;}
      p::first-letter {font-size: 1.5rem;
                        font-weight: bold;
                        color: red;}
    </style>
  </head>
  <body>
    <p>Flatland: A Romance of Many Dimensions
    is a satirical novella by the English
    schoolmaster Edwin Abbott Abbott, first
    published in 1884. The story describes a two-
    dimensional world inhabited by geometric
    figures; women are line segments, while men
    are polygons with various numbers of sides.
  </p>
    <a href="https://www.youtube.com/watch?v=yBbZmwROv84">animation</a>
```



VARIABLE CSS

- proprietăți CSS custom (nestandard, definite de utilizator)
- definite pentru a fi reutilizate în cadrul unui document

VARIABLE CSS

- globale: valabile în întreg documentul

```
:root {  
  --nume-proprietate: valoare;}
```

- locale: valabile în interiorul (toți descendenții) elementului în care au fost definite

```
element {  
  --nume-proprietate: valoare;}
```

- se folosesc cu funcția

```
var(--nume-proprietate)
```

CSS LAYOUT

PROPRIETATEA POSITION

- tipul de poziționare a unui element în pagină
- poate avea valorile:

```
position:static;  
position:relative;  
position:absolute;  
position:fixed;  
position:sticky;
```

- pentru poziționare se folosesc proprietățile *left, right, top* și *bottom*

POSITION:STATIC

- poziția implicită a oricărui element HTML
- cu poziția *static* elementul va avea un flux normal în pagină
- elementele cu poziția *static* nu sunt afectate de proprietățile left, right, top, bottom

POSITION:RELATIVE

- un element cu *position:relative* este poziționat relativ față de poziția pe care ar fi avut-o în mod normal în pagină
- pentru poziționare se folosesc proprietățile *left, right, top* și *bottom*

POSITION:ABSOLUTE

- elementul este poziționat relativ față de primul părinte care are *position:absolute/relative/fixed*
- elementul este scos din fluxul documentului
- se pot suprapune cu alte elemente (exit Flatland!)

POSITION:FIXED

- elementul are o poziție fixă în fereastra de browser
- elementul este scos din fluxul documentului și nu e afectat de scroll
- pentru poziționare se folosesc proprietățile *left, right, top* și *bottom*

POSITION:STICKY

- elementul comută între poziționarea relativă și cea fixă în funcție de scroll
- dacă poziția scrollului nu depășește elementul, acesta va fi poziționat relativ, altfel se comportă ca un element fix

SUPRAPUNERI: Z-INDEX

```
z-index: ..., -100 /* 0, 100, ... */
```

- prin poziționare, elementele se pot suprapune; pot apărea stive de elemente
- ordinea elementelor în stivă este dată de *z-index*
- elementele cele mai vizibile au *z-index* mai mare
- *z-index* poate fi setată doar pentru elementele care sunt poziționate *absolute*, *relative* sau *fixed*

PROPRIETATEA FLOAT

```
float: left; /* right, none */
```

- elementele cu proprietatea *float* sunt scoase din fluxul documentului și poziționate conform valorii, la stânga sau la dreapta
- ele afectează celelalte elemente: se aranjează înconjurând ("wrapping") elementele *float*
- elementele poziționate absolut ignoră prop. *float*

PROPRIETATEA CLEAR

```
clear: left; /* right, none, both */
```

- determină dacă un element trebuie poziționat sub elemente *float* care îl precedă
- elementele cu proprietatea *clear* nu se aranjează în jurul elementelor *float*, ci se poziționează sub acestea

FLEXBOX

- metodă de layout unidimensională pentru aranjarea itemurilor în rânduri și coloane
- itemurile se extind (*flex*) pentru a umple spațiul disponibil sau se micșorează pentru a încăpea în spații mai mici

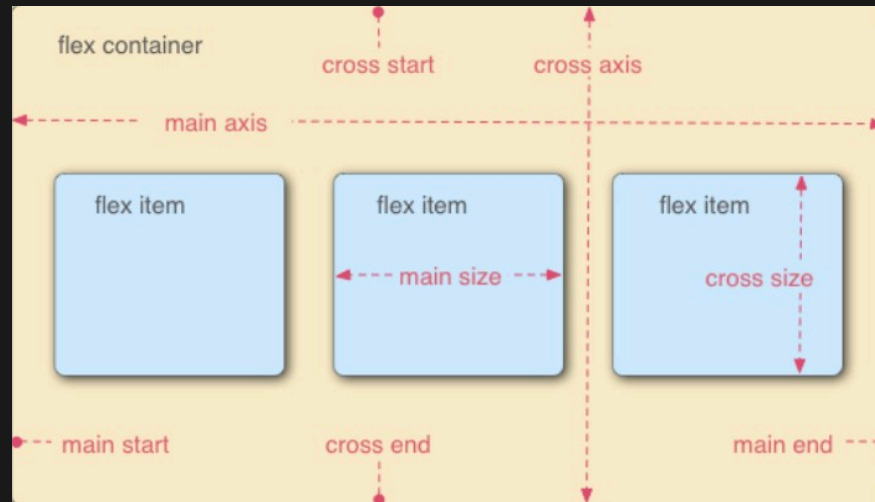
FLEXBOX

- layoutul flexibil oferă o metodă eficientă de a așeza, alinia și a distribui spațiul între elementele din document chiar și atunci când dimensiunea viewportului și a elementelor este necunoscută sau dinamică

tutorial flexbox

FLEXBOX

- layoutul flexibil presupune definirea a două tipuri de elemente:
 - **flex container** - element părinte
 - **flex items** - elemente copii



FLEX CONTAINER

proprietăți pentru părinte:

```
display  
flex-direction  
flex-wrap  
flex-flow  
justify-content  
align-items  
align-content
```

DISPLAY FLEX

```
display: flex; /* inline-flex */
```

- definește un container flexibil inline sau block, în funcție de valoarea dată
- creează un context flexibil pentru toți copiii săi direcți

FLEX DIRECTION

```
flex-direction: row; /* row-reverse, column, column-reverse*/
```

- stabilește direcția în care sunt plasate itemurile în container: pe orizontală (stânga-dreapta sau dreapta-stânga) sau pe verticală (sus-jos sau jos-sus)

FLEX WRAP

```
flex-wrap: wrap; /* nowrap, wrap-reverse*/
```

- stabilește dacă itemurile vor fi aranjate pe o singură linie (coloană) sau pe mai multe
- implicit, itemurile se vor aranja toate pe o linie (coloană)

FLEX FLOW

```
flex-flow: flex-direction flex-wrap;
```

- prescurtare pentru proprietățile flex-direction și flex-wrap
- implicit: flex-flow: row nowrap

JUSTIFY CONTENT

```
justify-content: start; /* end, center, space-between,  
                        space-around, space-evenly */
```

- specifică modul în care sunt aliniat itemurile de-a lungul axei principale

ALIGN ITEMS

```
align-items: stretch; /* start, end, center, baseline */
```

- specifică modul în care sunt aliniate itemurile pe axa secundară (axa perpendiculară pe axa principală)

ALIGN CONTENT

```
align-content: stretch; /* start, end, center,  
                           space-between, space-around */
```

- distribuie spațiul dintre și în jurul itemurilor pe axa transversală
- nu are efect dacă există o singură linie în container

FLEX ITEMS

proprietăți pentru copii:

```
order  
flex-grow  
flex-shrink  
flex-basis  
flex  
align-self
```

ORDER

```
order: <întreg>; /* implicit 0*/
```

- stabilește ordinea în care apar itemurile în containerul flexibil
- implicit, itemurile apar în ordinea în care sunt scrise în documentul HTML

FLEX GROW

```
flex-grow: <număr>; /* implicit 0*/
```

- stabilește capacitatea itemurilor de a-și mări dimensiunea dacă este spațiu disponibil în interiorul containerului
- dacă toate itemurile au flex-grow egal cu 1, ele vor împărți spațiul disponibil în mod egal

FLEX SHRINK

```
flex-shrink: <număr>; /* implicit 1*/
```

- definește capacitatea itemurilor de a se micșora dacă este necesar

FLEX BASIS

```
flex-basis: <length>; /* auto (implicit) */
```

- definește dimensiunea implicită a unui element înainte de distribuirea spațiului rămas

FLEX

```
flex: flex-grow flex-shrink flex-basis;
```

- prescurtare pentru proprietățile *flex-grow*, *flex-shrink* și *flex-basis*
- implicit flex: 0 1 auto

ALIGN SELF

```
align-self: stretch; /* start, end, center, baseline, auto */
```

- permite alinierea individuală a itemurilor pe axa secundară
- înlocuiește alinierea specificată cu *align-items*

WHY FLEX?

situații tipice în care folosim flexbox

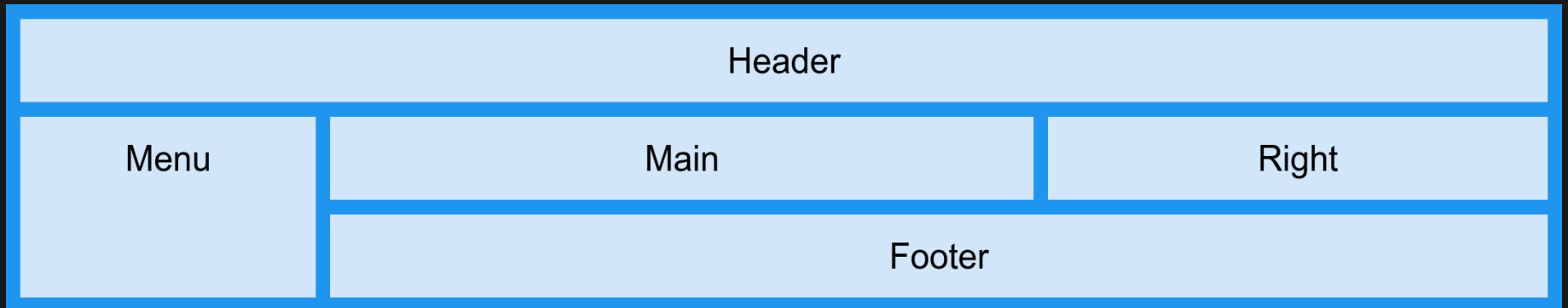
GRID LAYOUT

- sistem de layout bidimensional, bazat pe rețea, cu rânduri și coloane, ce ușurează modul de a așeza, alinia și a distribui spațiul între elementele din document

tutorial grid

GRID LAYOUT

- CSS grid presupune definirea a două tipuri de elemente:
 - **grid container** - element părinte
 - **grid items** - elemente copii



GRID CONTAINER

proprietăți pentru părinte:

```
display  
grid-template-columns & grid-template-rows  
grid-template-areas  
column-gap & row-gap  
justify-content  
align-content  
justify-items & align-items
```

DISPLAY GRID

```
display: grid; /* inline-grid */
```

- definește elementul ca un grid container inline sau block, în funcție de valoarea dată
- stabilește un context de formatare a grilei pentru toți copiii săi direcți

GRID TEMPLATE

`grid-template-columns`
`grid-template-rows`

- definește numărul de coloane și rânduri în care este împărțit gridul și lățimea/înălțimea acestora (unități de măsură: %, px, auto, fr)

`grid-template-areas`

GAPS

column-gap

row-gap

- definește spațiul dintre coloane și dintre rânduri

mind the gap?

JUSTIFY CONTENT

```
justify-content: start; /* end, center, space-between,  
                        space-around, space-evenly */
```

- specifică modul în care este aliniată grila în interiorul containerului
- alinierea se face pe orizontală

ALIGN CONTENT

```
align-content: stretch; /* start, end, center,  
                           space-between, space-around, space-evenly
```

- specifică modul în care este aliniată grila în interiorul containerului, pe verticală

```
place-content: align-content justify-content
```

JUSTIFY & ALIGN ITEMS

```
justify-items: stretch; /* start, end, center */
```

- specifică modul în care sunt aliniate itemurile pe axa principală

```
align-items: stretch; /* start, end, center, baseline */
```

- specifică modul în care sunt aliniate itemurile pe axa secundară

```
place-items: align-items justify-items
```

GRID ITEMS

proprietăți pentru copii:

```
grid-column & grid-row  
grid-area  
justify-self & align-self  
place-self
```

GRID COLUMN & ROW

```
grid-column: start-coloană / end-coloană
```

- definește coloanele în care se plasează un element
- prescurtare pentru proprietățile grid-column-start și grid-column-end

```
grid-row: start-linie / end-linie
```

- definește rândurile în care se plasează un element
- prescurtare pentru proprietățile grid-row-start și grid-row-end

GRID AREA

```
grid-area: start-linie / start-coloană  
          / end-linie / end-coloană
```

- prescurtare pentru proprietățile
grid-row-start, grid-column-start, grid-row-end,
grid-column-end

GRID AREA

```
grid-area: nume-item
```

- poate denumi un item al gridului care este referit apoi folosind proprietatea `grid-template-areas` pentru container

JUSTIFY & ALIGN SELF

```
justify-self: stretch /* start, end, center */
```

- aliniază un item al gridului în interiorul celulei, pe axa principală

```
align-self: stretch /* start, end, center */
```

- aliniază un item al gridului în interiorul celulei, pe axa secundară

```
place-self: align-self justify-self
```

FLEXBOX VS. GRID

TEHNICI WEB

CSS #3

Claudia Chiriță . 2024/2025

CSS LAYOUT: MULTICOLOANE

- permite aranjarea conținutului pe mai multe coloane, ca într-un ziar
- în esență, un container cu proprietatea *column-count* sau *column-width*

```
column-count: numărul de coloane
```

```
column-width: lățimea unei coloane;
```

MULTICOL

- proprietăți pentru controlarea stilului într-un layout multicol:

```
column-gap: spațiul dintre coloane
```

```
column-span: câte coloane ocupă un element;
```

```
column-rule: stilul liniei care desparte coloanele;  
/* prescurtare column-rule: column-rule-style  
    column-rule-width column-rule-color*/
```

CSS ACROSS DEVICES

- adaptarea layoutului unei pagini web la diversele dispozitive ale utilizatorilor:
mobile (telefon, tabletă) & calculator

Adaptive/Mobile Friendly Design

VS.

Responsive Web Design

ADAPTIVE DESIGN

- aplicația server detectează dispozitivul utilizatorului și răspunde cu pagini diferite
- ex: webpage.com vs. m.webpage.com

RESPONSIVE DESIGN

o singură versiune a paginii care funcționează pe orice dispozitiv

avantaje:

- mai ușor de menținut
- SEO friendly
- posibilitatea de sharing a aceluiași URL între diverse dispozitive

MEDIA QUERIES

- aplicarea unui stil CSS în funcție de tipul general al unui dispozitiv sau alte caracteristici precum rezoluția ecranului sau lățimea browserului
- aplicarea condiționată a unui stil folosind regulile @media și @import
- pentru specificarea media pentru elemente precum <style>, <link>, <source> folosind atributul media=
- pentru **accesibilitate**

MEDIA QUERIES

```
media type: screen, print, speech
```

```
media features: width, height, max-height, max-width,  
                orientation, resolution, aspect-ratio
```

sintaxă media query:

```
media type and (med-feature1:val) and (med-feature2:val)...
```

MEDIA QUERIES HTML

- atributul HTML *media*
- folosirea unor stiluri CSS diferite în funcție de proprietățile dispozitivului media

```
<head>  
  <link rel="stylesheet" type="text/css" media="screen and (ma  
  <link rel="stylesheet" type="text/css" media="print" href="p  
</head>
```

MEDIA QUERIES CSS

- CSS at-rule @media

```
@media media-query1, media-query2
```

exemplu:

```
@media screen and (max-width: 960px) and (min-width: 500px)
{
  body { background-color: pink; }
}
```

OPERATORI @MEDIA

and, only, not și ","

- *only* e folosit pentru browsere mai vechi care nu-l recunosc, css-ul din media query fiind ignorat
@media only x este echivalent cu @media x
- operatorul "," are rol de disjuncție

VIEWPORT

```
<head>  
  <meta name="viewport" content="width=device-width, initial  
</head>
```

indică browserului că pagina trebuie să fie scalată
pentru a se potrivi cu dimensiunea ecranului

UNITĂȚI DE MĂSURĂ RELATIVE

- unități de lungime relative la dimensiunea elementului părinte / a fontului elementului părinte, ori a dimensiunii viewportului
- ex: em, rem, vw, vh, vmin, vmax

TEXT RESPONSIVE

- folosind unități de măsură ale viewportului (vw, vh, vmin, vmax)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      h1 {font-size: 5vw;}
      h2 {font-size: 4vw;}
      p {font-size: calc(0.5rem + 1vw);}
    </style>
  </head>
  <body>
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
    <p>Paragraph</p>
  </body>
</html>
```

>

TEXT RESPONSIVE

- folosind media queries

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      @media screen and (min-width:768px){
body{font-size:20px;}
      }
      @media screen and (max-width:768px){
body{font-size:12px;}
      }
      h1 {font-size: 3em;}
      h2 {font-size: 1.5em;}
      p {font-size: 0.9em;}
    </style>
  </head>
</body>
```



IMAGINI RESPONSIVE

- elementele `img`, `video`, `iframe` nu se redimensionează cu cod HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #container{
        width:80%;
        border: 2px solid red;
        float: left;}
    </style>
  </head>
  <body>
    <div id="container">
      
      <p>The red panda (Ailurus fulgens),
also known as the lesser panda, is a small
```

>

IMAGINI RESPONSIVE

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #container{
        width:80%;
        border: 2px solid red;
        float: left;}
      img{
        max-width:100%;
        height:auto;}
    </style>
  </head>
  <body>
    <div id="container">
      
      <p>The red panda (Ailurus fulgens),
```



IMAGINI RESPONSIVE

pentru dimensiuni diferite se vor afișa imagini diferite;
folosind elementele html <picture> și <source>

```
<picture>  
  <source srcset="small.jpg" media="(max-width:500px)">  
  <source srcset="med.jpg" media="(max-width:1000px)">  
    
</picture>
```

IMAGINI BACKGROUND RESPONSIVE

```
background-image: url(adresă-imagine);  
background-size: cover;  
/* imaginea acoperă întregul container*/
```

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <link rel="stylesheet" href="demo.css">  
    <style type="text/css">  
      body {margin-left:auto;  
        margin-right:auto; width:90%;  
        background-image:url(boo.jpg);  
        background-repeat: no-repeat;  
        background-size:cover;}  
    </style>  
  </head>  
  <body>  
  </body>  
</html>
```



FLASH QUIZ: ALINIERI

centrare pe orizontală

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        /* margin-left: ?;
        margin-right: ?; */
      }
      a { /*margin: ?;
        width: 100?;
        display: ?*/
      }
    </style>
  </head>
```



FLASH QUIZ: ALINIERI

to the right!

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        /*position: ?; */
      }
    </style>
  </head>
  <body>
    <div id="square">The Square</div>
  </body>
</html>
```



FLASH QUIZ: ALINIERI

centrare pe verticală

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        /* display: ?;
        vertical-align: ?;*/
      }
      #parinte{ line-height:300px;
        height: 300px; }
    </style>
  </head>
  <body>
    <div id="parinte">
```

>

TRANSFORMĂRI

TRANSFORMĂRI CSS

- schimbarea poziției și forme conținutului unui element HTML fără afectarea fluxului normal al documentului
- transformări afine ale elementelor (box model)
- rotații, scalări, translații, forfecări în plan și 3D

TRANSFORM

```
transform: metodă(valori);
```

- metode de transformare 2D:

```
translate(), rotate(), scale(), skew(), matrix()
```

- listă de transformări separate prin spațiu;
transformările sunt aplicate pe rând

-

```
transform-origin
```

specifică poziția originii

TRANSFORM: TRANSLATE

```
transform: translate(tx,ty)
          translateX(tx), translateY(ty);
```

mută elementul de la poziția inițială, conform parametrilor dați: [axa-X] și [axa-Y]

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        /*transform: translate(10px,30px);*/
      }
    </style>
  </head>
  <body>
    <div id="square">The Square</div>
  </body>
</html>
```



TRANSFORM: ROTATE

```
transform: rotate(grade);
```

rotește elementul în sensul acelor de ceasornic dacă valoarea grade este pozitivă și invers dacă valoarea este negativă

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px;
        width: 150px;
        /*rotate: -45deg;*/
      }
    </style>
  </head>
  <body>
    <div id="square">The Square</div>
  </body>
</html>
```

>

TRANSFORM: SCALE

```
transform: scale(sx, sy),  
          scaleX(sx), scaleY(sy);
```

mărește sau micșorează dimensiunile elementului în funcție de parametrii *sx* și *sy*

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link rel="stylesheet" href="demo.css">  
    <style type="text/css">  
      #square { background-color: khaki;  
        color: black;  
        height: 150px; width: 150px;  
        /*transform: scale(0.66);*/  
      }  
    </style>  
  </head>  
  <body>  
    <div id="square">The Square</div>  
  </body>  
</html>
```



TRANSFORM: SKEW

```
transform: skew(ax, ay),  
          skewX(ax), skewY(ay);
```

distorsionează elementul de-a lungul axelor X și Y în funcție de parametrii specificați

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link rel="stylesheet" href="demo.css">  
    <style type="text/css">  
      #square { background-color: khaki;  
        color: black;  
        height: 150px; width: 150px;  
        /*transform: skew(12deg, 12deg);*/  
      }  
    </style>  
  </head>  
  <body>  
    <div id="square">The Square</div>  
  </body>  
</html>
```



TRANSFORM

unui element îi pot fi aplicate mai multe transformări
în același timp

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square { background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        /*transform: translate(25px,30px)
rotate(45deg) scale(1.5);*/
      }
    </style>
  </head>
  <body>
    <div id="square">The Square</div>
  </body>
</html>
```

>

ACCESIBILITATE

- animațiile cu scalare/zoom pot ridica probleme din punct de vedere al accesibilității, putând provoca migrene
- e recomandată adăugarea unui mijloc de control pentru oprirea animațiilor dintr-un site
- e recomandată folosirea feature-ului media `prefers-reduced-motion`

TRANSFORMĂRI 3D

- extensii ale transformărilor 2D
- efectul 3D se obține folosind proprietatea *perspective* (distanța pe axa Z)

```
translateZ(tz) -> translate3d(tx,ty,tz)
```

```
rotateZ(az) -> rotate3d(x,y,z,grade)
```

```
scaleZ(sz) -> scale3d(sx,sy,sz)
```

```
matrix3d(n,n,...,n)
```

```
perspective(n)
```

TRANZIȚII

- controlul vitezei de animație la schimbarea valorii unei proprietăți CSS: schimbarea poate să aibă loc de-a lungul unei perioade de timp, nu doar instantaneu
- declanșate de evenimente

```
:hover, :focus, :active
```

TRANZIȚII

proprietățile necesare definirii unei tranziții:

```
transition-property: proprietatea CSS de schimbat;  
transition-duration: s; /* durata tranziției */  
transition-timing-function: linear; /* ease, ease-in,  
                                ease-out, step-start, step-end  
                                (funcții predefinite) */  
transition-delay: s; /* durata de așteptare înainte  
                    de începerea tranziției */  
  
transition: property duration timing-function delay;
```

proprietăți cărora li se poate aplica o tranziție

TRANZIȚII

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square {background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        transition: width 2s, height 2s,
          background-color 2s, transform 2s;}
      #square:hover { background-color:
pink;
        height: 100px;
        width: 100px;
        transform: rotate(45deg); }
    </style>
  </head>
```



ANIMAȚII CSS

- constau din două componente:
 - un stil care descrie animația
 - o mulțime de *keyframes* care indică stările inițiale și finale ale animației și posibile puncte intermediare

KEYFRAMES

```
@keyframes nume-animatie {  
  0% {...}  
  25% {...}  
  ...  
  75% {...}  
  100% {...}  
}
```

```
@keyframes nume-animatie {  
  from {...} /* echivalent cu 0% */  
  to {...} /* echivalent cu 100% */  
}
```

ANIMATION

```
animation-name: nume-animatie;
```

```
animation-duration: s; /* obligatorie */
```

```
animation-timing-function: linear /*ease, ease-in, ease-out*/
```

```
animation-delay: s;
```

```
animation-iteration-count: val; /* infinite */
```

```
animation-direction:normal /* alternate, reverse */
```

```
animation-fill-mode: none /* forwards, backwards, both */
```

ANIMATION

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style type="text/css">
      #square {background-color: khaki;
        color: black;
        height: 150px; width: 150px;
        animation-name: thesquare;
        animation-duration: 6s;
        animation-iteration-count:
infinite;}

      @keyframes thesquare {
        0% {background-color: khaki;}
        25% {background-color: yellow;}
        50% {background-color: orange;}
        100% {background-color: red;}
      }
    </style>
  </head>
  <body>
```



THE BRB

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style>
      .brb {
        height: 350px;
        width: 350px;
        display: inline-block;
        margin: 60px 0 60px 0;
      }
      div { text-align:center; }
    </style>
  </head>
  <body>
    <div>
      <span class="brb"></span>
    </div>
  </body>
</html>
```

>

THE BRB

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style>
      .brb {
        height: 350px;
        width: 350px;
        border-radius: 50%;
        display: inline-block;
        margin: 60px 0 60px 0;
      }
      div { text-align:center; }
    </style>
  </head>
  <body>
    <div>
      <span class="brb"></span>
    </div>
  </body>
</html>
```

>

THE BRB

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style>
      .brb {
        height: 350px;
        width: 350px;
        border-radius: 50%;
        display: inline-block;
        margin: 60px 0 60px 0;
      }
      .brb:active {
        background-color: red;
      }
      div { text-align:center; }
    </style>
  </head>
  <body>
    <div>
      <span class="brb"></span>
    </div>
  </body>
</html>
```



THE BRB

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="demo.css">
    <style>
      .brb {
        height: 350px;
        width: 350px;
        border-radius: 50%;
        display: inline-block;
        margin: 60px 0 60px 0;
        background-color: red;
      }
      .brb:active {
        animation-name: thebrb;
        animation-duration: .5s;
      }

      @keyframes thebrb {
        0% {height: 350px; width: 350px;}
        50% {height: 330px; width: 330px;
opacity:0.75;}
        100% {height: 350px; width: 350px;
opacity:1;}
      }
      div { text-align:center; }
    </style>
  </head>
```



THE BRB

```
<!DOCTYPE html>
<html lang="en">
  <link rel="stylesheet" href="demo.css">
  <style>
    .brb {
      height: 350px;
      width: 350px;
      border-radius: 50%;
      display: inline-block;
      margin: 60px 0 60px 0;
      background-color: red;
    }
    .brb:active {
      animation-name: thebrb;
      animation-duration: 1.25s;
      animation-iteration-count: infinite;
    }

    @keyframes thebrb {
      0% {height: 350px; width: 350px;}
      20% {height: 360px; width: 360px;
opacity:0.75;}
      40% {height: 350px; width: 350px;
opacity:1;}
      60% {height: 370px; width: 370px;
opacity:0.75;}
      80% {height: 350px; width: 350px;
opacity:1;}
    }
  </style>
</html>
```



TEHNICI WEB

JAVASCRIPT

Claudia Chiriță . 2024/2025

JAVASCRIPT

- dezvoltat de **Brendan Eich** în 1995, la Netscape, sub numele de Mocha și LiveScript
- standardizat în 1997 de asociația ECMA (European Computer Manufacturer's Association) sub numele de *ECMAScript*
- versiunea actuală: **ECMAScript 2025**

JAVASCRIPT - WHAT?

- limbaj lightweight de **scripting** pentru pagini web
- combină mai multe paradigme de programare:
imperativă, funcțională, orientată pe obiecte, bazată pe **prototipuri** și pe evenimente

JAVASCRIPT - WHAT?

- limbaj **interpretat** (scriptul este executat direct, fără compilare prealabilă)
- codul rulează în browserul clientului
- single-thread, **dinamic**

JAVASCRIPT - RTM

JavaScript The Definitive Guide, David Flanagan

Understanding ECMAScript 6, Nicholas C. Zakas

JAVASCRIPT + HTML

- o pagină web este reprezentată în memorie ca un arbore de elemente (obiecte)

Document Object Model – DOM

- JavaScript poate interacționa cu documentul HTML prin intermediul DOM-ului

JAVASCRIPT + HTML

- element HTML --- > object JavaScript

```
<h1 id="identif">...</h1>  
ob = document.getElementById("identif")
```

- atribut al unui element HTML --- > proprietate a obiectului JavaScript

```
<h1 id="identif" class="cls">...</h1>  
ob.id, ob.className  
  
ob.src
```

JAVASCRIPT + HTML

- atributul style: proprietăți CSS ---> proprietatea style
-> obiectul style -> proprietăți de stilizare CSS

```
<h1 style="color:pink; text-align:center;">...</h1>  
ob.style.color, ob.style.textAlign
```

JAVASCRIPT @ HTML

codul JavaScript poate fi plasat inline, în head, folosind tagul `<script>`

```
<head>  
<script type="text/javascript">  
/* cod JavaScript */  
</script>  
</head>
```

JAVASCRIPT @ HTML

codul JavaScript poate fi plasat într-un fișier extern, importat în documentul HTML [varianta recomandată]

```
<script type="text/javascript" src="script-file.js"></script>
```

atribute suplimentare, doar pentru scripturi externe:

- **defer** - întârzie executarea codului până se încarcă complet documentul HTML
- **async** permite încărcarea codului asincron față de document

JAVASCRIPT - HOW?

JavaScript este case sensitive și folosește setul de caractere Unicode

- identificatorii
 - denumesc variabile, cuvinte cheie, funcții, etichete
 - formați din: cifre, litere, _, \$;
 - primul caracter: literă, _, \$
- ; separator
- { } bloc de instrucțiuni

```
-  
/* comentariu  
pe mai multe linii  
*/  
// comentariu pe o singură linie
```

JAVASCRIPT - WHERE?

JavaScript debugger in browser

<https://jsfiddle.net/>

<https://codepen.io/>

DIALOG: ALERT & PROMPT

un tab al unui browser e reprezentat de un obiect
window (interfața **Window**)

```
alert(mesaj); // afișează o casetă de alertare care  
              // conține un mesaj și un buton OK
```

DIALOG: ALERT & PROMPT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /* var x = prompt("gimme gimme gimme a
number!");
      var y = prompt("gimme another
number!");
      alert(typeof(x));
      alert(x+y);
      alert(parseInt(x)+parseInt(y)); */
    </script>
  </head>
  <body>
  </body>
</html>
```

>

DIALOG: ALERT & PROMPT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /* var n = prompt("gimme some number!");
       alert('suma este ' +
suma(parseInt(n)));

      function suma(x) {
        var s = 0;
        for (var i=1; i<x; i++) s=s+i;
        return s;} */
    </script>
  </head>
  <body>
  </body>
</html>
```



CORE JAVASCRIPT

TIPURI DE DATE

- primitive:
 - boolean
 - null, undefined
 - number, bigint, string
 - symbol
- obiecte (tipul **Object**)
obiecte predefinite: **Array, String, Number, Boolean, Math, Date, Set, Function, Error, RegExp**

CORE JAVASCRIPT

INSTRUCȚIUNI

- atribuire: =
- control flow: if...else, switch
- loops & iteration: while, for
- return

CORE JAVASCRIPT

FUNCTII

```
function nume(param1, param2, ...)  
{  
    corpul functiei  
}
```

CONVERSIE TIPURI

- JavaScript e *dynamically typed*
- nu este necesară precizarea tipului de date, ca în alte limbaje de programare
- tipurile de date sunt convertite automat, la nevoie, în timpul execuției

```
x = 23; //number
y = "abc"; //string
z = true; //boolean
var a; //undefined
cat = {nume:"Tigger", culoare:negru}; //obiect
```

VARIABLE - DECLARARE

- variabilele declarate
 - în interiorul funcțiilor sunt locale pentru acele funcții
 - în afara oricărei funcții se numesc variabile *globale*
 - într-un domeniu (*scope*) sunt accesibile în funcțiile copil
- variabilele trebuie declarate mereu înainte de folosire; JavaScript permitea asignarea de valori unei variabile nedecarate, creând o variabilă globală undeclared - EROARE în strict mode

VARIABLE - INIȚIALIZARE

- se poate atribui o valoare inițială la declarare
- dacă o variabilă este declarată fără inițializare, i se asignează valoarea *undefined*

VARIABLE - DECLARARE - VAR

- declarate explicit folosind cuvântul cheie var

```
var x = 42;
```

- putem declara variabile fie global, fie local la nivel de funcție

VARIABLE - DECLARARE - VAR

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /* var x = 0; // var globală
      function f(){
        var y = 1; // var locală
        var x = 5; // var locală
        function g(){
          y++;
          z = 1; // var globală; eroare?
          console.log(y);
        }
        g();
      }
      f(); //va afișa 2
      console.log(x); //va afișa 0
```

>

VARIABLE - DECLARARE - LET

- declarate explicit folosind cuvântul cheie `let`

```
let x = 42;
```

- declară o variabilă locală vizibilă doar în blocul, instrucțiunea sau expresia în care este folosită

VARIABLE - DECLARARE - LET

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /* var g = 0; // var globală
       { let g = 1; }
       console.log (g); // va afișa 0
      */
    </script>
  </head>
  <body>
  </body>
</html>
```

>

VARIABLE - DECLARARE - LET

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /* function letTest() {
        let x = 1;
        if (true) {
          let x = 2;
          console.log(x); // 2
        }
        console.log(x); // 1
      }
      letTest(); */
    </script>
  </head>
  <body>
  </body>
</html>
```

>

VARIABLE - DECLARARE - CONST

- putem declara constante *block-scoped* folosind cuvântul cheie **const**

```
const x = 42;
```

- se inițializează la declarare
- nu se pot modifica pe parcursul execuției programului prin re-atribuire; nu pot fi redeclarate

VARIABLE - TYPEOF

- variabilele au tipuri dinamice: pot fi declarate (și pot primi valori) cu orice tip de date
- tipul variabilelor nu este specificat explicit, dar poate fi aflat cu

```
typeof (x) ;
```

- tipul unei variabile nedeclarate este *undefined*

VARIABLE - TYPEOF

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /*
        x = 42;
        console.log(typeof x); // "number"
        x = "fish";
        console.log(typeof x); // "string"
        x = true;
        console.log(typeof x); // "boolean"
        console.log(typeof undeclaredVar); //
"undefined"
        */
      </script>
    </head>
    <body>
    </body>
  </html>
```



SCOPE - DOMENIU VIZIBILITATE

- domeniul de vizibilitate (*scope*) al variabilelor: zona din program în care sunt declarate variabilele
- scope global, scope funcție (local), scope bloc
- variabilele declarate de o funcție părinte sunt vizibile în descendenții ei

```
// scope global
function fA () {
  //scope A
  function fB () {
    // scope B
  }
}
```


SCOPE - DOMENIU VIZIBILITATE

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script type="text/javascript">
      /*
        function calc(an){
          var varsta = 2024 - an;
          function print(){
            const str = nume + " are " +
varsta
              + " ani, născută în " +
an;
            if (varsta >= 7) {
              const oldCat = true;
            }
            console.log(str);
            //console.log(oldCat); // eroare
          }
          print();
        }
      */
    </script>
  </head>
</html>
```

>

SCOPE - HOISTING

- ne putem referi la variabilele declarate cu **var** oriunde în domeniul lor de vizibilitate, chiar și înainte de punctul în care sunt declarate
- înainte de a fi executat, codul JavaScript este "rearanjat" a.î. toate declarațiile de variabile (nu și operațiile de atribuire) sunt mutate (ridicate) la începutul zonei de vizibilitate

SCOPE - HOISTING

```
var x = 5; //globală
function hoist() { if (x == 5) { var x = 10; }
                  alert(x); }
hoist(); // va afișa undefined
alert(x); // va afișa 5
```

~

```
var x = 5; //globală
function hoist() { var x; // variabilă locală
                  if (x == 5) { // undefined
                      var x = 10; }
                  alert(x); }
hoist(); // va afișa undefined
alert(x); // va afișa 5
```

TIPUL NUMBER

- reprezentare binară pe 64 biți; întregi și float

```
var i = 4;  
var r = 42.2;
```

- operatorii aritmetici specifici: + - * / % ++ --
- conversie de tip automată:

```
x = "2" * 7; // 14  
y = "2" + 7; // "27"  
z = parseInt("2") + 7; // 9  
t = "2" * "7"; // 14  
u = 2 + 3 + "4"; // "54"  
z = "2" + 3 + 4; // "234"
```

TIPUL NUMBER

obiectul built-in **Math**

```
Math.PI //=> 3.14  
Math.pow(2,3) // => 8  
Math.round(4.7) // => 5  
Math.random() // între 0 și 1  
Math.sqrt(-1) // => NaN
```

TIPUL BOOLEAN

orice valoare poate fi convertită explicit folosind
obiectul predefinit Boolean

```
var nume = Boolean(valoare);  
true === Boolean("adevarat") // => true  
false === Boolean("") // => true
```

- operatori logici pentru tipuri primitive:
> < <= >= && || ! == === != !==
- putem folosi și alte tipuri de date în context logic:
 - pentru fals: 0, "", NaN, null, undefined
 - pentru adevărat: orice altă valoare

TIPUL STRING

șir de caractere scris între ' ' sau " "

```
var s = "Tigger Lilly";  
var t = 'Tigger Lilly';  
var pnume = s.slice (0, s.indexOf(" ")); // 'Tigger'  
var nume = s.slice(s.lastIndexOf(" ")+1,s.length); // 'Lilly'
```

proprietăți și metode:

```
length, charAt(), indexOf(), lastIndexOf(), replace(),  
split(), toLowerCase(), concat()
```

TIPUL STRING

- concatenare: "numărul" + "1", "numărul" + 1
- caractere speciale: \' \'\' \n \t \v \b \\\
- accesarea unui caracter: s[pozitie], s.charAt(pozitie)

```
var x = "abcde";  
alert(x[0]);  
x[0] = 'v';  
alert(x[0]); // => a
```


TIPURILE UNDEFINED & NULL

- variabilele care nu au primit încă o valoare au tipul **undefined**

```
var x
x == undefined // true
typeof(x) // undefined
```

- **null**: lipsa unei valori (intenționată)

```
var x = null
typeof(x) // object !
null == undefined // true
null === undefined // false
```

TIPUL OBJECT

- un obiect este o colecție de perechi proprietate-valoare
- dacă valoarea este o funcție, atunci proprietatea se numește metodă
- ```
var ob = {prop1: val1, prop2: val2, ... , propn: valn};
```
- accesarea proprietăților:

```
ob.prop1; // val1
ob["prop1"]; // val1
```

# TIPUL OBJECT

```
var student = { nume: "Tigger",
 nota1: 9,
 nota2: 10,
 media: function() {
 return (this.nota1 + this.nota2)/2;
 }
 }
```

```
student.nume \\ "Tigger"
student.nota1 \\ 9
student.nota2 \\ 10
student.media() \\ 9.5
student.media \\ funcția
```

# TIPUL OBJECT

- toate datele din tipurile primitive sunt transmise prin valoare
- datele de tip object sunt transmise prin referință

```
var a = {nume: "Tigger"} // object
var b = a; // a și b referă aceeași zonă
b.nume = b.nume + " Lilly"; // se modifică și b și a
alert(a.nume); // "Tigger Lilly"
```

```
var s = "Tigger"; // string
var t = s; // t copiază valoarea lui s
t = t + " Lilly" ; // se modifică doar t
alert(s) // => "Tigger"
```

# CREAREA OBIECTELOR

- prin **object literal**
- proprietățile, metodele, împreună cu valorile lor sunt enumerate între acolade
- se creează un singur obiect

```
var cat = {nume:"Tigger", culoare:"neagră", vârstă:14}
```

# CREAREA OBIECTELOR

- cu ajutorul **obiectului generic**
- se apelează constructorul `new Object()` și se adaugă apoi proprietățile și metodele
- se creează un singur obiect

```
var cat = new Object();
cat.nume = "Tigger";
cat.culoare = "neagră";
cat.vârstă = 14;
```

# CREAREA OBIECTELOR

- cu ajutorul unui constructor de obiecte
- se definește o funcție constructor(param) care apoi va fi apelată cu new constructor(param) pentru fiecare obiect care va fi creat

```
function cat(n, c, v) {
 this.num = n;
 this.culoare = c;
 this.vârsta = v;
}

var c1 = new cat("Tigger", "neagră", 14);
var c2 = new cat("Fluff", "albă", 2);
```

# BUILT-IN OBJECTS

proprietăți globale: returnează o valoare simplă

- Infinity
- NaN
- undefined



# BUILT-IN OBJECTS

metode globale: apelate global, nu pentru un obiect

- `isNaN()` - determină dacă valoarea este un număr invalid
- `parseInt()` - convertește un șir într-un întreg
- `parseFloat()` - convertește un șir într-un float
- `Number()` - convertește un obiect într-un număr
- `String()` - convertește un obiect într-un șir

# OBIECTE PREDEFINITE

- obiecte corespunzătoare tipurilor primitive  
String, Number, Boolean
- Array, Set, Map, Function, RegExp, Date
- se pot crea obiecte noi cu new

# OBIECTE PREDEFINITE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /*
 var y = new Number(123);
 alert(typeof(y)); // object

 var ob = new Object();
 ob.x = 1; ob.y = 2; // ob = {x:1, y:2}

 var d = new Date(2023,2,27);
 alert(d.getUTCDay()); // 0 (ziua din
 săptămână (0-6))
 */
 </script>
 </head>
 <body>
 </body>
</html>
```

>

# OBIECTE PREDEFINITE - ARRAY

```
var v = new Array();
v[0] = "a"; v[1] = "b";
```

```
var v = new Array("a", "b");
```

```
var v = ["hi", 2, [5, 7]];
v = [6, 4, 7, 3];
```

```
v.length; // 4
v.push(10); // => v = [6, 4, 7, 3, 10]
v.pop(); // => v = [6, 4, 7, 3]
v.shift(); // => v = [4, 7, 3]
v.unshift(10); // => v = [10, 4, 7, 3]
v.sort(); // => v = [3, 4, 6, 7]
```

# OBIECTE PREDEFINITE - ARRAY

```
var s = "azi este luni";
var a = s.split(" "); // a = ["azi", "este", "luni"]
a.reverse(); // a = ["luni", "este", "azi"]
var s = a.join('/'); // s = "joi/este/azi"
```

# FUNCȚII

```
function nume(arg1, arg2, ..., argn) {
 instrucțiuni;
 [return valoare;] // optional
}
```

o funcție JavaScript poate fi apelată cu un număr variabil de parametri

```
function suma(a,b) {
 return a+b;
}

suma(2,3); // 5
suma(); // NaN
suma(2); // NaN
suma(3,4,1,5,6,7) // 7
```

# FUNCȚII - ARGUMENTS

orice funcție poate accesa un obiect "arguments" (asemănător unui array) care conține valorile argumentelor cu care se apelează funcția

```
function fun() {
 return arguments.length;
}
```

```
fun(2, "sss", 5); // 3
```

# FUNȚII - ARGUMENTS

```
function func(a, b, c) {
 console.log(arguments[0]); // expected output: 1
 console.log(arguments[1]); // expected output: 2
 console.log(arguments[2]); // expected output: 3
}

func(1, 2, 3);
```



# FUNCȚII - ARGUMENTS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* var n = prompt("gimme some number!");
 alert('suma este ' + suma(parseInt(n)));

 function suma() {
 var s = 0;
 for(var i=0; i<arguments.length; i++)
 s+=arguments[i];
 return s;
 } */
 </script>
 </head>
 <body>
 </body>
</html>
```

>

# FUNȚII ANONIME

## function expressions

```
function (arg1, ..., argn) {
 instrucțiuni;
}
```

```
const fun = function () {
 return arguments.length;
}
```

## arrow functions

```
var f = (x) => { x+1 }
```

# ARRAY - MAP()

```
array.map(function(currentValue, index, arr), thisValue)
```

- creează un nou array prin apelarea unei funcții (dată ca parametru) pentru fiecare element din array
- nu modifică array-ul inițial
- *index, arr, thisValue* sunt opționale

# ARRAY - MAP()

```
function f(x) {
 return x+1;
}
var array = [1,2,3,4];
var array1 = array.map(f);
console.log(array1); // [2,3,4,5]
```

```
function g(x) {
 return x * this.a;
}
var o1 = {a:2}, o2 = {a:3};
var array = [1,2,3,4];
var array1 = array.map(g,o1); // [2,4,6,8]
var array2 = array.map(g,o2); // [3,6,9,12]
```

# ARRAY - FOREACH()

```
array.forEach(function(currentValue, index, arr), thisValue)
```

- apelează o funcție dată ca parametru pentru fiecare element dintr-un array
- *index, arr, thisValue* sunt opționale

# ARRAY - FOREACH()

```
array.forEach(function(currentValue, index, arr), thisValue)
```

```
function f(x, i) {
 alert(i + " : " + x);
}
```

```
var array = ['luni', 'marți', 'miercuri', 'joi'];
array.forEach(f);
```

```
var d = [1, 2, 3, 4];
d.forEach(function(v, i, a) {a[i] = v + 1;});
console.log(d); // [2,3,4,5];
```

# ARRAY - FILTER()

```
array.filter(function(currentValue, index, arr), thisValue)
```

- creează un nou array cu elementele care verifică condiția implementată de funcția dată ca parametru
- nu modifică array-ul inițial
- *index, arr, thisValue* sunt opționale

# ARRAY - FILTER()

```
function check(word) {
 return word.length < 6;
}

var animals = ["cat", "tapir", "platypus", "red panda"];
var smol = animals.filter(check);
console.log(smol); // ["cat", "tapir"]
```



# ARRAY - REDUCE()

```
array.reduce(function(total, currentValue, currentIndex, arr),
```

- execută o funcție de reducere (dată ca parametru) pentru fiecare element al array-ului rezultând o singură valoare de ieșire
- nu modifică array-ul inițial
- *currentIndex, arr, initialValue* sunt opționale

# ARRAY - REDUCE()

```
function suma(total, val) {
 return total+val;
}

var array = [1,2,3,4];
var s = array.reduce(suma);
console.log(s); // 10;
```

# ARRAY

```
var cats = [{nume:"Tigger",ani:7},{nume:"Whiskers",ani:6},
 {nume:"Fluff",ani:3},{nume:"Kit",ani:2},
 {nume:"Tom",ani:6},{nume:"Berlitz",ani:4}];
var copy = cats.slice();
var oldCats = copy.filter((x) => (x.ani >= 6))
 .map((x) => (x.nume+" are "+x.ani+" ani"))
 .forEach((x) => {alert(x)});
```

**CLASE?**





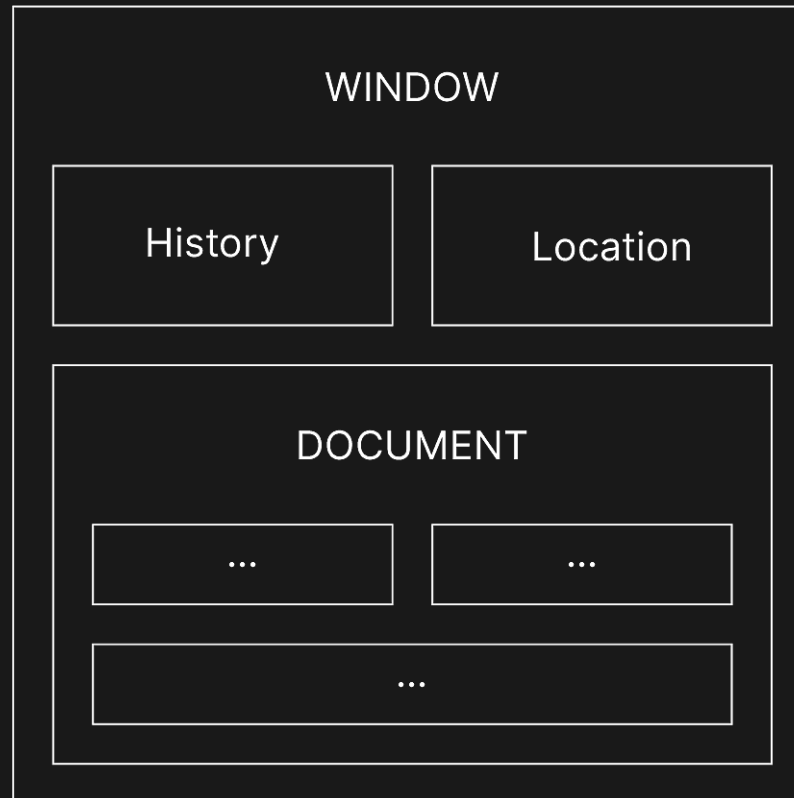
# TEHNICI WEB

# DOM

*Claudia Chiriță . 2024/2025*

# WINDOW

- orice tab al unui browser conține un obiect *window* al interfeței *Window*





# WINDOW

- proprietatea *document* a obiectului *window* are ca valoare obiectul document asociat paginii web

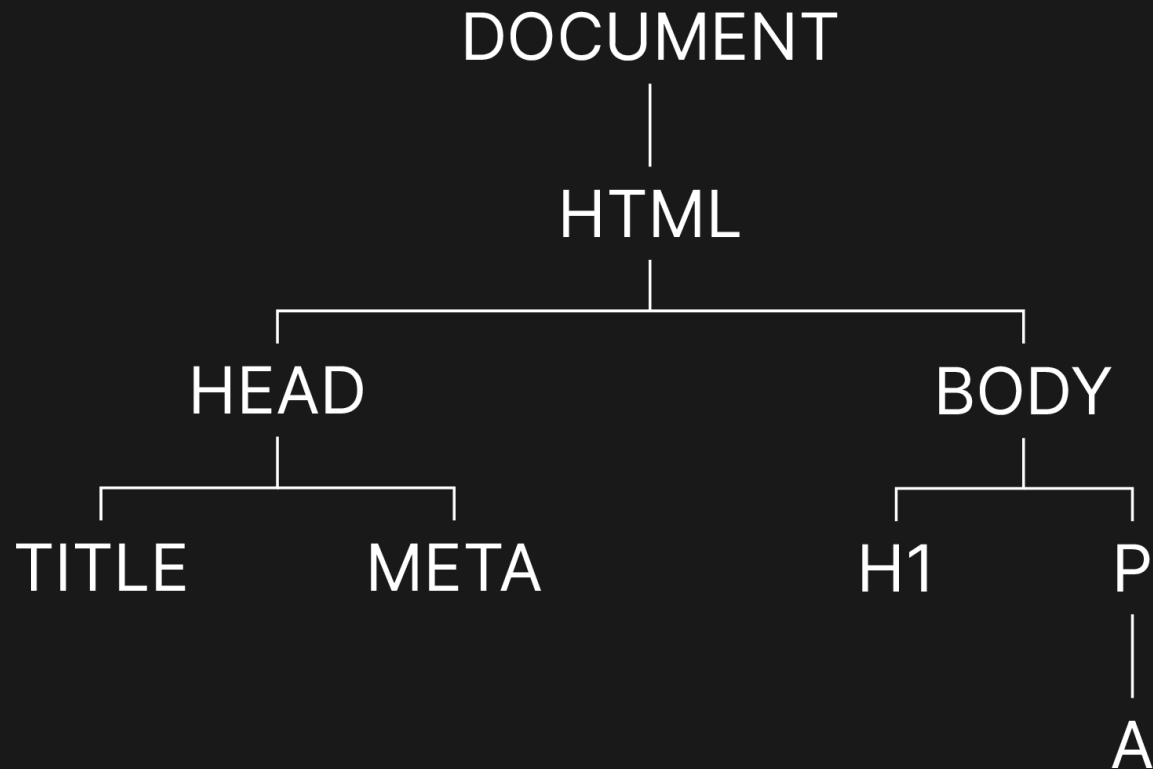
```
window.document // referința
```

- alte proprietăți: history, location, console, localStorage, sessionStorage

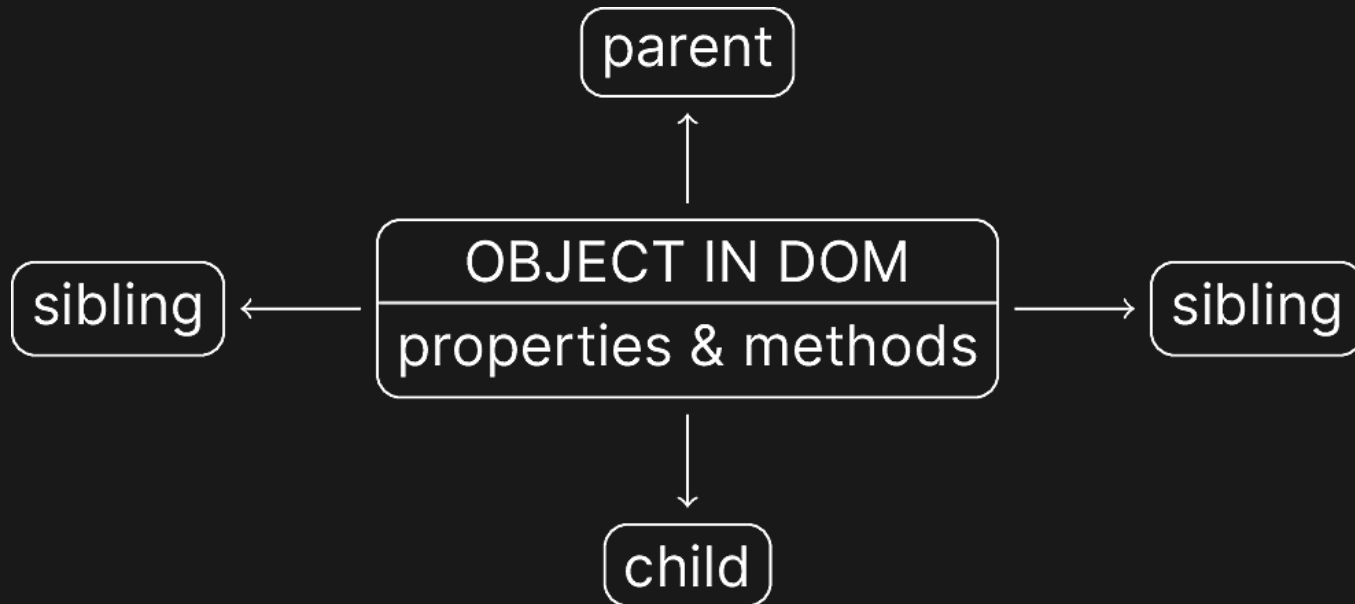
# DOM (DOCUMENT OBJECT MODEL)

- API web
- JavaScript folosește DOM pentru accesarea și modificarea documentelor HTML
- orice pagină web este reprezentată în DOM ca un arbore de obiecte și este accesată folosind variabila globală *document*

# DOM (DOCUMENT OBJECT MODEL)



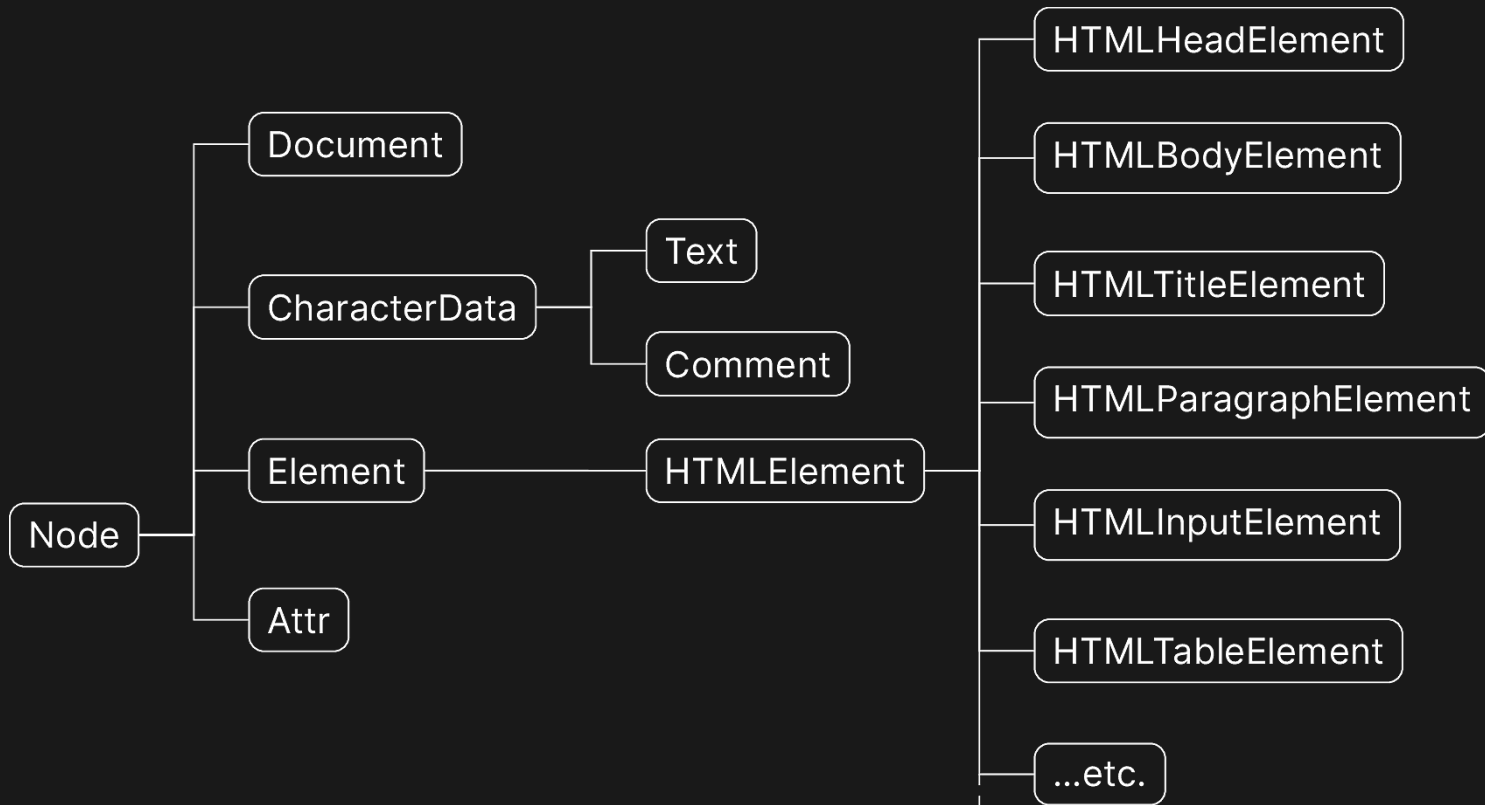
# DOM - NODURI



## DOM - NODURI

- orice nod din arborele DOM are un tip
- în funcție de elementul pe care îl reprezintă, obiectul corespunzător are proprietăți și metode specifice

# DOM - NODURI



ierarhia tipurilor nodurilor din DOM

# DOM VS. HTML

- unui tag HTML îi corespunde un obiect (care implementează interfața *Element*)
- unui atribut al unui tag HTML îi corespunde o proprietate a obiectului
- attributele (generale) HTML *id*, *class*, *style* corespund proprietăților *id*, *className*, *style* ale obiectului corespondent

# DOM VS. HTML

- proprietățile obiectului *style* (care implementează interfața *Style*) asociat atributului HTML *style* corespund proprietăților de stilizare CSS pentru elementele HTML

```
/* element.style.proprietateCSS = valoare */
```

```
el.style.color = "red"
```

```
el.style.backgroundColor = "blue"
```



# DOM VS. HTML - EXEMPLU

```

```

```
object-Element-în-DOM = {tagName: "img", src: "cat.png",
 alt: "kitty", id: "catImg"}
```

```
function change() {
 var x = document.getElementById('catImg'); // object
 x.alt = "image of a soft warm happy kitty";
 x.style.border = "10px solid pink";
}
```

# LOAD

- elementul `<script>` din `<head>` este procesat înaintea elementului `<body>`
- arborele DOM este creat mai apoi și deci elementele lui nu pot fi accesate
- încărcarea paginii declanșează evenimentul *load* care are ca țintă obiectul *window*

# LOAD

- un handler pentru evenimentul *load* se înregistrează setând proprietatea *onload* pentru obiectul *window*

```
window.onload = function() { ... }
```

```
window.onload = myMain;
function myMain() { ... }
```

# LOAD

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload = change;
 function change(){
 var x =
document.getElementById('catImg'); // object
 x.alt = "image of a happy kitty";
 x.style.border = "10px solid pink";
 } */
 </script>
 </head>
 <body>

 </body>
</html>
```



# SELECTAREA ELEMENTELOR

```
document.getElementById(numeId) // un obiect
document.querySelector(selectorCSS) // primul obiect
```

## colecții *live*

```
document.getElementsByClassName(numeClasa)
document.getElementsByTagName(numeTag)
document.getElementsByName(nume)
```

## colecții *statice*

```
document.querySelectorAll(selectorCSS)
```

# COLECȚII

- colecții generice similare unui Array
- au proprietatea *length*
- metodele Array nu pot fi invocate direct
- exercițiu:

Afișați numărul elementelor `<h1>` care sunt descendenți direcți ai elementelor `<section>` cu clasa "special".

# COLECȚII

- colecții generice similare unui Array
- au proprietatea *length*
- metodele Array nu pot fi invocate direct
- exercițiu:

Afișați numărul elementelor <h1> care sunt descendenți direcți ai elementelor <section> cu clasa "special".

```
var col = document.querySelectorAll("section.special > h1");
alert(col.length);
```

# COLECȚII

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload=function() {
 var c =
document.getElementsByClassName("abc");
 var v = [];
 for(var i = 0; i < c.length; i++)
 v[i] = c[i];
 alert("lungimea colecției: " +
c.length + '\n' +
 "lungimea vectorului: " +
v.length);

document.getElementById("p1").className="abc";
 alert("lungimea colecției: " +
c.length + '\n' +
 "lungimea vectorului: " +
v.length);
 }*/
```

>



# COLECȚII

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload=function() {
 var c =
document.querySelectorAll(".abc");
 var v = [];
 for(var i = 0; i < c.length; i++)
 v[i] = c[i];
 alert("lungimea colecției: " +
c.length + '\n' +
 "lungimea vectorului: " +
v.length);

document.getElementById("p1").className="abc";
 alert("lungimea colecției: " +
c.length + '\n' +
 "lungimea vectorului: " +
v.length);
 }*/
```

>

# COLECȚII - QUIZ

Schimbarea numelui unei clase "old" în "new":

```
var colectie = document.getElementsByClassName("old");

for(var i = 0; i < colectie.length; i++)
 colectie[i].className = "new";

/* merge? */
```

# CONȚINUTUL ELEMENTELOR

accesarea și modificarea conținutului unui element  
HTML ca String folosind

- **innerHTML**
  - întoarce un text HTML, adică un text cu marcate
  - la setarea proprietății, browserul interpretează textul

```
<p>It was <i> the best </i> of times.</p>
<--! innerHTML -->
It was <i> the best </i> of times.
```

# CONȚINUTUL ELEMENTELOR

accesarea și modificarea conținutului unui element  
HTML ca String folosind

- **textContent**
  - întoarce text fără marcaje
  - are ca rezultat concatenarea conținuturilor descendenților de tip Text

```
<p>It was <i> the best </i> of times.</p>
<--! textContent -->
It was the best of times.
```

# NODURI DOM

tipuri de noduri:

- un nod special numit **document**
- noduri de tip **Element** (modelează tagurile HTML)
- noduri de tip **Text** (modelează textul)

# NODURI DOM

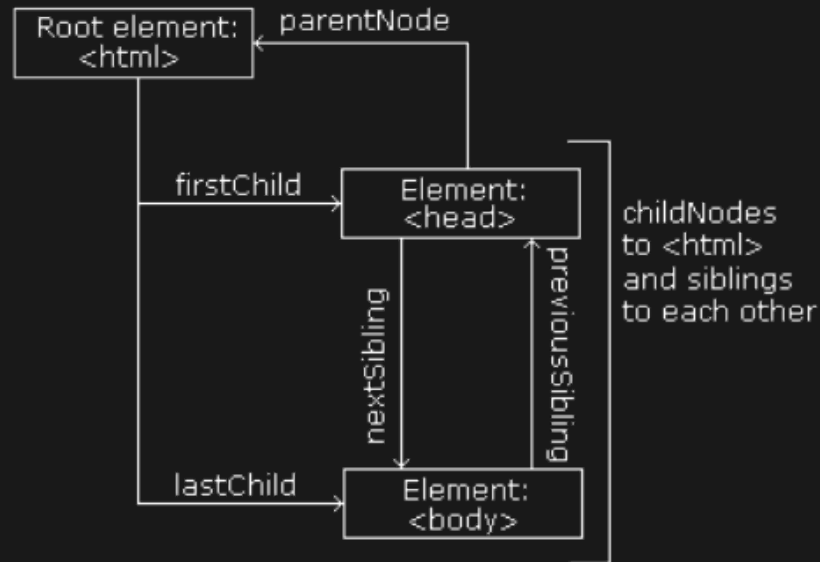
manipularea informației din noduri cu:

```
nodeValue /* pentru noduri Text, Comment
 pentru noduri Element = null */
nodeName // numele tagului
nodeType /* Document=9, Element=1,
 Text=3, Comment=8 */
```

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload=function() {
 var p =
document.getElementById("par");
 alert("nodeValue: " + p.nodeValue +
'\n' + "nodeName: "
 + p.nodeName + '\n' + "nodeType: " +
p.nodeType);
 }*/
 </script>
 </head>
 <body>
 <div id="par">Everybody wants to be a cat!
```



# NAVIGAREA ÎN ARBORE



```
node.parentNode // un obiect
node.childNodes // obiect NodeList
node.firstChild // un obiect
node.lastChild // un obiect
node.nextSibling // un obiect
node.previousSibling // un obiect
```

# NAVIGAREA ÎN ARBORE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload=function() {
 var c = document.body.childNodes;
 var txt = "";
 var i;
 for (i = 0; i < c.length; i++) {
 txt = txt + c[i].nodeName + "

";
 }

 document.getElementById("lista").innerHTML +=
txt;

 alert(document.getElementById("lista").childNodes[0].nodeName);
 }*/
 </script>
 </head>
```

>



# NAVIGAREA ÎN ARBORELE DE ELEMENTE

<code>node.parentNode</code>		<code>node.parentElement</code>
<code>node.childNodes</code>		<code>node.children</code>
<code>node.firstChild</code>		<code>node.firstElementChild</code>
<code>node.lastChild</code>		<code>node.lastElementChild</code>
<code>node.nextSibling</code>		<code>node.nextElementSibling</code>
<code>node.previousSibling</code>		<code>node.previousElementSibling</code>

putem defini metode noi

```
function secondChild(e) {
 return e.firstElementChild.nextElementSibling;
};
```

# NAVIGAREA ÎN ARBORELE DE ELEMENTE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload=function() {
 var c = document.body.children;
 var txt = "";
 var i;
 for (i = 0; i < c.length; i++) {
 txt = txt + c[i].nodeName + "

";
 }

 document.getElementById("lista").innerHTML +=
txt;

 alert(document.getElementById("lista").childNo
des[0].nodeName);
 }*/
 </script>
 </head>
```

>

# NAVIGAREA ÎN ARBORE - QUIZ

- o funcție find(e, className) care pentru un element e din DOM, întoarce primul strămoș care conține clasa className

```
function find(e, className) {
 var p = e.parentElement;
 while (p != document.documentElement) {
 if(p.classList.contains(className))
 return p;
 p = p.parentElement;
 }
 return null;
}
```

# NAVIGAREA ÎN ARBORE - QUIZ

- o funcție `frati(e)` care pentru un element `e` din DOM, calculează numărul fraților de același tip

```
function frati(e) {
 var nr = -1;
 var p = e.parentElement;
 var copii = p.children;
 for(var i = 0; i < copii.length; i++) {
 if(e.nodeName == copii[i].nodeName) nr++;
 }
 return nr;
}
```

# Operații cu Elemente

## crearea unui element

```
document.createElement("tag")
document.createTextNode("text")
```

## inserarea unui element

```
parinte.appendChild(copil)
// dacă nodul copil există în arbore, doar mută nodul
parinte.insertBefore(CopilNou, CopilVechi)
```

## ștergerea / înlocuirea unui element

```
parinte.removeChild(copil)
parinte.replaceChild(CopilNou, CopilVechi)
```

# Operații cu Elemente

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* function create(tag, text) {
 var elnou =
document.createElement(tag);
 var textnou =
document.createTextNode(text);
 elnou.appendChild(textnou);
 return elnou;
 }
 window.onload=function() {
 var list =
document.getElementById("lista");
 var el1 = create("li","everything
nice");
 list.appendChild(el1);
 var el2 = create("h2","Powerpuff");

document.body.insertBefore(el2,list);
```



# MODIFICAREA ATRIBUTELOR

- attributele elementelor HTML devin proprietăți ale obiectelor corespunzătoare
- pot fi accesate prin
  - numele proprietății
  - metode specifice
  - proprietatea `attributes`

# MODIFICAREA ATRIBUTELOR

- proprietăți: `el.id`, `el.className`, `el.alt`, `el.href`, `el.src`
- metode:

```
el.getAttribute("nume-atribut")
// întoarce un string (valoarea atributului specificat)

el.setAttribute("nume-atribut", "valoare")
// adaugă un atribut și valoarea lui

el.hasAttribute("nume-atribut") // întoarce boolean
el.removeAttribute("nume-atribut") // șterge atributul
```

```
el.proprietateNoua = valoare // adăugare de proprietăți noi
```



# MODIFICAREA ATRIBUTELOR

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* window.onload = function() {
 var x =
document.getElementById("imag1");
 x.src = "2.jpg"; // schimb sursa
imaginii
 var y =
document.getElementById("imag2");
 y.src = "3.jpg"; // schimb sursa
imaginii
 }
 */
 </script>
 </head>
 <body>

 " +
 attrs[i].value + " // ";
}
alert(output);
```

# JAVASCRIPT + CSS = STYLE

- orice obiect asociat unui element HTML (interfața `Element`) are proprietatea `style`, a cărei valoare este un obiect care implementează `CSSStyleDeclaration`
- proprietăților CSS le corespund proprietăți ale obiectului *style*
- schimbarea stilului unui element HTML:

```
element.style.proprietate = stil nou
```

# JAVASCRIPT + CSS = STYLE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* function schimbaStil(el) {
 el.style.color = "pink";
 el.style.fontFamily = "Arial";
 el.style.fontSize = "5em";
 }
 window.onload = function() {

schimbaStil(document.getElementById("par"));
 } */
 </script>
 </head>
 <body>
 <p id="par">Bravo, ai stil!</p>
 </body>
</html>
```

>

# JAVASCRIPT + CSS : CLASE

- clasele asociate unui element pot fi accesate folosind proprietatea `classList`, care implementează `DOMTokenList`
- lista claselor elementului:

```
ecl = element.classList
```

# JAVASCRIPT + CSS : CLASE

```
<script>
 window.onload = function() {
 var pclas = document.getElementById("par").classList;
 alert(pclas);
 }
</script>
</head>
<body>
<p id="par" class="c1 c2 c3 c4"> Hakuna Matata! </p>
```

# JAVASCRIPT + CSS : CLASE

```
ecl = element.classList;

ecl.length // nr. de clase asociate elementului
ecl.item(i) // numele clasei cu indexul i (i = 0,..)
ecl.add("clasa1", "clasa2", ...)
ecl.remove("clasa1", "clasa2", ...)
ecl.contains("clasa") // returns boolean

ecl.toggle("clasa", expresie)
/* șterge clasa dacă aceasta există, altfel o adaugă;
 expresie => boolean: if true, se adaugă clasa,
 if false, se șterge clasa */
```



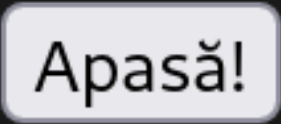




# TEHNICI WEB EVENIMENTE

*Claudia Chiriță . 2024/2025*

# ELEMENTUL HTML BUTTON



Apasă!

```
<button name="button"> Apasă! </button>
```

# ATRIBUTUL HTML ONCLICK

```
<tag onclick="cod JavaScript"></tag>
```

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 </head>
 <body>
 <button onclick="alert('Hello, kitty!
Inline event handlers are bad practice. Do not
use them!')">
 Apasă!
 </button>
 </body>
</html>
```



# ATTRIBUTE EVENT HANDLERS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 function change(alttext){
 // event handler
 var x =
document.getElementById('kitty');
 x.src = "kitty2.jpg";
 x.alt = alttext;
 alert('stop using inline handlers!') }
 </script>
 </head>
 <body>

 <button id="button"
 onclick="change('hendel')">
 Handle kitty
 </button>
```



# JAVASCRIPT: PROPRIETATEA ONCLICK

```
object.onclick = function() {cod JavaScript};
object.onclick = nume-functie-js;
```

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = load;
 function load() {
 document.getElementById("button")
 .onclick = function(){
 change("hendl");} }
 function change(alttext){
 var x =
document.getElementById('kitty');
 x.src = "kitty3.jpg";
 x.alt = alttext; }
 </script>
 </head>
 <body>

```

>

# EVENIMENTE

- evenimentele au loc în DOM; nu sunt elemente JavaScript
- browserul detectează un eveniment și îl anunță
- unui eveniment îi sunt asociate unele elemente JavaScript specifice: name, target, handler
- **tipuri de evenimente**: form events, window events, mouse events, key events etc.

# EVENTIMENTE

- la apariția unui eveniment este creat un obiect de tip **Event** care poate fi referit prin identificatorul *event*

- referință HTML (*event*):

```
<tag onclick="f(event)"></tag>
```

- *e* - parametrul implicit al funcției apelate de eveniment

```
element.onclick = function(e) {}
```



# OBIECTE EVENT: PROPRIETĂȚI

```
event.type // numele evenimentului
```

```
event.target // ținta inițială a evenimentului
```

```
event.currentTarget /* ținta curentă a evenimentului
 (la capturare sau propagare) */
```

```
event.timeStamp /* numărul de milisecunde de la
 încărcarea documentului până la
 crearea evenimentului */
```

# OBIECTE EVENT: PROPRIETĂȚI

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 document.body.onclick = clicked;
 function clicked(event) {
 alert("event.type: " + event.type +
 '\n' +
 "event.target: " +
event.target.nodeName + '\n' +
 "event.currentTarget: " +
event.currentTarget.nodeName + '\n' +
 "event.timeStamp: " +
event.timeStamp); }
 }
 </script>
 </head>
 <body>
 <p>Soft Kitty</p>

```



# MOUSE EVENTS

- evenimente determinate de mouse
- reprezentate prin obiecte de tipul MouseEvent

```
click, dblclick
mouseover, mouseout
mousedown, mouseup
mouseleave
```

# MOUSE EVENTS

## proprietăți speciale:

```
event.button // 0 (stânga) 1(mijloc) 2(dreapta)

event.clientX, event.clientY /* poziția în fereastră
 (zona vizibilă) */
event.pageX, event.pageY // poziția în document
event.screenX, event.screenY // poziția în ecran

event.relatedTarget /* elementul legat de elementul care
 a declanșat evenimentul */
```

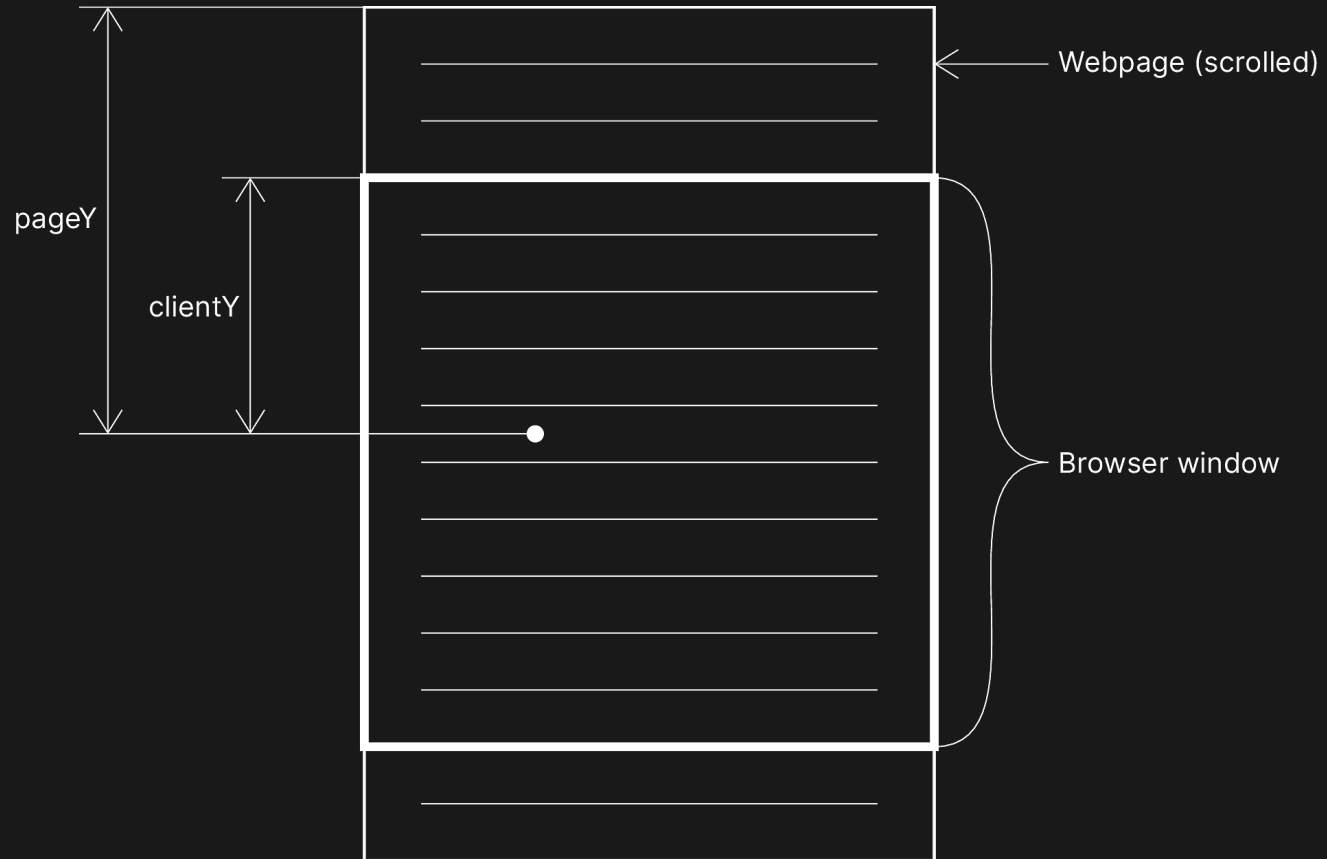
# MOUSE EVENTS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 document.body.onclick = positie;
 function positie(event) {
 alert("event.clientX-Y " +
event.clientX + ' ' + event.clientY + '\n' +
 "event.pageX-Y: " + event.pageX +
' ' + event.pageY + '\n' +
 "event.screenX-Y " +
event.screenX + ' ' + event.screenY); }
 }
 </script>
 </head>
 <body>
 <p>Warm Kitty</p>

 </body>
```



# MOUSE EVENTS



# KEYBOARD EVENTS

- evenimente determinate de interacțiunea cu tastatura
- reprezentate prin obiecte de tipul `KeyboardEvent`

```
keydown, keyup
keypress // deprecated!
```

- proprietăți:

```
event.key // tasta apasată
event.altKey // boolean
event.ctrlKey // boolean
event.keyCode, event.which // deprecated!
```

# KEYBOARD EVENTS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 var par =
document.getElementById("par");
 document.body.onkeyup = function(event)
{
 switch (event.key) {
 case "p":
 par.style.color = "pink";
 break;
 case "y":
 par.style.color = "yellow";
 break;
 case "b":
 par.style.color = "lightBlue";
 break;
 default:
 alert("Altă tastă"); return; }

```

>



# EVENT LISTENERS

- metode care înregistrează funcții handler pentru evenimente
- un eveniment poate avea înregistrate mai multe funcții handler

```
// addEventListener(type, listener, useCapture)
el.addEventListener("click", handleClick, false)
```

```
// removeEventListener(type, listener, useCapture)
el.removeEventListener("click", handleClick, true)
```

# EVENT LISTENERS

```
<button id="buton">Apasă!</button>
```

```
var b = document.getElementById("buton");

// b.onclick = function(){ alert("Hello, kitty!"); }
b.addEventListener("click",
 function(){alert("Hello, kitty!");},
 false)
```

# HANDLERS

captarea evenimentelor: obiectul *event* este transmis  
ca prim argument al funcției handler

```
// el.onclick = myfct;
el.addEventListener("click", myfct);

function myfct (event) { alert(event.type); }
```

```
// el.onclick = function (event) { alert(event.type); }
el.addEventListener("click", function (event) { alert(event.ty
```

# HANDLERS

la apariția unui eveniment, funcțiile handler sunt executate în ordinea în care sunt definite

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = main;
 function main(){
 el =
document.getElementById("button");
 el.addEventListener("click", hand1,
false);
 el.addEventListener("click", hand2,
false);
 function hand1(event) {alert(1)};
 function hand2(event) {alert(2)};
 }
 </script>
 </head>
 <body>
 <button id="button"> Apasă! </button>
 </body>
</html>
```

>

# EVENT BUBBLING & CAPTURE

ordinea de execuție a handlerelor în ceea ce privește nodurile imbricate poate fi controlată prin parametrul boolean *useCapture* al metodei *addEventListener* la setarea unui handler pentru un nod părinte

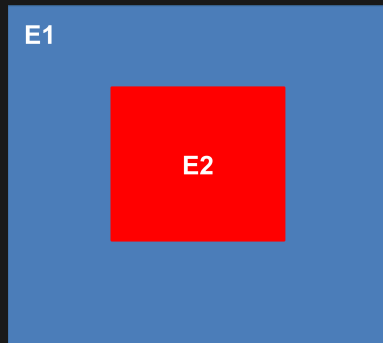
```
false // BUBBLING (implicit)
true // CAPTURING
```

# EVENT BUBBLING & CAPTURE

modele de execuție:

- bubbling: se execută mai întâi handlerul pentru copil, apoi handlerul pentru părinte
- capturing: se execută mai întâi handlerul pentru părinte, apoi cel pentru copil

# EVENT BUBBLING & CAPTURE



E1, E2: handler pentru onclick  
eveniment: click pe E2

modele de executie:

CAPTURING - handler E1, handler E2

BUBBLING - handler E2, handler E1

# EVENT BUBBLING & CAPTURE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <style>
 .alb{ background-color: white;
 color: black; }
 .roz{ background-color: pink; }
 .gal{ background-color: yellow; }
 div{ border: 3px solid pink;
 width: 300px; height:300px; }
 p { border: 3px solid yellow; }
 </style>
 <script type="text/javascript">
 window.onload = function() {
 var bunic = document.body;
 var parinte =
document.getElementById("parinte");
 var copil =
document.getElementById("copil");
 bunic.addEventListener("click",
function() {
```

>



# OBIECTE EVENT: METODE



```
event.stopPropagation()
```

oprește propagarea evenimentului în DOM

# STOPPROPAGATION()

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <style>
 .alb{ background-color: white;
 color: black; }
 .roz{ background-color: pink; }
 .gal{ background-color: yellow; }
 div{ border: 3px solid pink;
 width: 300px; height:300px; }
 p { border: 3px solid yellow; }
 </style>
 <script type="text/javascript">
 window.onload = function() {
 var bunic = document.body;
 var parinte =
document.getElementById("parinte");
 var copil =
document.getElementById("copil");
 bunic.addEventListener("click",
function() {
```

>

# OBIECTE EVENT: METODE

- `event.stopImmediatePropagation()`

dacă mai multe funcții listener sunt atașate  
aceluiași element, iar una conține  
`event.stopImmediatePropagation()`, funcțiile  
listener următoare nu mai sunt apelate

# EVENT BUBBLING & CAPTURE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <style>
 .alb{ background-color: white;
 color: black; }
 .roz{ background-color: pink; }
 .gal{ background-color: yellow; }
 div{ border: 3px solid pink;
 width: 300px; height:300px; }
 p { border: 3px solid yellow; }
 </style>
 <script type="text/javascript">
 window.onload = function() {
 var bunic = document.body;
 var parinte =
document.getElementById("parinte");
 var copil =
document.getElementById("copil");
 bunic.addEventListener("click",
function() {
```



# OBIECTE EVENT: METODE



`event.preventDefault()`

anulează acțiunea implicită a elementului

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 document.getElementById("link")
 .addEventListener("click",
function(event){
 event.preventDefault()});
 }
 </script>
 </head>
 <body>
 <a id="link"
href="https://archive.org/details/mogforgetful
cat0000kerr_g3o7">Mog, the forgetful cat
 </body>
</html>
```



# OBIECTUL WINDOW: TIMERS

- `window.setTimeout`

apelează o funcție sau execută un fragment de cod după un anumit timp (milisecunde)

```
vt = setTimeout(numeFuncție, întârziere, parametri);
vt = setTimeout(function() {}, întârziere, parametri);

clearTimeout(vt) // anulare funcție lansată
```

- vt este variabilă globală pentru a putea fi văzută de metoda clearTimeout
- parametrii sunt ai funcției care se va executa (numeFuncție sau anonimă)

# OBIECTUL WINDOW: TIMERS

- `window.setInterval`

execută funcția în mod repetat, la un anumit interval de timp

```
vt = setInterval(numeFuncție, interval, parametri);
vt = setInterval(function() {}, interval, parametri);

clearInterval(vt) // anulare funcție lansată
```

# OBJECTUL WINDOW: TIMERS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 /* var hello; var end = 15000;
 function sayHello() {
 alert("hello, kitty!"); }
 function sayHelloMany() {
 hello = setInterval(sayHello,
3000);}
 function stopHello(){
 clearInterval(hello); }
 sayHelloMany();
 setTimeout(stopHello, end); */
 </script>
 </head>
 <body>
 </body>
</html>
```

>



# PROBLEME CU VAR

```
var i = 0;
el.onclick = function() {
 alert(i); /* evaluare la click; va afisa 1 */
}
i = 1;
```

```
var i = 0;
{
 let il = i; // il e creat acum
 el.onclick = function(){
 alert(il); /* va afișa 0 */ }
} // se eliberează zona; il = 0
i = 1;
```

# PROBLEME CU VAR

la click pe o imagine din document să i se afișeze sursa

```
var imagini = document.getElementsByTagName("img");
for(var i = 0; i < imagini.length; i++)
 imagini[i].onclick = function() { alert(imagini[i].src); }
// nu funcționează
// i va fi egal cu imagini.length
```

```
// soluție
for(let i = 0; i < imagini.length; i++)
 imagini[i].onclick = function() { alert(imagini[i].src); }
```

# OBIECTUL WINDOW: TIMERS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 var pl =
document.getElementsByTagName("p");
 for(var i = 0; i < pl.length; i++)
 setTimeout(colorare, 1000*(i+1),
"red", pl[i]);
 function colorare(culoare,ob) {
 ob.style.color=culoare; }
 }
 </script>
 </head>
 <body>
 <p>Buton 1</p>
 <p>Buton 2</p>
```

>

# OBIECTUL WINDOW: TIMERS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 window.onload = function() {
 var pl =
document.getElementsByTagName("p");
 for(let i = 0; i < pl.length; i++)
 setTimeout(function(){
 colorare("red",
pl[i]));},
 1000*(i+1));
 function colorare(culoare,ob) {
 ob.style.color=culoare; }
 }
 </script>
 </head>
 <body>
```

>

# OBIECTUL WINDOW: GETCOMPUTEDSTYLE()

```
window.getComputedStyle(ob, pseudoel)
```

- determină stilul efectiv aplicat unui element
- întoarce un obiect read-only CSSStyleDeclaration
- *pseudoel* este opțional; pseudo-element sau null

```
var oStil = window.getComputedStyle(ob, ":after");
var x = oStil.color; // proprietatea css ob.style.color

window.getComputedStyle(ob, null).getPropertyValue("color");
```

# OBIECTUL WINDOW: COMPUTEDSTYLE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <style>
 div{ border-radius: 50%;
 background-color:red;
 width:100px; height:100px;}
 </style>
 <script type="text/javascript">
 window.onload = function() {
 var p =
document.getElementById("stil");
 var div =
document.getElementById("ou");
 var stil =
window.getComputedStyle(div);
 p.innerHTML+= '
' +
 "background-color: "+
 window.getComputedStyle(div).background-color + "
 }
 </script>
 </head>
 <body>
 <div id="ou">
 </div>
 </body>
</html>
```



# OBIECTUL WINDOW

- fereastra browserului care conține DOM asociat documentului
- poate fi selectat prin *document.defaultView*
- dacă un document HTML conține elemente `<iframe>`, browserul creează un obiect window pentru documentul HTML și un obiect window suplimentar pentru fiecare `<iframe>`

# MULTIPLE WINDOWS

- ferestrele iframe au proprietățile

```
contentWindow // obiectul window corespunzător iframe
parent // fereastra părinte
```

```
// colorează backgroundul primului iframe roz

var x = document.getElementsByTagName("iframe")[0].contentWindow
//x = obiectul window corespunzător primului iframe

x.document.getElementsByTagName("body")[0].style.backgroundColor
```



# OBIECTUL WINDOW: LOCATION

```
window.location
```

folosit pentru a determina URL-ul paginii curente și pentru a redirecționa browserul către o pagină nouă

```
window.location.href // adresa URL a documentului
window.location.hostname // numele de domeniu al gazdei web
window.location.pathname /* calea și numele fișierului
 paginii curente */
window.location.protocol /* protocolul folosit
 http: sau https: */
window.location.hash // partea din URL care începe cu #
window.location.assign(url) // încarcă un nou document
```

# OBIECTUL WINDOW: LOCATION

schimbarea locației

```
window.location.href="https://www.instagram.com/medievalcatsda
window.location="https://www.instagram.com/medievalcatsdaily/"
location="https://www.instagram.com/medievalcatsdaily/"
```

# OBIECTUL WINDOW: HISTORY

```
window.history
```

conține istoria browserului (adresele URL accesate de utilizator)

```
history.length // nr. de URL-uri din lista de istoric
history.back() /* încarcă adresa URL anterioară
 din lista de istoric */
history.forward() /* încarcă următoarea adresă URL
 din lista de istoric */
history.go(number/url) /* încarcă o adresă URL specificată
 din lista de istoric */
```

# OBIECTUL WINDOW: SCREEN

```
window.screen
```

conține informații despre ecranul utilizatorului

```
screen.availHeight // înălțimea ecranului (excluzând taskbarul)
screen.availWidth // lățimea ecranului (excluzând taskbarul)
screen.height // înălțimea totală a ecranului
screen.width // lățimea totală a ecranului
screen.colorDepth
screen.pixelDepth /* nr. de biți folosiți pentru
 culoarea unui pixel */
```

# OBIECTUL WINDOW: OPEN

```
window.open(URL, target, caracteristici)
```

- deschide o nouă fereastră de browser
- toți parametrii sunt opționali
- dacă nu se specifică URL se va deschide o fereastră cu about-blank

```
window.open("https://archive.org/details/the-cat-in-the-hat-by
"toolbar=yes,scrollbars=yes,resizable=yes,top=500,left=500,
width=400,height=400");
```

# OBIECTUL WINDOW: OPEN

```
var myWindow=window.open();
myWindow.close();
```

Închide fereastra deschisă cu metoda open()

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script type="text/javascript">
 var myWindow; // globală pt. a fi
 vazută la apelul close()
 function openWin() {
 myWindow =
 window.open("https://archive.org/details/the-
 cat-in-the-hat-by-dr.-seuss_"); }
 function closeWin() {
 myWindow.close();}
 </script>
 </head>
 <body>
 <button onclick="openWin()">Open
```

>







# TEHNICI WEB FORMS

*Claudia Chiriță . 2024/2025*

**CE ÎNVĂȚĂM SĂ FACEM AZI?**

# CLIENT-SIDE WEB STORAGE

- un obiect `Storage` este un array asociativ în care cheile și valorile sunt stringuri
- accesăm obiecte `Storage` prin proprietăți ale obiectului *window*:

```
localStorage // permanent
sessionStorage // până la închiderea tabului
```

# STORAGE

```
localStorage.length // nr. intrări în Storage
localStorage.key(i) // numele cheii cu indexul i
localStorage.setItem(numCheie, numeValoare)
/* adaugă o cheie și valoarea ei sau înlocuiește
 valoarea unei chei existente */
localStorage.getItem(numCheie) // valoarea cheii
localStorage.removeItem("x") // șterge cheia din Storage
localStorage.clear() // șterge toate cheile
localStorage.propNoua = valoare
```

# CLIENT-SIDE WEB STORAGE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <style>
 .brb {
 height: 350px;
 width: 350px;
 border-radius: 50%;
 display: inline-block;
 margin: 10px 0 10px 0;
 background-color: red;
 }
 div { text-align:center; }
 </style>
 <script>
 window.onload = function() {
 var buton =
document.getElementById("brb");
 buton.onclick= function() {
 let x =
Number(localStorage.getItem("nrc"));
 if (x) {
 localStorage.setItem("nrc", x +
1); }

```

>

# COOKIES?



# INCOGNITO?



# FORMS

elementul HTML `<form>` reprezintă o secțiune a unei pagini care conține elemente interactive (*controls*) și e folosit pentru a trimite informații către un server web

```
<form action="/my-handling-form-page" method="post">
<!-- poate conține orice element HTML, însă relevante sunt
 elementele de control -->
<!-- bad practice: imbricarea formularelor -->
</form>
```



# FORMS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 </head>
 <body>
 <div>
 <form action="/my-handling-form-page"
method="post">

 <label for="name">Name:</label>
 <input type="text" id="name"
name="user_name" />

 <label for="mail">Email:</label>
```



- după completare, datele sunt trimise serverului ca o listă de perechi cheie/valoare într-un request HTTP de tip post

# FORMS: CONTROLS

```
<form id="myform">

<!-- elemente specifice formularelor -->
<input>
<textarea>
<fieldset>
<select>
<button type="submit"> Submit </button>

</form>

<input type="submit" form="myform" value="Submit">
<button type="submit" form="myform"> Submit </button>
```

# STRUCTURARE: LABEL

folosit pentru etichetarea widget-urilor

```
<label for="name">Name:</label>
<input type="text" id="name" name="user_name">
```

sau

```
<label for="name">
 Name: <input type="text" id="name" name="user_name">
</label>
```

# STRUCTURARE: LABEL

etichetare **multiplă** a unui widget

```
<div>
 <label for="name">Name:
 *
 </label>
 <input type="text" id="name" name="user_name">
</div>
```

# STRUCTURARE: FIELDSET

folosit pentru gruparea elementelor într-un form

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 </head>
 <body>
 <div>
 <form>
 <fieldset>
 <legend>Cookies cu</legend>
 <p>
 <input type="radio" name="ingredient"
id="ingred_1" value="stafide" />
 <label for="ingred_1">Stafide</label>
 </p>
 <p>
 <input type="radio" name="ingredient"
```



# INPUT

```
<input type="..." name="..." id="...">
```

- poate lua multe forme, în funcție de valoarea atributului *type*
- pentru crearea majorității tipurilor de widget-uri: câmpuri de text pe o singură linie, controale pentru timp & dată, checkboxuri, butoane radio, selectoare de culoare, butoane

# TEXT INPUT

```
<input type="text" id="comment" name="comment" value="câmp tex
```

- attribute specifice: readonly, placeholder, required, size (width pentru câmp), maxlength (numărul maxim de caractere)

# TEXT INPUT

text input cu pattern:

```
<label for="cnp">
 <input type="text" id="cnp" pattern="[0-9]{13}">
 CNP</label>
```

```
<label for="ci">
 <input type="text" id="ci" pattern="[A-Z]{2}[0-9]{4}">
 C.I
</label>
```



# TEXT INPUT

- password type

```
<input type="password" id="pwd" name="pwd" maxlength="8">
```

- hidden type: control invizibil pentru user; este trimis serverului odată cu restul formularului

```
<input type="hidden" id="timestamp" name="timestamp" value="1286
```

# EMAIL

```
<input type="email" id="email" name="email" multiple="" require
```

- are validare client-side built-in
- atributul *multiple* permite scrierea mai multor adrese separate prin virgulă

# SEARCH

```
<input type="search" id="search" name="search">
```

- principala diferență față de input text este stilul:  
buton X de ștergere a conținutului, colțuri rotunjite

# CÂMP NUMERIC

```
<input type="number" name="age" id="age" min="1" max="10" step
```

- acceptă doar numere floating-point
- attribute: min, max, step (default 1, acceptă "any")

# SLIDER

```
<input type="range" name="price" id="price" min="5" max="500"
```

- se recomandă setarea atributelor: min, max, step

# FILE SELECTOR

```
<input type="file" name="file" id="file" accept="image/*" mult
```

- dispozitive mobile: selectorul de fișiere poate accesa fotografii, video și audio înregistrate direct de cameră și microfon

```
<input type="file" accept="image/*;capture=camera">
```

# RADIO

```
<input type="radio" name="ingredient" id="ingred_1" value="sta">
<label for="ingred_1">Stafide</label>

<input type="radio" name="ingredient" id="ingred_2" value="chi">
<label for="ingred_2">Choc chips</label>
```

- doar unul din butoanele radio poate fi selectat;  
atributul *name* trebuie să aibă aceeași valoare  
pentru toate opțiunile

# CHECKBOX

```
<input type="checkbox" id="cookie" name="cookie" checked="">
<label for="cookie">Cookies</label>
```

```
<input type="checkbox" id="biscuit" name="biscuit">
<label for="biscuit">Biscuits</label>
```

- permit selectarea mai multor valori, spre deosebire de butoanele radio



# HTML5 INPUT

- **color**: alegerea unei culori
- **date** : introducerea datei în formatul yyyy-mm-dd
- **time** : introducerea orei în formatul hh:mm
- **week** : introducerea datei în formatul yyyy-Www

# HTML5 INPUT

- **image**: buton grafic folosind o imagine
- **tel** : introducerea unui număr de telefon; fără validare implicită
- **url** : introducerea unui URL în formatul `urlscheme://restofurl`

# TEXTAREA

```
<textarea name="comments" rows="10" cols="48" wrap="hard" resi
```

- câmp de editare a textului pe mai multe linii
- implicit: cols = 20

# SELECT

```
<select id="ingrediente" name="ingrediente">
 <option>stafide</option>
 <option selected="">choc chips</option>
 <option>afine</option>
</select>
```

- control care oferă un meniu de opțiuni selectabile

# TRIMITEREA FORMULARELOR: INPUT

- **submit**: trimiterea datelor din formular către server

```
<input type="submit" value="Send Request">
```

- **reset**: resetarea tuturor câmpurilor din formular (rar folosit)

```
<input type="reset" value="Reset the form">
```

# TRIMITEREA FORMULARELOR: BUTTON

- submit: trimiterea datelor din formular către server

```
<button type="submit" value="Send Request" />
```

- reset: resetarea tuturor câmpurilor din formular

```
<button type="reset" value="Reset the form" />
```

- poate fi folosit în afara formularelor cu atributul

```
type="button"
```

# TRIMITEREA FORMULARELOR: ATRIBUTE

```
<form target="_blank" method="post" action="URL">
```

```
target = "_self" / "_blank" / "numeiframe"
/* locul în care este afișat răspunsul după
 trimiterea formularului (implicit: _self) */
```

```
method = "post"/"get"
/* metoda folosită pentru comunicarea cu serverul
 (implicit: get)
 get: query string în URL;
 post: query string în corpul requestului */
```

```
action = URL-ul care procesează formularul
/* (implicit: URL paginii curente) */
```

# FORMS

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 </head>
 <body>
 <div>
 <form action="" method="post"
target="numeiframe">

 <label for="name">Name:</label>
 <input type="text" id="name"
name="user_name" />

 <label for="mail">Email:</label>
```





**VALIDARE FORMULAR BUILT-IN**

**VALIDARE FORMULAR: E REZONABIL?**

# FORMS & JAVASCRIPT

- un form este reprezentat în JavaScript de un obiect *HTMLFormElement*
- proprietatea **elements** întoarce colecția câmpurilor formularului (array-like object)
- proprietatea *length* întoarce numărul de câmpuri

# FORMS & JAVASCRIPT

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script>
 /* window.onload=function() {
 var f =
document.getElementById("formular");
 alert(f.elements);
//[objectHTMLFormControlsCollection]
 alert(f.length); // 4
 alert(f.elements[2].value); // I love
cookies!
 } */
 </script>
 </head>
 <body>
```



# PROPRIETATEA VALUE

- pentru `<input type="text">` și `<textarea>` reprezintă valoarea introdusă de utilizator la trimiterea formularului și este un String
- pentru `<option>` reprezintă valoarea opțiunii alese
- altfel, reprezintă valoarea atributului html *value*

```
<input type="text" value="5">
<input type="submit" value="Trimite">
<input type="radio" value="v1">
<input type="checkbox" value="v1">
```

## EVENIMENTE FORM

- **submit**: evenimentul lansat la trimiterea formularului
- **reset**: evenimentul lansat când un formular este resetat

# EVENIMENTE FORM

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script>
 /* window.onload = function() {

document.getElementById("read").onsubmit =
suma;
 }
 function suma() {
 let x =
document.getElementById("nr").value;
 let s = 0;
 for (let i = 1; i <= parseInt(x);
i++)
 s = s + i;
```



# PROPRIETĂȚI SPECIFICE CÂMPURILOR

- proprietatea *checked* (boolean) pentru elemente

```
<input type="radio" value="v1">
<input type="checkbox" value="v1">
```

- proprietatea *elements* a câmpului *fieldset*:  
array-like care îi conține componentele



# PROPRIETĂȚI SPECIFICE CÂMPURILOR

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script>
 /* window.onload = function() {

document.getElementById("quiz").onsubmit =
checkQuiz;
 alert("dd");
 }
 function checkQuiz() {
 let fe =
document.getElementById("quiz").elements;
 let q1 = fe[0].elements;
 let q2 = fe[4].elements;
 let v = 0;
```



# EVENIMENTE CONTROALE: INPUT

- pentru elemente cu conținut editabil  
ex: <input>, <textarea>
- se declanșează la fiecare modificare  
ex: fiecare caracter scris într-un input

# EVENIMENTE CONTROALE: CHANGE

- pentru `<input>`, `<textarea>`, `<select>`
- se declanșează după alegerea unei noi valori
- pentru `<input type="text">` și `<textarea>` se declanșează când încetează focusul pe element

# EVENIMENTE CONTROALE: CHANGE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script>
 /* window.onload = function() {

document.getElementById("tt").onchange =
function () {

alert(document.getElementById("tt").value); }
 } */
 </script>
 </head>
 <body>
 <div>
 <input type="text" id="tt" value="Change" />
 </div>
 </body>
</html>
```

>

# EVENIMENTE CONTROALE: CHANGE

```
<!DOCTYPE html>
<html>
 <head>
 <link rel="stylesheet" href="demo.css">
 <script>
 /* window.onload = function() {
 obp =
document.getElementById("hello");
 obsel =
document.getElementById("sel");
 obsel.onchange = function () {
 obp.innerHTML +=
obsel.options[obsel.selectedIndex].value;
 }
 } */
 </script>
 </head>
```



**AI-AI, BAD UI!**

**AI-AI, BAD UI!**

**AI-AI, BAD UI!**



**AI-AI, BAD UI!**

**AI-AI, BAD UI!**

**AI-AI, BAD UI!**

**AI-AI, BAD UI!**

```

 WW
 / _ \
 | oo |
 (| _ () _ |) / _ o o \
W" "W \ _ / (| _ o _ |)
W -===== W / | \ / | \ \ _ / /
 "W W" | | | | | | /-----
W" " " " " " " " "W | | | | | | =====
W W | | | | | | =====

```

întrebări?



# TEHNICI WEB

# AJAX

*Claudia Chiriță . 2024/2025*

# OBIECTE

- un obiect este o colecție de perechi proprietate-valoare
- dacă valoarea este o funcție, atunci proprietatea se numește metodă
- ```
var ob = {prop1: val1, prop2: val2, ... , propn: valn};
```
- accesarea proprietăților:

```
ob.prop1; // val1  
ob["prop1"]; // val1
```


PROTOTIPURI

- prototipul unui obiect este desemnat prin

```
obiect.prototype
```

- orice obiect moștenește proprietățile obiectului prototip (prototypal inheritance)
- obiectele care au același prototip formează o clasă
- toate obiectele sunt descendenți ai obiectului generic *Object*

```
Object.getPrototypeOf() // prototipul obiectului specificat
```

CREAREA OBIECTELOR

- prin **object literal**
- proprietățile, metodele, împreună cu valorile lor sunt enumerate între acolade
- se creează un singur obiect

```
var cat = {nume:"Tigger", culoare:"neagră", vârstă:14}
```

CREAREA OBIECTELOR

- cu ajutorul **obiectului generic**
- se apelează constructorul `new Object()` și se adaugă apoi proprietățile și metodele
- se creează un singur obiect

```
var cat = new Object();  
cat.nume = "Tigger";  
cat.culoare = "neagră";  
cat.vârstă = 14;
```

CREAREA OBIECTELOR

- cu ajutorul unui **constructor de obiecte**
- se definește o funcție `constructor(param)` care apoi va fi apelată cu `new constructor(param)` pentru fiecare obiect care va fi creat

```
function cat(n, c, v) {  
  this.nume = n;  
  this.culoare = c;  
  this.vârstă = v;  
}  
  
var c1 = new cat("Tigger", "neagră", 14);  
var c2 = new cat("Fluff", "albă", 2);
```

CREAREA OBIECTELOR

- cu metoda `Object.create()`

- `Object.create` (*ob*)

creează un nou obiect, folosind un obiect existent *ob* ca prototip al obiectului nou creat

EXEMPLU: INTERVAL

```
var interval = {  
  mx: 2,  
  my: 4,  
  apartine: function(z) {  
    return (z <= this.my) && (z >= this.mx);  
  }  
}; // clasa  
  
var obi = Object.create(interval);  
// obiect din clasa interval  
obi.mx = 5;  
obi.my = 7; // suprascriem proprietățile prototipului
```

EXEMPLU: INTERVAL

```
var intervalD = Object.create(interval);
intervalD.apartine = function(z) {
    return (z < this.my) && (z > this.mx);
}; // subclasa

var obid = Object.create(intervalD);
obid.mx = 5;
obid.my = 10;
```

EXEMPLU: INTERVAL

```
interval.valid = function() {  
    return (this.my >= this.mx);  
};
```

```
intervalD.vid = function() {  
    return (this.mx == this.my);  
};
```

```
alert(obid.valid()); // true  
alert(obid.vid());  // false
```


CUVÂNTUL-CHEIE THIS

- în interiorul unui constructor sau al unei metode asociate unui obiect, *this* se referă la obiectul curent
- într-o funcție folosită ca event handler, se referă la elementul pentru care este definit listenerul
- altfel, *this* se referă la obiectul *window*

EXEMPLU: INTERVAL (2)

```
function Interval(x, y) {  
    this.mx = x;  
    this.my = y;  
} // clasa
```

```
Interval.prototype.apartine = function(z) {  
    return (z <= this.my) && (z >= this.mx);  
} /* metodă adăugată în prototipul obiectelor  
    create cu funcția constructor */
```

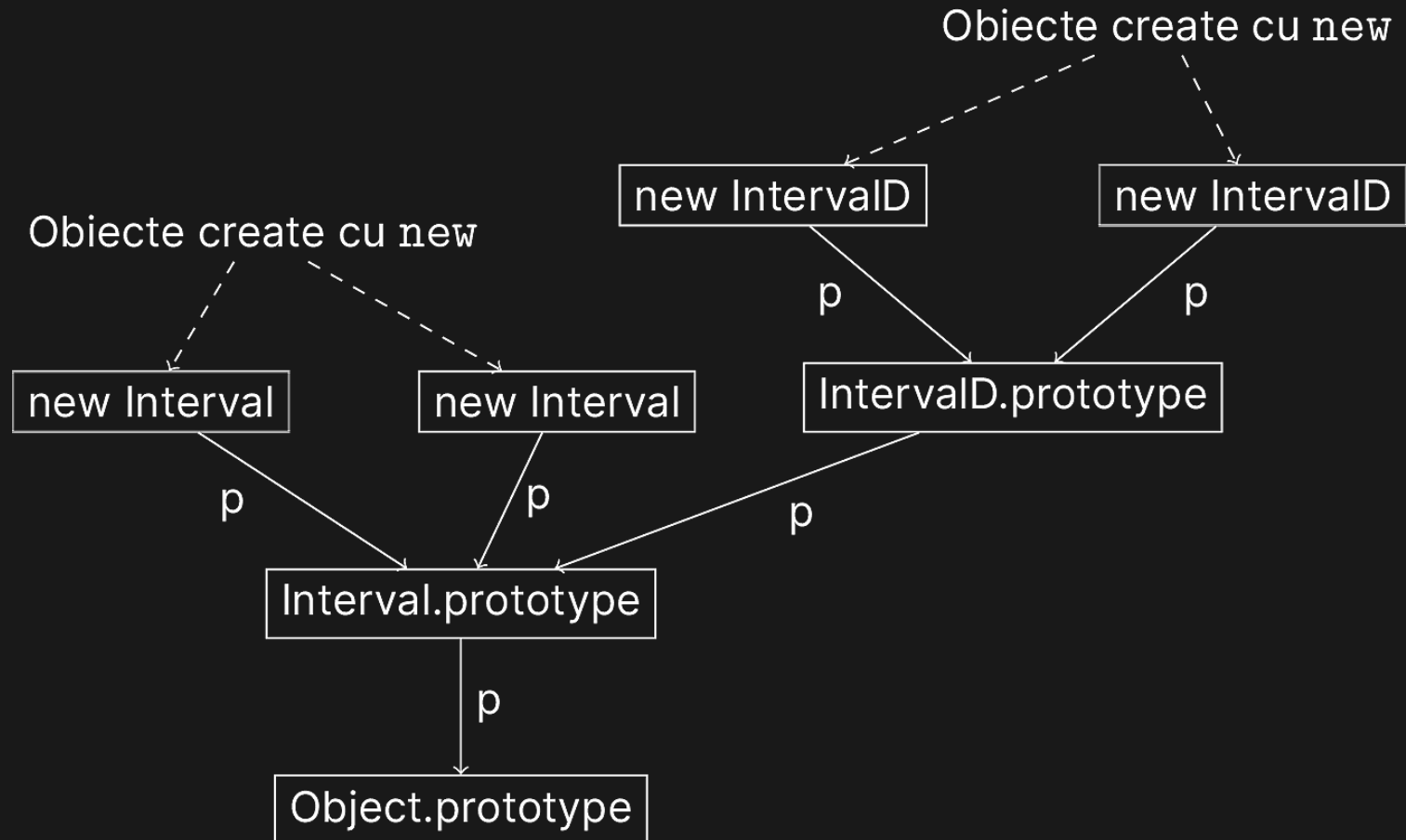
EXEMPLU: INTERVAL (2)

```
var obi = new Interval(1, 4); // obiect din clasa Interval

Interval.prototype.valid = function() {
    return (this.my >= this.mx);
};

alert(obi.valid()); // true
```

EXEMPLU: INTERVAL (2)



EXEMPLU: INTERVAL (2)

definirea subclaselor

```
function IntervalD(x, y) {  
    Interval.call(this, x, y);  
} // this este obiectul care se construiește  
  
IntervalD.prototype = Object.create(Interval.prototype);  
// schimbăm prototipul obiectelor create cu IntervalD  
IntervalD.prototype.constructor = IntervalD;  
// restaurăm proprietatea constructor  
  
IntervalD.prototype.apartine = function(z) {  
    return (z < this.my) && (z > this.mx);  
};  
  
var obid = new IntervalD(5, 10);
```

EXEMPLU: INTERVAL (2)

definirea subclaselor

```
function IntervalD(x, y) {  
    Interval.call(this, x, y);  
}  
  
IntervalD.prototype = Object.create(Interval.prototype);  
IntervalD.prototype.constructor = IntervalD;  
  
IntervalD.prototype.apartine = function(z) {  
    return (z < this.my) && (z > this.mx);  
};  
  
IntervalD.prototype.valid = function() {  
    return (Interval.valid.call(this) && (this.mx != this.my));  
}
```

XML

Extensible Markup Language

XML

- limbaj de marcare similar cu HTML
- nu conține taguri predefinite, utilizatorul își definește propriile taguri și structura documentului
- conceput pentru stocarea și transmiterea datelor
- independent de software și hardware
- utilizat pentru a crea limbaje de marcare precum XHTML, SVG, MathML

STRUCTURA UNUI DOCUMENT XML

- Un document XML este format din:
 - elemente: taguri `<nume_tag>`
case sensitive
 - date caracter: conținutul elementelor

STRUCTURA UNUI DOCUMENT XML

- se definește versiunea XML și codificarea caracterelor (prima linie în document)

```
<?xml version="1.0" encoding="UTF-8"?>
```

- document XML: structură arborescentă
- toate documentele XML trebuie să conțină un element rădăcină (root)

```
<root>  
  <child>  
    <subchild>...</subchild>  
  </child>  
</root>
```

SINTAXA XML

- există un singur element root într-un document XML
- fiecare element trebuie să aibă tag de închidere
- elementele XML trebuie să fie corect imbricate
- attributele asociate elementelor nu pot conține mai multe valori

ENTITĂȚI

- pentru a ne referi la caractere rezervate

Entity	Character	Description
<	<	Less than sign
>	>	Greater than sign
&	&	Ampersand
"	"	One double-quotation mark
'	'	One apostrophe (or single-quotation mark)

EXEMPLU

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore> // elementul rădăcină
  <book category="fantasy"> // element copil
    <title lang="en">Piranesi</title>
    <author>Susanna Clarke</author>
    <year>2020</year>
    <price>30.00</price>
  </book>
  <book category="science-fiction">
    <title lang="en">Klara and the Sun</title>
    <author>Kazuo Ishiguro</author>
    <year>2021</year>
    <price>29.99</price>
  </book>
</bookstore>
```

XML-PARSER

- browserele au un analizor XML încorporat pentru a accesa și manipula documente XML
- înainte ca un document XML să poată fi accesat, acesta trebuie convertit într-un obiect XML DOM

XML-PARSER

- XML DOM definește proprietăți și metode pentru accesarea și editarea XML
 - proprietăți: nodeName, nodeValue, parentNode, childNodes, attributes
 - metode: getElementByTagName(), appendChild(), removeChild()

XML-PARSER

```
text = "<bookstore><book>" + "<title>Piranesi</title>" +  
"<author>Susanna Clarke</author>" + "<year>2020</year>" +  
"</book></bookstore>"; // XML ca string  
  
parser = new DOMParser(); // se creează un analizor XML DOM  
  
xmlDoc = parser.parseFromString(text, "text/xml");  
// se creează un obiect XML DOM din stringul text  
  
xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue  
// extragem informația din nodurile XML DOM
```


JSON

JavaScript Object Notation

JSON

- oferă o modalitate de reprezentare a datelor, ca alternativă la XML
- bazat pe JavaScript; în prezent un format independent de limbaj
- multe limbaje pot prelucra date în format JSON
- folosit pentru schimbul de informații cu serverul

JSON

elemente de bază:

```
Object: {"cheie1":val1, "cheie2":val2}  
Array: [val1, val2, val3]  
Value: string, number, object, array, true, false, null
```

date.json

```
[{"pers": {"nume": "Klara", "varsta": 12} },  
 {"pers": {"nume": "Josie", "varsta": 14} } ]
```

SINTAXA JSON

- câmpul **cheie** trebuie să fie scris cu ghilimele

```
"nume": "Klara"
```

- câmpul **valoare** poate fi:

```
string, number, obiect (JSON), array, boolean, null
```

nu poate fi function, date, undefined

- obiectele JSON sunt reprezentate între acolade

```
{"nume": "Klara", "varsta": 12, "porecla": "Robo"}
```

EXEMPLU JSON

```
JSON String:  { "nume":"Klara" }
JSON Number:  { "varsta":12 }
JSON Object:  { "pers": { "nume":"Josie", "varsta":14 } }
JSON Array:   { "copii": [ "Klara", "Josie", "Rick" ] }
JSON Boolean: { "robot": true }
JSON null:    { "porecla": null }
```

OBIECTE JSON

```
myObj = { "cheie1":val1, "cheie2":val2, "cheie3":val3 }
```

- accesarea obiectelor:

```
myObj.cheie1
```

```
myObj["cheie1"]
```

OBIECTE JSON

- iterarea proprietăților unui obiect:

```
var myObj = { "copil": "Klara", "varsta": 12, "robot": true }  
for (x in myObj) {  
    document.getElementById("prop").innerHTML += x + "<br>"  
}
```

```
<p id="prop"></p>
```

paragraful va conține:

copil

varsta

robot

OBIECTE JSON

- iterarea proprietăților unui obiect:

```
var myObj = { "copil": "Klara", "varsta": 12, "robot": true }  
for (x in myObj) {  
    document.getElementById("val").innerHTML += myObj[x] + "  
}
```

```
<p id="prop"></p>
```

paragraful va conține: Klara 12 true

OBIECTE JSON ÎNCORPORATE

```
myObj = { "nume": "Klara",  
          "varsta": 12,  
          "componente": { "baterie": "solara", "motoare": "10" } }
```

- accesarea obiectelor încorporate:

```
myObj.componente.motoare // 10  
myObj.componente["motoare"] // 10
```

OBIECTE JSON ÎNCORPORATE

```
myObj = { "nume": "Klara",  
          "varsta": 12,  
          "componente": { "baterie": "solara", "motoare": "10" } }
```

- modificarea valorilor proprietăților:

```
myObj.componente.motoare="11";
```

- ștergerea proprietăților:

```
delete myObj.componente.baterie;
```

JSON ARRAYS

```
[val1, val2, ....., valn]
```

val1,...,valn pot fi string, number, object, array, boolean, null

- Array în interiorul obiectelor JSON

```
myObj = { "nume": "Klara",  
          "varsta": 12,  
          "componente": ["baterie", "motoare", "senzori"] }
```

accesarea valorilor:

```
myObj.componente[0] // baterie
```

JSON ARRAYS

- Array în interiorul obiectelor JSON

```
myObj = { "nume": "Klara",  
          "varsta": 12,  
          "componente": ["baterie", "motoare", "senzori"] }
```

iterarea valorilor în Array:

```
for (i in ob.componente) { x += ob.componente[i]; }  
  
for (i = 0; i < ob.componente.length; i++) {  
    x += ob.componente[i];  
}
```

OBIECTUL JSON ÎN JAVASCRIPT

```
JSON.stringify(valoare)  
// transformă un obiect JavaScript într-un string JSON  
  
JSON.parse(text)  
// transformă un string JSON într-un obiect JavaScript
```

poate fi folosit pentru memorare în
localStorage și *sessionStorage*

OBIECTUL JSON ÎN JAVASCRIPT

```
var o1 = {copil: {nume:"Klara", varsta:12}},  
    o2 = {copil: {nume:"Josie", varsta:14}},  
    o = [o1,o2];  
  
var s = JSON.stringify(o);  
// "[{"copil":{"nume":"Klara","varsta":12}},{"copil":{"nume":"  
  
localStorage.setItem("myarray", s);  
var st = localStorage.getItem("myarray");  
  
var jo = JSON.parse(st);
```

AJAX

Asynchronous JavaScript and XML

AJAX

- termenul AJAX a fost introdus de Jesse James Garrett în 2005
- azi se referă la un grup de tehnologii folosite pentru procesele client ce descarcă informație de la server fără reîncărcarea completă a unei pagini web
- inițial se foloseau documente XML pentru request-uri și response-uri
- acum formatul JSON e mult mai uzual

AJAX

- permite actualizarea unor părți ale unei pagini web fără reîncărcarea completă a paginii
- trimite cereri către un server web și citește datele primite de la server
- nucleul său îl reprezintă obiectul *XMLHttpRequest* care este folosit pentru a schimba date asincron cu serverul web

AJAX

Browser

An event occurs...

- Create an XMLHttpRequest object
- Send HttpRequest

Internet

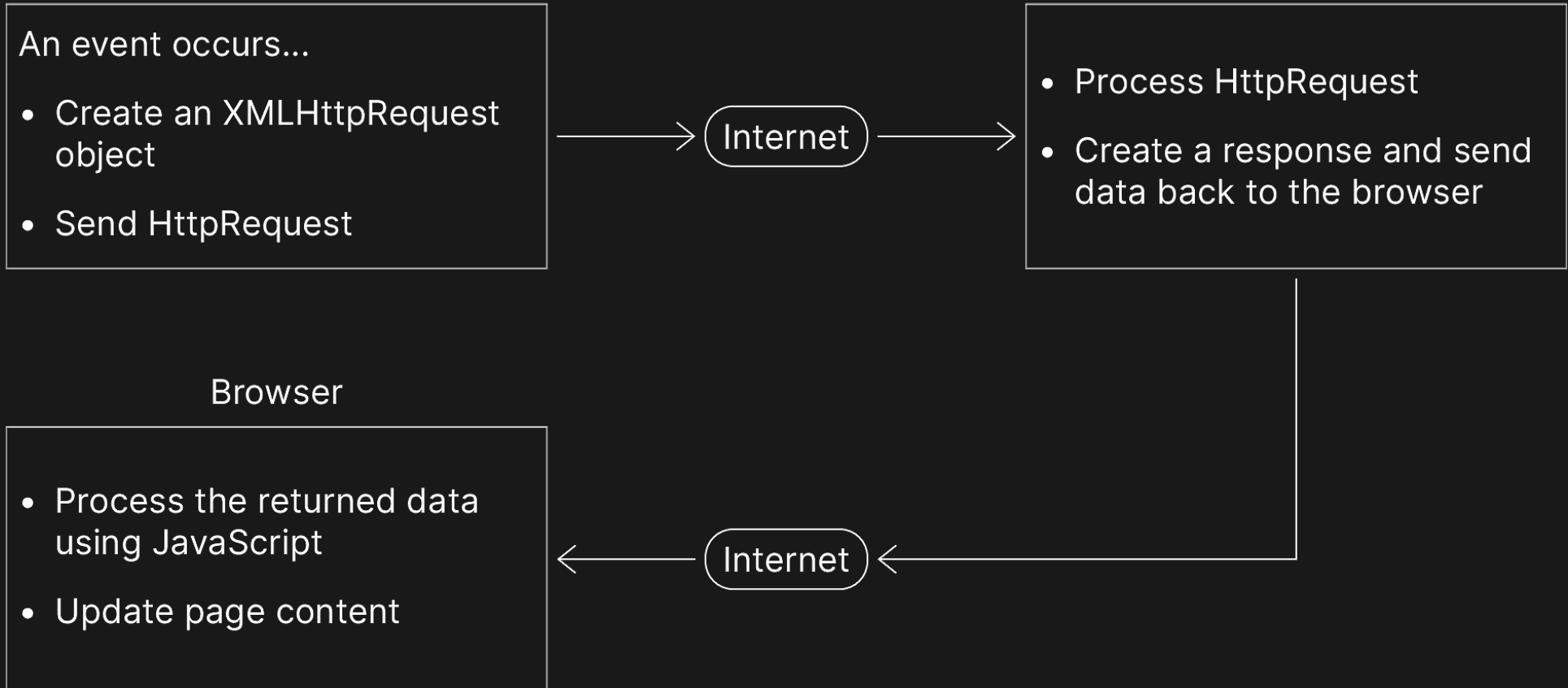
Server

- Process HttpRequest
- Create a response and send data back to the browser

Browser

- Process the returned data using JavaScript
- Update page content

Internet



XMLHTTPREQUEST

- obiect JavaScript ce permite trimiterea de cereri către un server și primirea rezultatului în script
- pot fi procesate în paralel mai multe conexiuni cu serverul, fără blocarea browser-ului până la primirea răspunsului
- înainte de utilizarea XMLHttpRequest, trebuie creată o instanță

```
var xhr = new XMLHttpRequest()
```

XMLHTTPREQUEST

- pune la dispoziție mai multe proprietăți și metode pentru comunicarea client/server

```
open() // creează cererea
send() // trimite cererea

readyState // starea cererii (0,1,2,3,4)
onreadystatechange
// funcția care se execută la schimbarea stării cererii

status // codul de stare HTTP (200 OK)
responseText
// răspunsul primit de la server în format text
responseXML
// răspunsul primit de la server în format XML
```

CERERI HTTP

```
open( method, url, async ) // specifică tipul de cerere
/* method: poate fi GET sau POST
   url: adresa serverului
   async: true (asynchronous) sau false (synchronous) */
```

```
send() // trimite cererea către server ( GET )
send(data) // trimite cererea către server ( POST )
```

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "http://localhost:3000/test.txt", true);
xhr.send();
```

GESTIONAREA RĂSPUNSULUI

- proprietatea *readyState* reprezintă starea cererii:
 0. cererea este neinițializată
 1. conexiune stabilită cu serverul
 2. cererea a fost primită
 3. se procesează cererea
 4. cererea este finalizată și răspunsul este gata
- proprietatea *onreadystatechange* definește o funcție care trebuie executată când se schimbă *readyState*

```
xhr.onreadystatechange = nume-functie;
```

GESTIONAREA RĂSPUNSULUI

- proprietatea *status*: codul de stare HTTP al răspunsului de la server, în format numeric (200 "OK", 403 "Forbidden", 404 "Not found")
- proprietatea *statusText*: statusul în format text ("OK", "Not Found")

GESTIONAREA RĂSPUNSULUI

- proprietatea *responseText*: returnează răspunsul primit de la server, în format text (string)
- proprietatea *responseXML*: returnează răspunsul primit de la server, în format XML

EXEMPLU - TEXT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      var httpRequest;

      document.getElementById("ajaxButton").addEventListener('click', makeRequest);
      function makeRequest() {
        httpRequest = new XMLHttpRequest(); //
        creează un obiect XMLHttpRequest
        if (!httpRequest) {
          alert('Giving up :( Cannot create an
XMLHTTP instance');
          return false;
        }
        httpRequest.onreadystatechange
=alertContents;
        httpRequest.open('GET',
'https://raw.githubusercontent.com/mdn/content
/main/files/en-us/_wikihistory.json');
        httpRequest.send();
      }
      function alertContents() {
```



GESTIONAREA RĂSPUNSULUI

- Obiectul XML DOM în JavaScript poate fi parcurs cu metode asemănătoare celor din DOM

```
<?xml version="1.0" encoding="UTF-8"?>
<persoane>
  <pers nume="Klara" varsta="12"></pers>
  <pers nume="Josie" varsta="14"></pers>
</persoane>
```

```
var xml = httpRequest.responseXML;
var vpers= xml.getElementsByTagName('pers');
alert(vpers[0].getAttribute('nume'));
```

EXEMPLU - XML

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      var httpRequest;

      document.getElementById("ajaxButton").addEventListener('click', makeRequest);
      function makeRequest() {
        httpRequest = new XMLHttpRequest(); //
        creează un obiect XMLHttpRequest
        if (!httpRequest) {
          alert('Giving up :( Cannot create an
XMLHTTP instance');
          return false;
        }
        httpRequest.onreadystatechange =
alertContents;
        httpRequest.open('GET', 'test.xml');
        httpRequest.send();
      }
      function alertContents() {
        if (httpRequest.readyState == 4) {
          if (httpRequest.status == 200) {
```



EXEMPLU - JSON

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      var httpRequest;

      document.getElementById("ajaxButton").addEventListener('click', makeRequest);
      function makeRequest() {
        httpRequest = new XMLHttpRequest(); //
        creează un obiect XMLHttpRequest
        if (!httpRequest) {
          alert('Giving up :( Cannot create an
XMLHTTP instance');
          return false;
        }
        httpRequest.onreadystatechange =
alertContents;
        httpRequest.open('GET',
'https://mdn.github.io/learning-
area/javascript/apis/fetching-data/can-
store/products.json');
        httpRequest.send();
      }
    }
  </script>
</head>
</html>
```



TRIMITEREA FORMULARELOR

```
<!DOCTYPE html>
<html>

  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <form
action="https://www.w3schools.com/action_page.
php" target="_blank">
      <label for="fname">Prenume:</label><br>
      <input type="text" id="fname"
name="fname" value="Klara"><br>
      <label for="lname">Nume:</label><br>
      <input type="text" id="lname"
name="lname" value="Sun"><br><br>
      <input type="submit" value="Trimite">
    </form>
  </body>
</html>
```

>

TRIMITEREA FORMULARELOR VIA JS

cu JSON?

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
  var httpRequest;
  var myform =
document.getElementById("testform");
  myform.onsubmit = function (event){
    event.preventDefault();

    var data =
{prenume:document.getElementById("fname").valu
e,
  nume:document.getElementById("lname").value};

    httpRequest = new XMLHttpRequest(); //
creează un obiect XMLHttpRequest
    if (!httpRequest) {
```

>

TRIMITEREA FORMULARELOR VIA JS

cu FormData

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  <script>window.onload=function() {
    var httpRequest;
    var myform =
document.getElementById("testform");
    var FD;
    myform.onsubmit = function (event){
      event.preventDefault();
      FD = new FormData(myform);
      httpRequest = new XMLHttpRequest(); //
creează un obiect XMLHttpRequest
      if (!httpRequest) {
        alert('Giving up :( Cannot create an
XMLHTTP instance');
        return false;
      }
    }
  }
```

>

ASYNC JAVASCRIPT

ASYNC JAVASCRIPT

- `sync vs. async programs`
- event handlers sunt un tip de programare asincronă: apelăm o funcție (handlerul) nu imediat, ci când se declanșează un eveniment
event -> încheierea operației asincrone
- XMLHttpRequest - API asincron bazat pe evenimente: adăugăm event listenere obiectului XMLHttpRequest

ASYNC JAVASCRIPT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      var httpRequest;

      document.getElementById("ajaxButton").addEventListener('click', makeRequest);
      function makeRequest() {
        httpRequest = new XMLHttpRequest(); //
        creează un obiect XMLHttpRequest

        httpRequest.addEventListener('loadend', ()
=> {
          alert('Finished with status:' +
httpRequest.status));});

        httpRequest.open('GET',
'https://raw.githubusercontent.com/mdn/content
/main/files/en-us/_wikihistory.json');
        httpRequest.send();
      }
    }
  </script>
```



CALLBACKS

- un event handler e un tip particular de callback
- callback = funcție transmisă unei alte funcții ca parametru, în vederea apelării ei la momentul potrivit
- callbacks în callbacks = *callback hell* sau *pyramid of doom*
- API-urile asincrone moderne nu folosesc callbacks

PROMISES

- cod producător: de obicei necesită mult timp și întoarce un rezultat (ex: ia date dintr-o bază de date)
- cod consumator: dorește rezultatul *codului producător* de îndată ce este gata
- promise: obiect JavaScript care leagă *codul producător* și *codul consumator*

PROMISES

- promise: obiect returnat de o funcție asincronă, ce reprezintă starea curentă a operației executate
- la momentul returnării promisiunii în funcția caller, de regulă operația nu s-a terminat; obiectul oferă însă metode pentru gestiunea succesului sau eșecului operației

PROMISES

crearea unei promisiuni

```
promise = new Promise( function(resolve, reject) {  
    /* cod producător care se execută asincron:  
       setInterval, setTimeout, cereri Ajax, evenimente */  
    resolve(value) // pentru succes  
    reject(error)  // pentru eroare  
} );
```

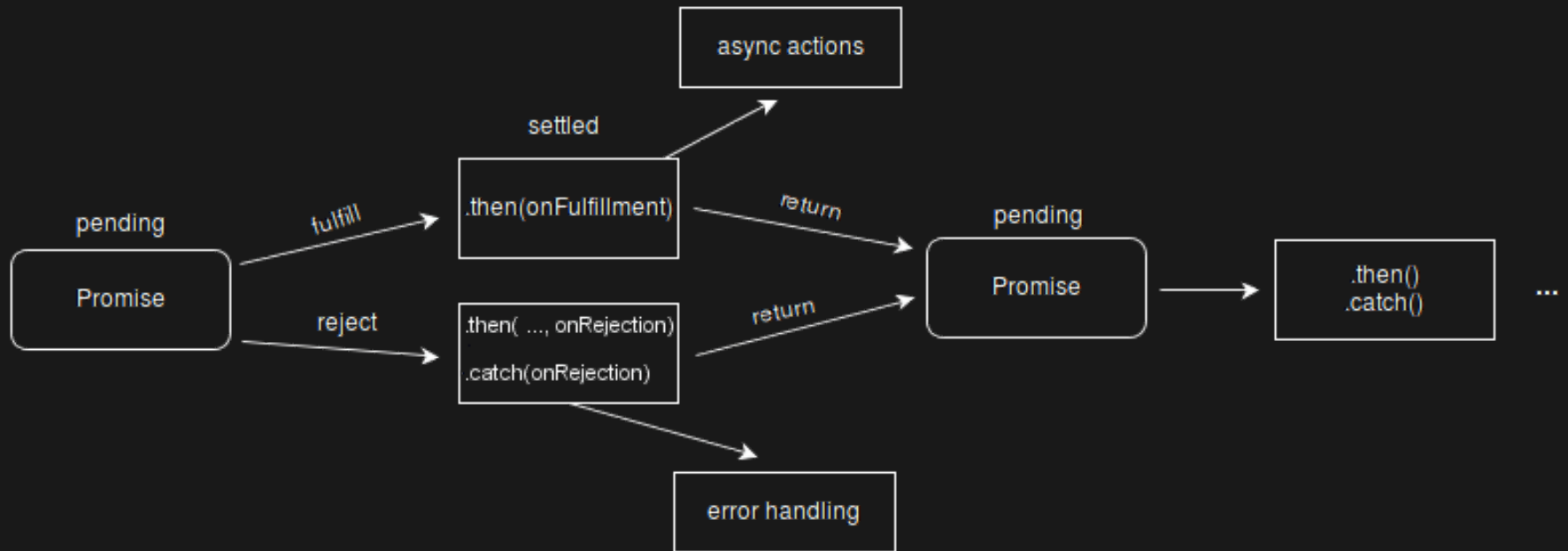
PROMISES

- la crearea unei promisiuni, se apelează automat funcția transmisă ca argument promisiunii (numită funcție *executor*)
- *executorul* are ca parametri două funcții predefinite (resolve & reject) și conține *codul producător* care ar trebui să producă în cele din urmă rezultatul dorit

PROMISES

- în funcție de rezultatul obținut (codul asincron va decide dacă e cu succes sau eșec) se va apela una din cele două funcții (funcția de succes - resolve sau funcția de eșec - reject)
- un obiect promise se poate afla într-una din stările: pending (în așteptare), fulfilled (îndeplinită), rejected (respinsă)

PROMISES



PROMISES

```
// cod consumator care așteaptă rezultatul
promise.then(function (value) { /* cod pentru succes */ },
             function(error) { /* cod pentru eroare */ })
             .catch(function(error) { /* tratarea erorii */ })
```

- *funcțiile consumatoare* pot fi înregistrate folosind metodele `.then`, `.catch` și `.final`

PROMISES

- primul argument al lui `.then` este o funcție care rulează atunci când promisiunea este rezolvată și primește rezultatul așteptat
- al doilea argument al lui `.then` este o funcție care rulează atunci când promisiunea este respinsă și primește eroarea

PROMISES

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      /* let p = new Promise(function(resolve,
reject) { // se creează o promisiune
        setTimeout(function(){
          var nota = Math.floor(Math.random()
* 10) + 1;
          if(nota >= 5)
            resolve("Bravo, ai promovat cu
nota " + nota+ "!"); // apelăm funcția pentru
succes
          else
            reject(nota); // apelăm funcția
pentru eșec
        },2000)}});
        console.log("Aștept răspunsul...");
        p.then(succes, esec).catch(function(err)
{console.log(err)}); // dacă primim o eroare,
o afișăm
      }
      function succes(rez){ // funcția pentru
succes
```



FETCH

- API pentru efectuarea unei cereri Ajax în JavaScript bazat pe promisiuni
- alternativă modernă la XMLHttpRequest
- metoda `fetch()`:

```
const promise = fetch(url, [options]);  
/* î ntoarce o promisiune folosită pentru a obține  
răspunsul de la server  
  
url: adresa url a severului  
options: method, headers, body (opționale) */
```

FETCH

```
const promise = fetch(url, [options]);  
promise.then(function(response) {  
    return response.text();  
});
```

- promisiunea returnată de fetch() este rezolvată cu un obiect din clasa Response imediat ce serverul răspunde cu un header
- promisiunea este respinsă dacă cererea HTTP nu poate fi trimisă (există probleme de conexiune) sau nu se primește răspunsul

FETCH

cerere Ajax cu XMLHttpRequest

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload = function() {
      /* var data;
      var httpRequest = new XMLHttpRequest();
      httpRequest.open('GET',
      'https://mdn.github.io/learning-
      area/javascript/apis/fetching-data/can-
      store/products.json');
      httpRequest.onreadystatechange = function()
      {
        if (httpRequest.readyState == 4) {
          if (httpRequest.status == 200) {
            alert(httpRequest.status + " OK");
          } else {
            alert('There was a problem with
the request.');
```



FETCH

cerere Ajax cu Fetch

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload = function() {
      /* var data;
      var promiseFetch =
      fetch('https://mdn.github.io/learning-
      area/javascript/apis/fetching-data/can-
      store/products.json');

      promiseFetch.then(function(response){
        if(response.status == '200')
          return response.status;
        else
          throw "eroare";})
        .then(function(status) {
          alert(status);})
        .catch(function(err){
          alert(err);});    */
    }
  </script>
```



TRIMITEREA FORMULARELOR VIA FETCH

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>window.onload=function() {
      var httpRequest;
      var myform =
document.getElementById("testform");
      myform.onsubmit = function (event){
        event.preventDefault();

        var data =
{prenume:document.getElementById("fname").valu
e,
name:document.getElementById("lname").value};

        fetch(myform.action,
          { method:"post",
            headers: {'Content-Type':
. . . . . }
```



TEHNICI WEB

CANVAS & SVG

Claudia Chiriță . 2024/2025

CANVAS

CANVAS

- `<canvas>` element HTML folosit pentru a desena via scripturi (de regulă cod JavaScript)
- permite desenarea de grafice, editare de imagini, creare de animații simple
- dimensiune default: 300px X 150px

```
<canvas id="tutorial" width="150" height="150">  
  fallback text  
</canvas>
```

RENDERING CONTEXT

- elementul <canvas> crează o suprafață de desen de dimensiune fixă, ce afișează unul sau mai multe contexte de randare
- contextele de randare sunt folosite pentru crearea și manipularea conținutului afișat
- ne concentrăm pe contextul de randare 2D

```
const canvas = document.getElementById("tutorial");  
const ctx = canvas.getContext("2d");
```

TEMPLATE DE BAZĂ

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <style>
      canvas {
        border: 5px solid pink;
      }
    </style>
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");
        }
      }
    </script>
  </head>
  <body>
    <canvas id="tutorial" width="150"
```



TEMPLATE DE BAZĂ

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");

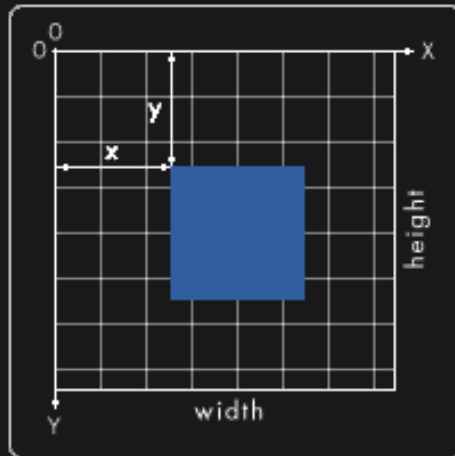
          ctx.fillStyle = "rgb(200, 0, 0)";
          ctx.fillRect(10, 10, 50, 50);

          ctx.fillStyle = "rgba(0, 0, 200,
0.5)";
          ctx.fillRect(30, 30, 50, 50);
        }
      }
    </script>
  </head>
```



GRID - SPAȚIU DE COORDONATE

- o unitate în grid corespunde unui pixel pe canvas
- originea gridului este în colțul din stânga sus, la coordonatele (0,0)



FIGURI PRIMITIVE

- rectangles
- paths

DREPTUNGHURI

```
fillRect(x, y, width, height)  
// desenează un dreptunghi plin
```

```
strokeRect(x, y, width, height)  
// desenează un contur de dreptunghi
```

```
clearRect(x, y, width, height)  
// șterge zona dreptunghiulară, făcând-o transparentă
```

DREPTUNGHIURI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");

          ctx.fillStyle = "pink";
          ctx.fillRect(25, 25, 100, 100);
          ctx.clearRect(45, 45, 60, 60);
          ctx.strokeStyle = "pink";
          ctx.strokeRect(50, 50, 50, 50);
        }
      }
    </script>
  </head>
  <body>
```



PATHS

- liste de puncte conectate de segmente de linii care pot fi curbate, pot avea grosimi și culori diferite
- un path poate fi închis cu `closePath()`
- pentru a crea figuri folosind paths:
 1. creăm path-ul cu `beginPath()`
 2. folosim comenzi de desenat în path
 3. adăugăm contur cu `stroke()` ori umplem path-ul cu `fill()` pentru a îl randa

PATHS

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");

          ctx.beginPath();
          ctx.moveTo(75, 50);
          ctx.lineTo(100, 75);
          ctx.lineTo(100, 25);
          ctx.fillStyle = "pink";
          ctx.fill();
        }
      }
    </script>
  </head>
```



LINII

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");

          ctx.beginPath();
          ctx.moveTo(25, 25);
          ctx.lineTo(105, 25);
          ctx.lineTo(25, 105);
          ctx.fillStyle = "pink";
          ctx.fill();

          ctx.beginPath();
          ctx.moveTo(125, 125);
          ctx.lineTo(125, 45);
```



ARCE DE CERC

```
arc(x, y, radius, startAngle, endAngle, counterclockwise)  
/* arc centrat în (x, y) cu raza r  
   începând de la startAngle și până la endAngle,  
   în direcția dată de counterclockwise */
```

```
arcTo(x1, y1, x2, y2, radius)  
/* arc cu punctele de control și raza date,  
   conectat la punctul anterior printr-o linie dreaptă */
```

ARCE DE CERC

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");
          ctx.fillStyle = "pink";
          ctx.strokeStyle = "pink";

          for (let i = 0; i < 4; i++) {
            for (let j = 0; j < 3; j++) {
              ctx.beginPath();
              const x = 25 + j * 50; // x
coordinate
              const y = 25 + i * 50; // y
coordinate
              const radius = 20; // arc
```



CURBE BÉZIER

```
quadraticCurveTo(cp1x, cp1y, x, y)  
/* curbă Bézier cuadratică de la poziția curentă  
   la punctul specificat de x și y,  
   folosind punctul de control (cp1x, cp1y) */
```

```
bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)  
/* curbă Bézier cubică de la poziția curentă  
   la punctul specificat de x și y,  
   folosind punctele de control  
   (cp1x, cp1y) și (cp2x, cp2y) */
```

[citește mai multe despre curbe Bézier](#)

CURBE BÉZIER

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");
          ctx.fillStyle = "pink";

          ctx.beginPath();
          ctx.moveTo(75, 40);
          ctx.bezierCurveTo(75, 37, 70, 25,
50, 25);
          ctx.bezierCurveTo(20, 25, 20, 62.5,
20, 62.5);
          ctx.bezierCurveTo(20, 80, 40, 102,
75, 120);
          ctx.bezierCurveTo(110, 102, 130,
```



OBIECTE PATH2D

- folosim obiecte **Path2D** pentru a simplifica codul și îmbunătăți performanța, înregistrând comenzile de desen
- putem folosi toate metodele de path de mai devreme (moveTo, rect, arc etc.)
- putem combina path-uri:

```
Path2D.addPath(path [, transform])  
/* adaugă un path la path-ul curent  
   cu o matrice de transformare opțională */
```

OBIECTE PATH2D

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        draw();
      }

      function draw() {
        const canvas =
document.getElementById("tutorial");
        if (canvas.getContext) {
          const ctx = canvas.getContext("2d");
          ctx.fillStyle = "pink";
          ctx.strokeStyle = "pink";

          const rectangle = new Path2D();
          rectangle.rect(10, 10, 50, 50);

          const circle = new Path2D();
          circle.arc(100, 35, 25, 0, 2 *
Math.PI);

          ctx.stroke(rectangle);
```



STILURI ȘI CULORI

- culori

```
fillStyle = color;  
strokeStyle = color;
```

- transparență

```
globalAlpha = transparencyValue; // valori în [
```

STILURI ȘI CULORI

- stiluri linie

```
lineWidth = value;  
lineCap = type; // butt, round, square  
lineJoin = type; // round, bevel, miter  
  
getLineDash(); // patternul de linie dash curent  
setLineDash(segments); // setează patternul de linie dash  
lineDashOffset = value; // unde să înceapă patternul dash
```


STILURI ȘI CULORI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        const ctx =
document.getElementById("canvas").getContext("
2d");
        ctx.strokeStyle = "pink";
        let offset = 0;

        function draw() {
          ctx.clearRect(0, 0, canvas.width,
canvas.height);
          ctx.setLineDash([4, 2]);
          ctx.lineDashOffset = -offset;
          ctx.strokeRect(10, 10, 100, 100);
        }

        function march() {
          offset++;
          if (offset > 16) {
            offset = 0;
          }
        }
      }
    </script>
  </head>
  <body>
    <canvas id="canvas" width="300px" height="150px">
  </body>
</html>
```



STILURI ȘI CULORI

- gradienti

```
createLinearGradient(x1, y1, x2, y2)
```

```
// gradient liniar de la (x1,y1) la (x2,y2)
```

```
createRadialGradient(x1, y1, r1, x2, y2, r2)
```

```
/* gradient radial: un cerc cu centrul în (x1,y1) și raza r1  
și unul cu centrul în (x2, y2) și raza r2 */
```

```
createConicGradient(angle, x, y)
```

```
/* gradient conic cu unghi de angle radiani  
la poziția (x, y) */
```

STILURI ȘI CULORI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        const ctx =
document.getElementById("canvas").getContext("
2d");

        // Create gradient
        const lingrad =
ctx.createLinearGradient(0, 0, 0, 150);
        lingrad.addColorStop(0, "#00ABEB");
        lingrad.addColorStop(0.95, "#fff");

        ctx.fillStyle = lingrad;
        ctx.fillRect(10, 10, 130, 130);
      }
    </script>
  </head>
  <body>
    <canvas id="canvas" width="150"
height="200"></canvas>
  </body>
```



STILURI ȘI CULORI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {
        const ctx =
document.getElementById("canvas").getContext("
2d");

        // Create gradient
        const radgrad =
ctx.createRadialGradient(45, 45, 10, 52, 50,
30);
        radgrad.addColorStop(0, "#A7D30C");
        radgrad.addColorStop(0.9, "#019F62");
        radgrad.addColorStop(1, "rgba(1, 159,
98, 0)");

        ctx.fillStyle = radgrad;
        ctx.fillRect(0, 0, 150, 150);

      }
    </script>
  </head>
```



STILURI ȘI CULORI

- pattern

```
createPattern(image, type)  
/* image: sursa unei imagini  
   type: repeat, repeat-x, repeat-y, no-repeat */
```

STILURI ȘI CULORI

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {

        function draw() {
          const ctx =
document.getElementById("canvas").getContext("
2d");

          // create new image object to use as
pattern
          const img = new Image();
          img.src = "pattern.webp";
          img.onload = () => {
            // create pattern
            const ptrn = ctx.createPattern(img,
"repeat");
            ctx.fillStyle = ptrn;
            ctx.fillRect(0, 0, 400, 600);
          };
        }
        draw();
      }
    </script>
  </head>
  <body>
    <canvas id="canvas" width="400px" height="600px">
    </canvas>
  </body>
</html>
```



STILURI ȘI CULORI

- umbre

```
shadowOffsetX = float;  
// distanța orizontală de la obiect  
shadowOffsetY = float;  
// distanța verticală de la obiect  
shadowBlur = float;  
shadowColor = color;
```

TEXT

```
fillText(text, x, y [, maxWidth]);  
strokeText(text, x, y [, maxWidth]);
```

```
font = value; // default: 10px sans-serif  
textAlign = value; // start, end, left, right, center  
textBaseline = value; /* top, hanging, middle,  
                        alphabetic, ideographic, bottom */  
direction = value; // ltr, rtl, inherit
```


TEXT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {

        function draw() {
          const ctx =
document.getElementById("canvas").getContext("
2d");
          ctx.font = "48px serif";
          ctx.textBaseline = "hanging";
          ctx.fillStyle = "pink";
          ctx.fillText("Hakuna Matata!", 10,
50);
        }
        draw();
      }
    </script>
  </head>
  <body>
    <canvas id="canvas" width="400"
height="300"></canvas>
  </body>
```

>

IMAGINI

```
const img = new Image(); // crează element img nou
img.addEventListener("load", () => {
    // execute drawImage statements here
}, false);
img.src = "myImage.png"; // source path
```

```
drawImage(image, x, y);
// desenează o imagine la poziția (x,y)
drawImage(image, x, y, width, height);
// + scalare
drawImage(image, sx, sy, sWidth, sHeight, dx, dy, dWidth, dHeight);
/* ia din imaginea sursă un dreptunghi cu colțul din stânga
sus la (sx, sy) și cu lățimea sWidth și înălțimea sHeight
și o desenează în canvas la poziția (dx, dy), scalând-o */
```

CANVAS STATE

```
save(); // salvează starea canvasului  
restore(); // restaurează cel mai recent salvată stare
```

o stare de desen a canvasului constă din

- valorile atributelor: `strokeStyle`, `fillStyle`, `globalAlpha`, `lineWidth`, `lineDashOffset`, `shadowOffsetX`, `font` etc.
- transformările care au fost aplicate: `translate`, `rotate`, `scale`
- clipping path curent

TRANSFORMĂRI

```
translate(x, y);  
// mișcă canvasul și originea pe grid cu distanțele x și y  
rotate(angle); // rotirea canvasului în jurul originii  
scale(x, y); /* scalarea canvasului cu x și y unități;  
              valori nr. reale */  
transform(a, b, c, d, e, f);  
/* înmulțește matricea de transformare curentă cu  
   matricea dată prin argumente */
```

COMPOSITE

```
globalCompositeOperation = type;  
// tipul de compunere la desenarea unor figuri noi
```

- source-over, source-in, source-out
- destination-over, destination-in, destination-out
- lighter, copy, xor, multiply, screen, overlay, darken, lighten, color-dodge, color-burn, hard-light, soft-light
- difference, exclusion, hue, saturation, color, luminosity

CLIPPING

clipping path: mascarea unei zone din canvas

```
clip(); // path-ul construit devine clipping path-ul curent
```

- folosim clip() în loc de closePath() pentru a închide un path și a-l transforma într-o mască în loc să îl umplem (fill) sau conturăm (stroke)

CLIPPING

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {

        function draw() {
          const ctx =
document.getElementById("canvas").getContext("
2d");

          ctx.fillRect(0, 0, 150, 150);
          ctx.translate(75, 75);

          // Create a circular clipping path
          ctx.beginPath();
          ctx.arc(0, 0, 60, 0, Math.PI * 2,
true);
          ctx.clip();

          // draw background
          const lingrad =
ctx.createLinearGradient(0, -75, 0, 75);
          lingrad.addColorStop(0, "#232256");
          lingrad.addColorStop(1, "#143778");
```



ANIMAȚII

pași pentru desenarea unui frame:

1. clear canvas: putem folosi `clearRect()`
2. salvarea stării canvasului
3. desenarea figurilor de animat
4. restaurarea stării canvasului

ANIMAȚII

update-uri programate:

```
setInterval();  
setTimeout();  
requestAnimationFrame(callback);  
/* cere browserului să apeleze funcția callback pentru a  
   actualiza o animație înainte de următoarea redesenare */
```

ANIMAȚII

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
    <script>
      window.onload = function() {

        function clock() {
          const now = new Date();
          const canvas =
document.getElementById("canvas");
          const ctx = canvas.getContext("2d");
          ctx.save();
          ctx.clearRect(0, 0, 150, 150);
          ctx.translate(75, 75);
          ctx.scale(0.4, 0.4);
          ctx.rotate(-Math.PI / 2);
          ctx.strokeStyle = "black";
          ctx.fillStyle = "white";
          ctx.fill();
          ctx.lineWidth = 8;
          ctx.lineCap = "round";

          // Hour marks
          ctx.save();
```



OPTIMIZĂRI



SVG

Scalable Vector Graphics

SVG

- limbaj de marcare bazat pe XML pentru descrierea de elemente grafice vectoriale 2D
- standard Web open bazat pe text, dezvoltat de W3C din 1999
- imaginile SVG sunt definite în fișiere XML și pot fi randate la orice dimensiune fără pierderea calității

SVG

- suportat de toate browserele principale
- bine integrat cu CSS, DOM, JavaScript
- interfață DOM; nu necesită extensii third-party
- dezvantaj: încărcarea unei imagini SVG poate fi lentă
- fișierele SVG pot fi create și cu aplicații de desenat vectoriale precum [Inkscape](#)

INGREDIENTE PRINCIPALE

- un document SVG constă din rădăcina `<svg>` și elemente pentru figuri de bază
- elemente pentru figuri geometrice (cercuri, dreptunghiuri) și curbe (simple și complexe)
- figurile pot fi grupate folosind elementul `<g>`
- SVG permite definirea de gradienti, rotații, efecte de filtrare, animații și interactivitatea cu JavaScript

SINTAXĂ

- fiind bazat pe XML, SVG e case-sensitive: atenție la majuscule în cazul elementelor și al atributelor
- valorile atributelor se scriu între ghilimele, chiar și cele numerice

EXEMPLU

```
<svg version="1.1" width="300" height="200" xmlns="http://www.  
  <rect width="100%" height="100%" fill="pink"></rect>  
  <circle cx="150" cy="100" r="80" fill="yellow"></circle>  
  <text x="150" y="125" font-size="60" text-anchor="middle"  
</svg>
```

EXEMPLU

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg version="1.1"
      width="300" height="200"
      xmlns="http://www.w3.org/2000/svg">

      <rect width="100%" height="100%"
fill="pink" />
      <circle cx="150" cy="100" r="80"
fill="yellow" />
      <text x="150" y="125" font-size="60"
text-anchor="middle" fill="black">SVG</text>
    </svg>
  </body>
</html>
```



PROPRIETĂȚI

- ordinea randării elementelor: elementele noi sunt adăugate peste elementele mai vechi
- codul SVG poate fi adăugat direct în pagini HTML

```

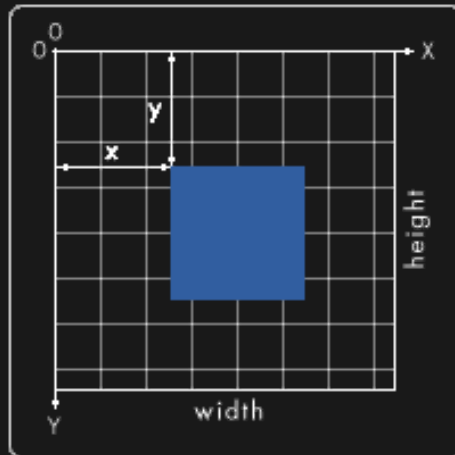
```

```
<iframe src="image.svg"></iframe>
```

```
<object data="image.svg" type="image/svg+xml"></object>
```

GRID - SPAȚIU DE COORDONATE

- o unitate în grid corespunde unui pixel
- originea gridului este în colțul din stânga sus, la coordonatele (0,0)
- pixeli? scalare?



FIGURI: RECT

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="200" height="250" version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <rect x="10" y="10" width="100"
height="100" stroke="white" fill="transparent"
stroke-width="5"/>
      <rect x="60" y="100" rx="10" ry="10"
width="75" height="75" stroke="pink"
fill="transparent" stroke-width="5"/>
    </svg>
  </body>
</html>
```



FIGURI: CIRCLE

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="200" height="250" version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <circle cx="100" cy="75" r="50"
stroke="pink" fill="transparent" stroke-
width="5"/>
      <ellipse cx="150" cy="75" rx="30" ry="15"
stroke="red" fill="transparent" stroke-
width="5"/>
    </svg>
  </body>
</html>
```



FIGURI: LINE

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="200" height="250" version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <line x1="10" x2="50" y1="50" y2="90"
stroke="orange" stroke-width="5"/>
      <polyline points="60 110 65 120 70 115 75
130 80 125 85 140 90 135 95 150 100 145"
stroke="yellow" fill="transparent" stroke-
width="5"/>
    </svg>
  </body>
</html>
```

>

FIGURI: POLYGON

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="200" height="250" version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <polygon points="50 60 55 80 70 80 60 90
65 105 50 95 35 105 40 90 30 80 45 80"
stroke="yellow" fill="transparent" stroke-
width="5"/>
    </svg>
  </body>
</html>
```

>

PATHS

- elementul <path> - forma pathului este dată de atributul "d" ce conține o serie de comenzi (specificate prin câte o literă) și parametri pentru acele comenzi
- fiecare comandă are două variante: o literă majusculă pentru coordonate absolute în pagină, și una minusculă pentru coordonate relative

PATHS

comenzi pentru linii:

- Move To: M x y
- Line To: L x y
- Horizontal Line: H x. Vertical Line: V y
- Close Path: Z

PATHS

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="200" height="250" version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <path d="M 10 10 h 80 v 80 h -80 Z"
fill="transparent" stroke="white"/>
    </svg>
  </body>
</html>
```



PATHS

comenzi pentru curbe:

- Curbe Béziere cubice: C x1 y1, x2 y2, x y
- Curbe Béziere cuadratice: Q x1 y1, x y
- Arce de cerc:

A rx ry x-axis-rotation large-arc-flag sweep-flag x y

FILLS & STROKES

folosind attributele

- *fill* și *stroke*
- *stroke-width*, *stroke-linecap*, *stroke-linejoin*, *stroke-dasharray*, *stroke-dashoffset*

GRADIENȚI

folosind elementele

- `<linearGradient>` împreună cu noduri `<stop>` și attributele *offset*, *stop-color*, *stop-opacity*
- `<radialGradient>` împreună cu noduri `<stop>` și attribute pentru setarea **punctelor de centru și focale**
- se adaugă într-un element `<defs>` și trebuie să aibă un atribut *id*

PATHS

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="demo.css">
  </head>
  <body>
    <svg width="120" height="240"
version="1.1"
xmlns="http://www.w3.org/2000/svg">
      <defs>
        <radialGradient id="RadialGradient1">
          <stop offset="0%" stop-color="red" />
          <stop offset="100%" stop-color="yellow"
/>
        </radialGradient>
        <radialGradient id="RadialGradient2"
cx="0.25" cy="0.25" r="0.25">
          <stop offset="0%" stop-color="red" />
          <stop offset="100%" stop-color="yellow"
/>
        </radialGradient>
      </defs>
```

>

ALTE ELEMENTE

- patterns
- text
- clip paths și masks
- imagini

TRANSFORMĂRI

atributul *transform* cu valorile:

- `translate()`
- `rotate()`
- `skewX()` și `skewY()`
- `scale()`
- transformări complexe cu `matrix()`

CANVAS VS. SVG

```

      WW
    /  _  \
   |  oo  |
  (| _ ( ) _ |) /  _  o o  \
W"          "W  \  _  / (|  _  o  _  |)
W  -=====  W  / | \ / | \  \  _  /  /
    "W          W" | | | | | | | /-----
W" " " " " " " " "W | | | | | | | ===== |
W                      W | | | | | | | ===== |

```

întrebări?

TEHNICI WEB

WEB FOR ALL

Claudia Chiriță . 2024/2025

CAN I USE...?

I18N



Internationalization

I18N

- adaptarea unei pagini web la mai multe limbi, diferențe regionale și cerințe tehnice pentru zone geografice și țări diferite
- procesul de design și proiectare a unei aplicații astfel încât să poată fi ușor și rapid adaptată la mai multe limbi și regiuni

I18N

- utilizatorul trebuie să poată naviga prin pagină în vederea traducerii conținutului fără să afecteze layoutul paginii
- permiterea mii multor seturi de caractere (via Unicode), unități de măsură (monedă, °C/°F, km/mile, etc.), formate de dată și timp, layouturi de tastatură, layouturi și direcție a textului

I18N - CE?

- ghid i18n
- specificație i18n

I18N - CUM?

- Internationalization API
- internationalization library

I18N - AŞA?

- i18n checker

A11Y



Accessibility

A11Y



Accessibility

A11Y



Accessibility

A11Y

- practica dezvoltării de pagini web ce pot fi folosite de cât mai mulți oameni
- adaptarea designului și conținutului pentru grupuri diverse de utilizatori (cu dizabilități, care folosesc dispozitive mobile, au conexiuni lente la Internet)
- nu ar trebui să fie un "afterthought", ci un principiu de dezvoltare
- "it's the law!"

A11Y - CUM?

- principii a11y
- tutoriale a11y

A11Y - AŞA?

- a11y checker
- a11y evaluation tools

A11Y - EXEMPLU

AI-AI, BAD UI!

decalog

decaflop

contrast checker

HAPPY SURFING!

broken-links checker



```

      WW
    /  _  \
   |  oo  |
  ( | _ ( ) _ | ) /  _  o o  \
W"          "W  \  _  / ( |  _  o  _  | )
W  -=====  W  / | \ / | \  \  _  /  /
    "W          W" | | | | | | | /-----
W" " " " " " " " "W | | | | | | | ===== |
W                      W | | | | | | | ===== |

```

întrebări?

TEHNICI WEB EXTRAS

Claudia Chiriță . 2024/2025

SASS

SASS



tutorial

SCSS

TAILWIND

[tell me more](#)

BOOTSTRAP

tell me more

tutorial

TYPESCRIPT

tell me more

tutorial

STATIC PAGE GENERATORS

generatoare de pagini statice

GitHub pages

WEBGL

tutorial

SHADERS

Shader Park (bibliotecă)



